VERIQTA

- ✓ Automate.
- ✓ Orchestrate.
- ✓ Deploy.
- ✓ Monitor.
- ✓ Optimize.

That's DevOps.

PYTHON FOR DEVOPS HANDBOOK

Code.
Automate.
Scale.
Repeat.

ENGINEER THE FUTURE.

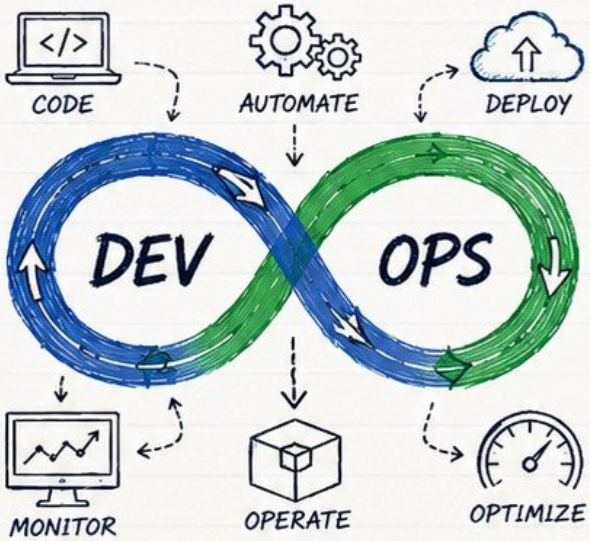
Build Intelligent Automation. Manage Infrastructure at Scale.
Ship Faster. Operate Smarter.

60 PAGES

COMPLETE HANDBOOK CURRICULUM

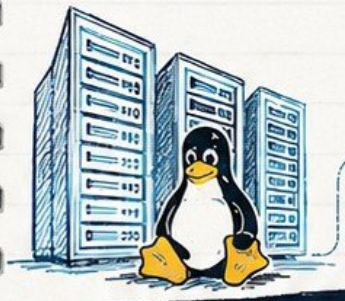
COVERING:

- ✓ Automation
- ✓ Cloud
- ✓ Linux
- ✓ Kubernetes
- ✓ CI/CD
- ✓ Monitoring
- ✓ APIs
- ✓ Security
- ✓ And Much More...




```
$ python automate.py
> Connecting...
> Authenticating...
> Executing Tasks...
> Success ✅
```

Write Once.
Automate Anywhere.
Scale Everywhere.



Automate Everything. Orchestrate Anything. Scale Limitlessly.

INTRODUCTION TO PYTHON FOR DEVOPS

Code is not just about syntax, it's about solving real-world problems at scale.

WHY DEVOPS ENGINEERS SHOULD LEARN PYTHON

- Automate repetitive and error-prone tasks.
- Integrate tools, APIs, and services across your stack.
- Build internal tools that accelerate engineering productivity.
- Improve reliability, consistency and speed of operations.
- Python is simple to learn but powerful to master.

AUTOMATION MINDSET

- ✓ Think in systems, not in manual steps.
- ✓ Identify repeatable tasks.
- ✓ Automate safely and idempotently.
- ✓ Build with scalability and reusability in mind.
- ✓ Automate for humans, not to replace humans.

PRODUCTION NOTE

The goal of automation is not just to save time, but to reduce risk, eliminate human error, and bring consistency across environments.

PYTHON IN MODERN INFRASTRUCTURE

- ☑ Cloud Automation (AWS, Azure, GCP)
- ☑ Infrastructure as Code (IaC) automation
- ☑ CI/CD pipeline orchestration
- ☑ Monitoring, alerting & observability
- ☑ Configuration & system management
- ☑ Kubernetes & container operations
- ☑ Security, compliance & auditing
- ☑ AI-driven operations & intelligent automation

WHERE PYTHON FITS IN THE DEVOPS LIFECYCLE

PLAN	CODE	BUILD	TEST	DEPLOY	OPERATE	MONITOR
Automate planning, resource estimation, reporting	Linting, code gen, scaffolding automation	Build scripts, dependencies, artifact management	Test automation, environment validation	Release pipelines, config management, IaC	Infrastructure automation, self-healing, scaling, backup	Metrics collection, alerting, incident response

Python connects and automates everything.

HANDBOOK OBJECTIVES

1. Build a strong foundation in Python for real-world DevOps.
2. Automate Linux administration, CI/CD workflows, and cloud operations.
3. Master APIs, Infrastructure as Code, and container orchestration with Python.
4. Build production-ready automation frameworks and internal tools.
5. Apply Python for monitoring, security, reliability, and AI-driven operations.
6. Think like a Platform Engineer. Build scalable, secure and maintainable automation.

REMEMBER

The best DevOps engineers are not just tool users, they are automation builders and problem solvers.

REAL WORLD IMPACT

Python turns manual operations into reliable, repeatable, and scalable systems that drive business value.

SENIOR ENGINEER TIP

Start small. Automate one task. Do it well. Then scale it.

VERIQTA

SETTING UP YOUR DEVELOPMENT ENVIRONMENT

A clean and consistent environment is the foundation of every reliable automation project.

BEST PRACTICE

Keep your tools updated. Use virtual environments for every project.

1 INSTALLING PYTHON

→ Go to https://www.python.org/downloads
→ Download the latest Python 3.x
→ Run the installer

IMPORTANT: Check
☑ Add Python to PATH
before clicking Install Now.

VERIFY INSTALLATION

```
$ python --version
Python 3.12.3
$ python -m pip --version
pip 24.0 from ... (python 3.12)
```

2 INSTALLING VISUAL STUDIO CODE

→ Go to https://code.visualstudio.com
→ Download for your OS
→ Install and open VS Code
→ Recommended Extensions:

- Python
- Pylance
- GitLens
- Prettier
- Code Runner

TIP

VS Code + Python extension gives you linting, IntelliSense, debugging and terminal all in one place.

3 INSTALLING GIT

→ Go to https://git-scm.com/downloads
→ Download and install Git
→ Use default options for most users

VERIFY INSTALLATION

```
$ git --version
git version 2.45.1
```

PRODUCTION NOTE

Git is essential for version control, collaboration, and storing automation code safely.

4 INSTALLING PIP

pip usually comes with Python.
If not, install it:

```
$ python -m ensurepip --upgrade
$ python -m pip install --upgrade pip
```

VERIFY

```
$ pip --version
pip 24.0 from ... (python 3.12)
```

5 CREATING YOUR FIRST PYTHON WORKSPACE

→ Choose a base directory for your projects.
→ Create a new folder for your first project.

Example (Linux/macOS):

```
$ mkdir -p ~/devops-python
$ cd ~/devops-python
$ code .    # open in VS Code
```

WINDOWS (PowerShell)

```
PS> mkdir $env:USERPROFILE\devops-python
PS> cd $env:USERPROFILE\devops-python
PS> code .
```

PROJECT STRUCTURE (EXAMPLE)

```
devops-python/
├── scripts/      # automation scripts
├── modules/      # reusable modules
├── config/       # configuration files
├── tests/        # test scripts
└── README.md     # project docs
```

6 RUNNING YOUR FIRST SCRIPT

① Create a new file: hello.py

```
# hello.py
print("Hello, DevOps Engineer! 🎉")
print("Your automation journey starts now.")
```

② Run the script

```
$ python hello.py
```

OUTPUT

```
$ python hello.py
Hello, DevOps Engineer! 🎉
Your automation journey starts now.
$
```

REMEMBER

- ☑ Keep Python updated
- ☑ Use virtual environments
- ☑ Commit your code often
- ☑ Automate everything

Good environment.
Great automation.
Awesome impact.

VERIQTA

PYTHON FUNDAMENTALS

Master the basics. Build anything.

REMEMBER
Strong fundamentals make advanced automation easy and reliable.

1 VARIABLES

Variables store values in memory.

```
name = "DevOps Engineer"
age = 28
is_engineer = True
experience_years = 4.5
tools = ["Linux", "Docker", "Kubernates"]
```

Give data a name so you can use it later.

2 DATA TYPES

Python has built-in data types.

TYPE	EXAMPLE	DESCRIPTION
str	"hello"	Text / String
int	25	Whole numbers
float	3.14	Decimal numbers
bool	True / False	Boolean (True or False)
list	[1, 2, 3]	Ordered, changeable collection
tuple	(1, 2, 3)	Ordered, unchangeable collection
dict	{"key": "value"}	Key-value pairs
set	{1, 2, 3}	Unordered collection of unique items

3 COMMENTS

Comments help explain your code.

```
# This is a single-line comment
age = 28  # Inline comment
"""
This is a
multi-line comment.
Use it for longer explanations.
"""
```

Good comments make future you happy!

4 PRINTING OUTPUT

Use `print()` to display output.

```
print("Hello, DevOps!")
print(25)
print(name)
print(f"Engineer: {name}, Age: {age}")
```

OUTPUT

```
$ Hello, DevOps!
25
DevOps Engineer
Engineer: DevOps Engineer, Age: 28
$
```

5 INPUT FROM USERS

Use `input()` to get data from users.

```
name = input("What is your name? ")
age = input("How old are you? ")
experience = input("Years of experience? ")
print(f"Hello {name}! You have {experience} years of experience.")
```

EXAMPLE RUN

```
$ python app.py
What is your name? Ann
How old are you? 28
Years of experience? 4
Hello Ann! You have 4 years of experience.
$
```

IMPORTANT
`input()` always returns a string.
Convert it using `int()` or `float()` if needed.

6 NAMING CONVENTIONS

Good names make code easy to read and maintain.

GOOD NAMES
- user_name
- server_ip_address
- is_service_running
- total_cpu_usage
- backup_file_path

AVOID THESE
- a, b, c
- name1, name2
- x1, x2, val
- Temp
- dataa

FOLLOW THESE RULES
- Use lowercase letters
- Use underscores to separate words
- Start with a letter or underscore (_)
- Do not use spaces or special characters
- Use meaningful, descriptive names
- Be consistent

SENIOR ENGINEER TIP
Readable code > Clever code.
Write code that your team can understand in 6 months, not just your future self.

PRODUCTION NOTE
Strong fundamentals reduce bugs, improve automation reliability, and save hours of troubleshooting in production.

Learn. Automate. Optimize. Ship Reliable Systems.

OPERATORS AND EXPRESSIONS

Operators perform operations on values and variables.
Expressions combine values, variables and operators to produce a result.

REMEMBER
Operators are the building blocks of logic and automation. Use them wisely.

1 ARITHMETIC OPERATORS

Used for mathematical operations.

OPERATOR	DESCRIPTION	EXAMPLE	RESULT
+	Addition	a + b	15
-	Subtraction	a - b	5
*	Multiplication	a * b	50
/	Division	a / b	2.5
//	Floor Division	a // b	2
%	Modulus	a % b	0
**	Exponentiation	a ** b	100

EXAMPLE:

```
a = 10   b = 4
print(a + b, a - b, a * b, a / b)     # 14 6 40 2.5
print(a // b, a % b, a ** b)          # 2 2 10000
```

2 COMPARISON OPERATORS

Used to compare two values.

OPERATOR	DESCRIPTION	EXAMPLE	RESULT
==	Equal to	a == b	False
!=	Not equal to	a != b	True
>	Greater than	a > b	True
<	Less than	a < b	False
>=	Greater than or equal	a >= b	True
<=	Less than or equal	a <= b	False

EXAMPLE:

```
a = 10   b = 4
print(a == b, a != b, a > b)     # False True True
print(a < b, a >= b, a <= b)     # False True False
```

3 LOGICAL OPERATORS

Used to combine conditional statements.

OPERATOR	DESCRIPTION	EXAMPLE	RESULT
and	True if both are True	a > 5 and b < 6	True
or	True if at least one is True	a < 5 or b < 6	True
not	Reverse the result	not (a == b)	True

EXAMPLE:

```
a = 10   b = 4
print(a > 5 and b < 6)     # True
print(a < 5 or b < 6)      # True
print(not (a == b))        # True
```

Logical operators are the brain of automation.

4 ASSIGNMENT OPERATORS

Used to assign values to variables.

OPERATOR	DESCRIPTION	EXAMPLE	EQUIVALENT TO
=	Assign	a = 10	a = 10
+=	Add and assign	a += 5	a = a + 5
-=	Subtract and assign	a -= 3	a = a - 3
*=	Multiply and assign	a *= 2	a = a * 2
/=	Divide and assign	a /= 2	a = a / 2
%=	Modulus and assign	a %= 3	a = a % 3
**=	Exponent and assign	a **= 2	a = a ** 2
//=	Floor divide and assign	a //= 2	a = a // 2

EXAMPLE:

```
a = 10      # 15        a %= 4      # 2
a += 5      # 12        a **= 3     # 8
a *= 2      # 24        a //= 3     # 2
a /= 4      # 6.0
```

5 MEMBERSHIP OPERATORS

Used to test membership in a sequence (like list, string, set, dict).

OPERATOR	DESCRIPTION	EXAMPLE	RESULT
in	Returns True if value is found	'DevOps' in skills	True
not in	Returns True if value is NOT found	'Java' not in skills	True

EXAMPLE:

```
skills = ['Python', 'Linux', 'Kubernetes']
print('Python' in skills)      # True
print('Go' not in skills)      # True
```

6 IDENTITY OPERATORS

Used to compare the memory location of objects.

OPERATOR	DESCRIPTION	EXAMPLE	RESULT
is	True if both variables refer to same object	a is b	True / False
is not	True if both variables refer to different objects	a is not b	True / False

EXAMPLE:

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)        # True
print(a is c)        # False
print(a is not c)    # True
```

NOTE
For immutable types (int, str, tuple), Python may optimize and reuse objects.

KEY TAKEAWAY
Understanding operators is essential for writing powerful conditions, loops, expressions and automation logic.

PRODUCTION TIP
Use parentheses to make expressions clear and avoid surprises.

```
result = (a + b) * (c - d) / e
```

VERIQTA

WORKING WITH STRINGS

Strings are immutable sequences of characters.
Python provides powerful tools to manipulate and format text.

REMEMBER

Strings are everywhere in DevOps: logs, configs, APIs, shell output, file paths, templates and user input.

1 CREATING STRINGS

Strings can be created using single quotes, double quotes or triple quotes.

```
s1 = 'hello'
s2 = "hello"
s3 = """This is
      a multi-line
      string"""
s4 = str(123)     # convert to string
s5 = 'Dev' + 'Ops'
   # concatenation -> 'DevOps'
```

Both single and double quotes work the same.

2 STRING METHODS (COMMON & USEFUL)

METHOD	DESCRIPTION	EXAMPLE	RESULT
lower()	Convert to lowercase	'DevOps'.lower()	'devops'
upper()	Convert to uppercase	'devops'.upper()	'DEVOPS'
strip()	Remove spaces	' hi '.strip()	'hi'
lstrip()	Remove left spaces	' hi '.lstrip()	'hi '
rstrip()	Remove right spaces	' hi '.rstrip()	' hi'
replace()	Replace substring	'a-b-c'.replace('-', '/')	'a/b/c'
split()	Split by separator	'a,b,c'.split(',')	['a','b','c']
join()	Join items	'-'.join(['a','b','c'])	'a-b-c'
find()	Find substring index	'hello'.find('e')	1
count()	Count occurrences	'banana'.count('a')	3
startswith()	Check prefix	'hello'.startswith('he')	True
endswith()	Check suffix	'config.yaml'.endswith('.yaml')	True
len()	Length of string	len('Python')	6

3 FORMATTING STRINGS (OLD STYLE)

Using % operator or str.format().

Using %

```
name = 'Ann'
age = 27
msg = 'My name is %s and I am %d years old.' % (name, age)
print(msg)
# My name is Ann and I am 27 years old.
```

Using str.format()

```
name = 'Ann'
age = 27
msg = 'My name is {} and I am {} years old.'.format(name, age)
print(msg)
# My name is Ann and I am 27 years old.
```

Still useful when working with legacy code or older Python projects.

4 F-STRINGS (PYTHON 3.6+)

The modern, clean and fastest way to format strings.

```
name = 'Ann'
age = 27
role = 'DevOps Engineer'
msg = f'My name is {name}, I am {age} years old and I work as a {role}.'
print(msg)
# My name is Ann, I am 27 years old and I work as a DevOps Engineer.
```

Readable. Fast. Powerful. Use it always!

5 MULTI-LINE STRINGS

Use triple single or triple double quotes.

```
config = """
app:
  name: veriqta-app
  env: production
  replicas: 3
  database: postgres
"""
print(config)
```

Great for:

- ☑ Configs
- ☑ YAML / JSON
- ☑ Templates
- ☑ SQL queries
- ☑ Documentation

.yaml

6 PRODUCTION EXAMPLES

Parsing Shell Output

```
output = 'User: root\nDisk: 80%\nMem: 65%'
lines = output.split('\n')
disk = next(l.split(':  ')[1]
        for l in lines if l.startswith('Disk'))
print('Disk usage:', disk)
# Disk usage: 80%
```

Working with File Paths

```
path = '/var/log/nginx/access.log'
filename = path.split('/')[-1]
dirname = '/'.join(path.split('/')[:-1])
print(filename)  # access.log
print(dirname)   # /var/log/nginx
```

Building API Query String

```
params = {'q': 'python', 'page': 1}
query = '&'.join(f'{k}={v}' for k, v in params.items())
url = f'https://api.example.com/search?{query}'
print(url)
# https://api.example.com/search?q=python&page=1
```

Cleaning Log Lines

```
line = '   [ERROR] Failed to connect   \n'
clean = line.strip().replace('  ', ' ')
print(clean)
#     [ERROR] Failed to connect
```

KEY TAKEAWAYS

- Strings are immutable – operations return new strings.
- Use built-in methods to write clean and efficient code.
- Prefer f-strings for new projects.
- Mastering strings = mastering real-world automation.

BEST PRACTICE

Always validate and clean input strings from users or external systems.

AVOID

- Using + repeatedly in loops for concatenation (use join() instead).
- Using old-style formatting in new code.
- Hardcoding large multi-line strings in code.

VERIQTA

LISTS, TUPLES, SETS AND DICTIONARIES

Python provides powerful built-in collections.
Choose the right one for the job.

REMEMBER
The right data structure makes your code faster, cleaner and easier to maintain.

1 CHOOSING THE RIGHT COLLECTION

COLLECTION	ORDERED	MUTABLE	UNIQUE	LOOKUP SPEED	BEST FOR
List	Yes	Yes	No	O(n)	Ordered data, duplicates, frequent changes
Tuple	Yes	No	No	O(n)	Fixed data, constants, safe records
Set	No*	Yes	Yes	O(1) avg	Unique items, membership tests, math ops
Dict	Yes**	Yes	Keys unique	O(1) avg	Key-value data, configs, lookups, mappings

* Sets are unordered (no guaranteed order)
** Dicts preserve insertion order (Python 3.7+)

Pick the right tool for clean, efficient automation.

2 CRUD OPERATIONS

OPERATION	LIST	TUPLE	SET	DICT
Create	[]	()	set()	{key: value}
Read	lst[i]	tpl[i]	x in s	d[key]
Update	lst[i] = v	✗	s.add(x)	d[key] = v
Delete	del lst[i]	✗	s.remove(x)	del d[key]

NOTE
Tuples are immutable. Lists, Sets and Dicts are mutable.
Sets don't support indexing.

3 LISTS

Ordered, mutable collection.
Allows duplicates.

```
# Create
servers = ['web01', 'web02', 'db01']
# Read
first = servers[0]        # 'web01'
# Update
servers[1] = 'web03'
# Delete
del servers[2]        # remove 'db01'
# Add
servers.append('cache01')
# Iterate
for s in servers:
    print(s)
```

DEVOPS USE CASE
Keep an ordered list of servers, jobs, or steps in a pipeline.
Example: deployment stages.

```
stages = ['build', 'test', 'deploy']
```

4 TUPLES

Ordered, immutable collection.
Allows duplicates.

```
# Create
coords = (10.5, 20.3)
# Read
x = coords[0]          # 10.5
# Use
def get_ip_config():
    return ('10.0.0.1', 22)

# Unpacking
host, port = coords
```

DEVOPS USE CASE
Store fixed values that should not change.
Example: default ports, configuration pairs.

```
DEFAULT_SSH = ('0.0.0.0', 22)
```

5 SETS

Unordered, mutable collection.
Stores unique items only.

```
# Create
users = {'alice', 'bob', 'alice'}
# 'alice' appears once
# Add
users.add('charlie')
# Remove
users.remove('bob')
# Membership test
"alice" in users        # True
# Set operations
admins = {'alice', 'dave'}
users | admins      # union
users & admins      # intersection
users - admins      # difference
```

DEVOPS USE CASE
Find unique items quickly.
Example: unique users, unique IPs, unique services.

```
unique_ips = set(ip_list)
```

6 DICTIONARIES

Ordered key-value mapping.
Keys must be unique.

```
# Create
server = {'name': 'web01',
          'ip':  '10.0.0.10',
          'role': 'web'}
# Read
ip = server['ip']
# Update
server['role'] = 'api'
# Add
server['env'] = 'prod'
# Delete
del server['ip']
# Keys / Values
server.keys()     # dict_keys(...)
server.values()   # dict_values(...)
server.items()    # dict_items(...)
```

DEVOPS USE CASE
Store configurations, metadata, API responses, inventory data.
Example: server inventory.

```
Inventory = { 'web01': {'ip': '10.0.0.10',
                        'os': 'ubuntu'} }
```

PRACTICAL DEVOPS EXAMPLES

1. Parse a servers list and build inventory (Dict)

```
servers = [
   {'name': 'web01', 'ip': '10.0.0.10', 'role': 'web'},
   {'name': 'db01',  'ip': '10.0.0.20', 'role': 'db'},
]
inventory = {s['name']: s for s in servers}
print(inventory['web01']['ip'])   # 10.0.0.10
```

2. Find unreachable hosts (Set)

```
all_hosts = {'web01', 'web02', 'db01', 'cache01'}
reachable = {'web01', 'db01'}
unreachable = all_hosts - reachable
print(unreachable)    # {'web02', 'cache01'}
```

3. App configuration using Dict

```
config = {
   'app': 'veriqta-api',
   'debug': False,
   'retry': 3,
   'log_level': 'INFO'
}
timeout = config.get('timeout', 30)   # default 30
```

KEY TAKEAWAYS

- ✓ Lists: ordered, mutable, allow duplicates.
- ✓ Tuples: ordered, immutable, fixed data.
- ✓ Sets: unordered, unique items, fast membership.
- ✓ Dicts: key-value pairs, fast lookups, preserve order.

BEST PRACTICE

Use the simplest collection that solves the problem.
Simplicity = reliability.
Reliability = great automation.

SENIOR ENGINEER TIP

Good automation starts with good data structures. It makes your code scalable, maintainable and production-ready.

VERIQTA

CONDITIONAL STATEMENTS

Conditional statements allow your programs to make decisions.
Python uses indentation to define blocks of code.

REMEMBER
Good automation makes the right decision at the right time based on the right condition.

1 IF STATEMENT

Executes a block if the condition is True.

```
cpu_usage = 85
if cpu_usage > 80:
    print("CPU is high!")
    print("Investigate immediately.")
```

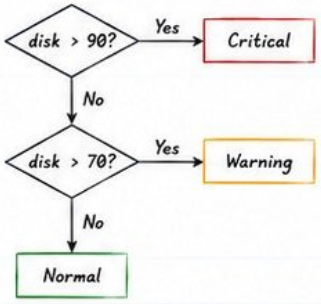
Only runs when the condition is True.

2 IF...ELIF...ELSE

Checks multiple conditions.

```
disk_usage = 45
if disk_usage > 90:
    print("Critical: Disk almost full!")
elif disk_usage > 70:
    print("Warning: Disk is getting full.")
else:
    print("Disk usage is normal.")
```

FLOW



3 NESTED CONDITIONS

Conditions inside other conditions.

```
service = 'nginx'
status = 'running'
if service == 'nginx':
    if status == 'running':
        print("Nginx is running ✅")
    else:
        print("Nginx is not running ❌")
else:
    print(f"Service {service} is not Nginx.")
```

TIP
Keep nested blocks as simple and shallow as possible. Complex nesting is hard to read and maintain.

4 PRODUCTION DECISION MAKING

Real-world automation decisions.

```
status_code = 503
if status_code >= 500:
    print("Server error! Escalate to On-call.")
elif status_code >= 400:
    print("Client error. Check request.")
elif status_code >= 300:
    print("Redirection. Verify endpoint.")
elif status_code == 200:
    print("Service is healthy ✅")
else:
    print("Unexpected status code.")
```

WHY THIS MATTERS

- ☑ Automates smart responses
- ☑ Reduces MTTR
- ☑ Enforces consistent behavior
- ☑ Prevents human errors

5 COMMON PATTERNS

CHECKING NONE

```
value = None
if value is None:
    print("No value found.")
```

CHECKING MEMBERSHIP

```
user = 'admin'
if user in ['admin', 'devops']:
    print("Access granted.")
else:
    print("Access denied.")
```

RANGE CHECK

```
limit = 5
retry = 3
if retry < limit:
    print("Retry allowed.")
else:
    print("Max retry reached.")
```

MULTIPLE CONDITIONS

```
cpu = 70
mem = 60
if cpu > 80 and mem > 80:
    print("High CPU and Memory!")
elif cpu > 80 or mem > 80:
    print("One resource is high.")
else:
    print("All resources are normal.")
```

COMPLEX CONDITION

```
env = 'prod'
user = 'admin'
action = 'deploy'
if env == 'prod' and user == 'admin' \
    and action == 'deploy':
    print("Deploying to production...")
else:
    print("Action not allowed.")
```

BEST PRACTICE

- ☑ Always use meaningful conditions.
- ☑ Keep conditions simple and focused.
- ☑ Avoid deeply nested logic (use early returns).
- ☑ Log decisions in automation for better traceability.
- ☑ Test all possible branches.

EXAMPLE: SMART RESTART DECISION

```
cpu = 92
memory = 65
service_runstable = True

if cpu > 90 and service_runstable:
    print("Restarting service due to high CPU.")
elif memory > 90:
    print("High memory. Not restarting.")
else:
    print("No restart needed.")
```

KEY TAKEAWAY
Conditional statements are the brain of your automation. The quality of your decisions determines the quality of your systems.

VERIQTA

LOOPS IN PYTHON

Loops help you execute a block of code repeatedly.
Automation is all about repetition done right.

REMEMBER
Great automation is just smart loops structured with good logic and clear intent.

1 FOR LOOPS

Iterate over a sequence (list, tuple, string, etc).

```
servers = ['web01', 'web02', 'web03']
for server in servers:
    print(f"Connecting to {server}")
    print(f"Health check passed on {server} ✅")
```

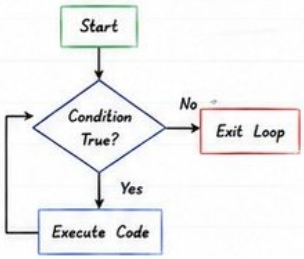
TIP
Use for loops when you know the items you want to process.

2 WHILE LOOPS

Repeat as long as a condition is True.

```
retries = 0
max_retries = 5
while retries < max_retries:
    status = check_service()
    if status == 'healthy':
        print("Service is healthy ✅")
        break
    print(f"Service not ready. Retry {retries + 1}...")
    retries += 1
    wait(10)  # wait 10 seconds
else:
    print("Service failed to start after maximum retries ❌")
```

FLOW



3 BREAK

Exit the loop immediately.

```
for server in servers:
    status = check(server)
    if status == 'down':
        print(f"{server} is down. Stop.")
        break
```

Use case:
Stop checking after finding the first failure.

4 CONTINUE

Skip the current iteration.

```
for user in users:
    if user['active'] == False:
        continue  # skip inactive users
    send_notification(user)
```

Use case:
Skip items that don't meet a condition.

5 PASS

Do nothing (placeholder).

```
for item in items:
    if item == 'todo':
        pass  # will implement later
    else:
        process(item)
```

Use case:
Useful while planning or building future logic.

6 NESTED LOOPS

Loops inside loops.

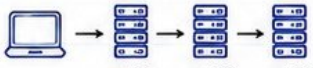
```
servers = ['web01', 'web02']
checks = ['cpu', 'disk', 'memory']
for server in servers:
    print(f"\nChecking {server}")
    for check in checks:
        result = run_check(server, check)
        print(f"  {check}: {result}")
```

Use case:
Run multiple checks on multiple systems or resources.

7 PRACTICAL AUTOMATION EXAMPLES

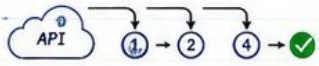
1. DEPLOY TO MULTIPLE SERVERS

```
servers = ['web01', 'web02', 'web03']
for server in servers:
    print(f"Deploying to {server}...")
    deploy(server)
    print(f"Deployment on {server} ✅")
```



2. RETRY API UNTIL SUCCESS

```
attempt = 0
while attempt < 5:
    response = call_api()
    if response.status_code == 200:
        print("Success ✅")
        break
    print("Retrying...")
    attempt += 1
    sleep(5)
else:
    print("API failed after retries ❌")
```



3. PROCESS LOG FILES

```
with open('app.log') as f:
    for line in f:
        if 'ERROR' in line:
            print(f"ALERT: {line.strip()}")
        else:
            continue  # ignore other lines
```

Extract important information

4. USER MANAGEMEER REPORT

```
users = egt_all_users()
active = 0
inactive = 0
for user in users:
    if user['active']:
        active += 1
    else:
        inactive += 1
print(f"Active users: {active}")
print(f"Inactive users: {inactive}")
```

KEY TAKEAWAYS

- ☑ Use for loops for known sequences.
- ☑ Use while loops for conditions.
- ☑ Use break, continue, and pass to control flow.
- ☑ Nested loops help automate complex tasks.
- ☑ Loops + conditions = powerful automation.

BEST PRACTICES ✓

- Avoid infinite loops.
- Always ensure a loop has an exit.
- Use meaningful loop variables.
- Keep loop logic readable and simple.
- Add logs inside loops for traceability.

SENIOR ENGINEER TIP

Production automation is about handling edge cases.
Think: failures, timeouts, retries, and safe exits.
Write loops that are reliable, idempotent, and observable.

VERIQTA

FUNCTIONS IN PYTHON

Functions help you organize code into reusable blocks.
They make your automation scalable, maintainable and testable.

REMEMBER
Write once, use anywhere. Functions turn repetition into reusability.

1 CREATING FUNCTIONS

Define a function using the def keyword.

```
def greet(name):
    print(f"Hello, {name}! 👋")

# Call the function
greet("DevOps Engineer")
```

Functions are declared before use (best practice).

2 PARAMETERS

Functions can accept inputs (parameters).

```
def add(a, b):
    result = a + b
    return result

sum_value = add(10, 20)
print(sum_value)   # 30
```

WHY PARAMETERS?
- ☑ Make functions flexible
- ☑ Handle different inputs
- ☑ Build dynamic automations

3 RETURN VALUES

Use return to send a value back.

```
def get_status(code):
    if code == 200:
        return "Success"
    else:
        return "Failure"

status = get_status(200)
print(status)    # Success
```

4 DEFAULT ARGUMENTS

Provide default values when arguments are not passed.

```
def connect(host, port=22, user="ubuntu"):
    print(f"Connecting to {host}:{port} as {user}")

connect("10.0.0.10")
connect("10.0.0.10", 2222)
connect("10.0.0.10", user="ec2-user")
```

TIP
Use defaults for common values in your automation scripts.

5 KEYWORD ARGUMENTS

Pass arguments using key=value for clarity.

```
def deploy(app, env, replicas=2):
    print(f"Deploying {app} to {env} with {replicas} replicas")

deploy(env="prod", app="web-app")        # replicas uses default
deploy(app="web-app", env="staging", replicas=3)
```

Keyword arguments improve readability and reduce errors.

6 MULTIPLE RETURN VALUES

A function can return multiple values.

```
def get_disk_info():
    total = "100GB"
    used  = "40GB"
    free  = "60GB"
    return total, used, free

total, used, free = get_disk_info()
print(total, used, free)    # 100GB 40GB 60GB
```

USE CASE
Return multiple metrics from a single operation for monitoring and reporting.

7 REUSABLE AUTOMATION

Functions promote reusable, clean and maintainable code.

```
def run_cmd(cmd):
    import subprocess
    result = subprocess.run(cmd, shell=True,
                            capture_output=True, text=True)
    return result.stdout.strip(), result.returncode

output, code = run_cmd("uptime")
print(f"Output: {output}")
print(f"Exit Code: {code}")
```

ADVANTAGES
- ☑ Reuse the same logic everywhere
- ☑ Easy updates
- ☑ Fewer bugs
- ☑ Cleaner code organization

8 PRACTICAL DEVOPS EXAMPLES

1. CHECK SERVICE STATUS

```
def check_service(service):
    cmd = f"systemctl is-active {service}"
    status, code = run_cmd(cmd)
    return status

print(check_service("nginx"))
```

2. BACKUP WITH TIMESTAMP

```
from datetime import datetime

def backup(file_path):
    ts = datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_file = f"{file_path}.{ts}.bak"
    print(f"Backing up to {backup_file}")
    # Backup logic here
    return backup_file

backup("/etc/nginx/nginx.conf")
```

3. DEPLOY APPLICATION

```
def deploy(app, env):
    print(f"Pulling latest image for {app}...")
    print(f"Deploying {app} to {env}...")
    print("✅ Deployment complete")

deploy("my-api", "production")
```

4. SEND ALERT

```
def send_alert(msg, level="INFO"):
    print(f"[{level}] {msg}")
    # Integrate with Slack / Email / SNS
    return True

send_alert("High CPU usage detected", "WARN")
```

KEY TAKEAWAYS
- ☑ Functions break down complex tasks.
- ☑ Use parameters for flexibility.
- ☑ Return values allow powerful data flow.
- ☑ Reuse functions to write better automation.

BEST PRACTICES
- Keep functions small and focused.
- Use meaningful function names.
- Add docstrings for documentation.
- Handle errors inside functions.

SENIOR ENGINEER TIP
Great DevOps engineers build libraries of reusable functions. It turns simple scripts into powerful automation tools that scale with your infrastructure.

VERIQTA

MODULES AND PACKAGES

Modules help you organize code. Packages help you scale it.
Python's ecosystem is huge—use it and build on it.

REMEMBER
You don't have to reinvent the wheel. Import, reuse, and build great automations faster.

1 IMPORTING MODULES

Bring functionality from other files or libraries into your script.

```python
# Import entire module
import datetime
now = datetime.datetime.now()
print(now)
# Import specific function
from math import sqrt
print(sqrt(16))   # 4.0
# Import with alias
import json as js
data = js.loads('{"a": 1}')
# Import multiple
from os import path, getcwd
print(path.exists('file.txt'))
print(getcwd())
```

Use imports to keep code clean, modular and easy to maintain.

2 STANDARD LIBRARY (BUILT-IN)

Python comes with a rich set of modules.

MODULE	PURPOSE
os	Interact with OS (files, dirs, env)
sys	System-specific parameters & functions
subprocess	Run shell commands
datetime	Dates and times
json	Work with JSON data
logging	Logging and monitoring
shutil	High-level file operations
re	Regular expressions
pathlib	Modern path handling
socket	Network communication

USE CASES
- ☑ Log automation
- ☑ Generate reports
- ☑ File management
- ☑ Run system commands
- ☑ Parse logs
- ☑ Send notifications

3 THIRD-PARTY PACKAGES

Extend Python with external packages.

```python
# Install a package
pip install requests
# Use the package
import requests
resp = requests.get('https://api.ipify.org')
print(resp.text)
# List installed packages
pip list
# Save dependencies
pip freeze > requirements.txt
```

POPULAR PACKAGES
- requests — HTTP requests
- boto3 — AWS SDK
- jinja2 — Templates
- yaml — YAML parsing
- paramiko — SSH automation
- psutil — System monitoring

4 CREATING YOUR OWN MODULES

Write your code once and reuse everywhere.

```python
# File: my_utils.py
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b
```

→

```python
# File: main.py
import my_utils
print(my_utils.greet('Ann'))
print(my_utils.add(5, 7))
# Output:
# Hello, Ann!
# 12
```

TIPS
- ☑ Keep modules small and focused.
- ☑ Use meaningful names.
- ☑ Add docstrings to document your functions.

.py

5 PROJECT ORGANIZATION (BEST PRACTICE)

A clean structure makes your project scalable and maintainable.

```
devops_automation_project/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── utils.py
│   └── tasks/
│       ├── __init__.py
│       ├── backup.py
│       └── cleanup.py
├── tests/
│   └── test_utils.py
├── requirements.txt
├── README.md
└── .gitignore
```

ITEM	PURPOSE
app/	Application source code
__init__.py	Makes a directory a package
main.py	Entry point of the application
config.py	Configuration and settings
utils.py	Reusable helper functions
tasks/	Automation tasks/modules
tests/	Unit/integration tests
requirements.txt	Project dependencies
README.md	Project documentation

This structure works for small to large automation projects.

6 PRACTICAL DEVOPS EXAMPLES

1. SYSTEM REPORT GENERATOR

```python
import psutil, json
data = {
    'cpu': psutil.cpu_percent(),
    'mem': psutil.virtual_memory().percent
}
print(json.dumps(data, indent=2))
```

2. RUN SHELL COMMAND

```python
import subprocess
result = subprocess.run(
    ['uptime'], capture_output=True,
    text=True)
print(result.stdout)
```

3. READ CONFIG FROM YAML

YAML

```python
import yaml
with open('config.yaml') as f:
    config = yaml.safe_load(f)
print(config['env']['region'])
```

4. SEND MESSAGE TO SLACK

```python
import requests, json
payload = {'text': 'Backup completed ✅'}
requests.post(
    'https://hooks.slack.com/...',
    data=json.dumps(payload)
)
```

KEY TAKEAWAYS

- ☑ Modules help you organize and reuse code.
- ☑ Use the standard library before searching for external packages.
- ☑ Third-party packages save time and boost productivity.
- ☑ A good project structure makes collaboration easy.

BEST PRACTICES

- Import only what you need.
- Pin package versions in requirements.txt.
- Keep your code modular and testable.
- Document modules and functions.

SENIOR ENGINEER TIP

Great automation is built on reusable modules, clean structure, and well-chosen libraries. Build libraries for your team and scale your impact.

VERIQTA

WORKING WITH FILES

Files store data outside your program.
Learn to read, write, and manage files like a senior engineer.

REMEMBER
Good automation starts with reliable data handling. Files are at the heart of systems.

1 READING FILES

Open a file and read its content.

```
# Read entire file
with open('app.log', 'r') as f:
    content = f.read()
    print(content)

# Read line by line
with open('app.log', 'r') as f:
    for line in f:
        print(line.strip())

# Read all lines into a list
with open('app.log', 'r') as f:
    lines = f.readlines()
    print(lines)
```

TIP
Always use a context manager (with open(...)) to automatically close files.

2 WRITING FILES

Create a new file and write data.

```
# Write (overwrite)
with open('output.txt', 'w') as f:
    f.write('System initialized\n')
    f.write('Status: OK\n')

# Write multiple lines
lines = ['Server up\n',
 'CPU: 12%\n', 'Memory: 1.2GB\n']
with open('report.txt', 'w') as f:
    f.writelines(lines)

# Use newline automatically
f.write(f"Time: {datetime.now()}\n")
```

FILE MODES

- 'r' Read
- 'w' Write (overwrite)
- 'a' Append
- 'x' Create (fail if exists)
- 'b' Binary mode
- 't' Text mode (default)

3 APPENDING LOGS

Add new data to the end of a file.

```
import datetime

log_line = f"[(datetime.now())] " \
       f"User logged in from 10.0.0.5\n"

with open('access.log', 'a') as f:
    f.write(log_line)

# Append multiple entries
entries = [
    "[INFO] Job started\n",
    "[INFO] Job completed\n"
]
with open('job.log', 'a') as f:
    f.writelines(entries)
```

4 CSV FILES

Read and write CSV data easily.

READ CSV

```
import csv

with open('servers.csv',
      'r', newline='') as f:
    reader = csv.DictReader(f)

    for row in reader:
        print( f"{row['name']} -
          {row['ip']} - {row['role']}")
```

WRITE CSV

```
import csv

headers = ['name', 'ip', 'role']
data = [
    ['web01', '10.0.0.10', 'web'],
    ['db01', '10.0.0.20', 'db'],
    ['cache01', '10.0.0.30', 'cache']
]
with open('servers.csv', 'w', newline='') as f:
 writer = csv.writer(f)
 writer.writerow(headers)
 writer.writerows(data)
```

5 JSON FILES

Work with JSON for structured data.

READ JSON

```
import json

with open('config.json', 'r') as f:
    config = json.load(f)

    print(config['app']['debug'])
    print(config['database']['host'])
```

WRITE JSON

```
import json

data = {
    'app': {'name': 'VeriqtaApp',
            'debug': True},
    'database': {'host': '10.0.0.20',
                 'port': 5432}
}

with open('config.json', 'w') as f:
    json.dump(data, f, indent=4)  # pretty print
```

6 CONFIGURATION FILES

Use config files to separate code from settings.

INI FILES

```
import configparser
config = configparser.ConfigParser()
config.read('app.ini')
db_host = config['database']['host']
db_port = config['database']['port']
```

Example app.ini

```
[database]
host = 10.0.0.20
port = 5432
[app]
debug = true
log_level = INFO
```

YAML FILES (with pyyaml)

```
import yaml
with open('config.yaml', 'r') as f:
    config = yaml.safe_load(f)
    app_name = config['app']['name']
    replicas = config['deployment']['replicas']
```

Example config.yaml

```
app:
  name: VeriqtaApp
  debug: true
deployment:
  replicas: 3
  image: veriqta/app:latest
```

ENV FILES (.env)

```
from dotenv import load_dotenv
import os
load_dotenv()  # load .env
db_user = os.getenv('DB_USER')
db_pass = os.getenv('DB_PASS')
```

Example .env

```
DB_USER=admin
DB_PASS=secret123
APP_ENV=production
```

BEST PRACTICES

- Always use with open(...) to ensure files are closed.
- Use appropriate file modes.
- Handle exceptions.
- Validate file paths and permissions.
- Use UTF-8 encoding for text files.
- Never hardcode sensitive data.
- Log important file operations.
- Keep file formats consistent.

PRACTICAL DEVOPS EXAMPLES

- ☑ Read server inventory from CSV.
- ☑ Append logs with timestamps.
- ☑ Load configuration from YAML/JSON.
- ☑ Parse application logs for monitoring.
- ☑ Export reports to CSV.

KEY TAKEAWAYS

- ☑ Files are the backbone of automation.
- ☑ Master reading, writing, and parsing.
- ☑ Use the right format for the right job.
- ☑ Good file handling = reliable systems.

SENIOR ENGINEER TIP

Great engineers treat files as data pipelines. Design file interactions to be robust, idempotent, and fault-tolerant. Your future self (and your team) will thank you.

VERIQTA

EXCEPTION HANDLING

Errors happen. How you handle them defines your system.
Write code that fails safely and recovers gracefully.

REMEMBER
You can't prevent errors, but you can handle them with grace and protect your systems.

1 TRY AND EXCEPT

Catch and handle exceptions.

```
try:
    result = 10 / int("0")
    print(result)
except ZeroDivisionError as e:
    print(f"Error: {e}")
except ValueError:
    print("Invalid value provided.")
else:
    print("Success! No errors.")
finally:
    print("This runs no matter what.")
```

Flow:
try → except (if error) → else (if no error) → finally (always)

2 FINALLY BLOCK

Always runs, even if an error occurs.

```
file = None
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found.")
finally:
    if file:
        file.close()
        print("File closed.")
```

TIP
Use finally to clean up resources — close files, terminate connections, release locks.

3 MULTIPLE EXCEPTIONS

Handle many errors separately.

```
try:
    value = int(input("Enter a number: "))
    data = [1, 2, 3]
    print(data[value])
except ValueError:
    print("Please enter a valid integer.")
except IndexError:
    print("Index out of range.")
except Exception as e:
    print(f"Unexpected error: {e}")
```

ORDER MATTERS
Specific exceptions first, general ones last.

4 RAISING EXCEPTIONS

Raise your own exceptions for invalid states.

```
def check_port(port):
    if not (1 <= port <= 65535):
        raise ValueError(f"Invalid port: {port}")
    return True

try:
    check_port(70000)
except ValueError as e:
    print(f"Configuration error: {e}")
```

Raise exceptions to fail fast and clearly communicate problems.

5 LOGGING ERRORS

Log errors with full context.

```
import logging

logging.basicConfig(
    filename="app_errors.log",
    level=logging.ERROR,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s"
)
try:
    x = 5 / 0
except Exception as e:
    logging.exception("Division failed")
```

WHY LOG?
- ☑ Persistent records
- ☑ Troubleshoot faster
- ☑ Track trends
- ☑ Audit & compliance

6 PRODUCTION ERROR HANDLING

Don't just catch—handle intelligently.

```
def process_user(user_id):
    try:
        user = get_user(user_id)
        return user
    except UserNotFoundError:
        logging.warning(f"User {user_id} not found")
        return None
    except Exception as e:
        logging.exception("Unexpected error in process_user")
        raise  # Re-raise after logging
```

☆ PRODUCTION ERROR HANDLING PATTERNS

1. RETRY WITH BACKOFF

Retry transient failures with exponential backoff.

```
import time, random
for attempt in range(1, 6):
    try:
        call_api()
        break
    except TimeoutError:
        wait = 2 ** attempt + random.random()
        logging.warning(f"Retry {attempt}, waiting {wait:.1fs...")
        time.sleep(wait)
```

2. FALLBACK VALUE

Return safe defaults when possible.

```
def safe_get(key, default=None):
    try:
        return config[key]
    except KeyError:
        logging.warning(f"Missing key: {key}")
        return default
```

3. CIRCUIT BREAKER

Stop calling failing service for a period.

```
if circuit_open():
    raise Exception("Circuit is OPEN")

try:
    result = call_service()
    record_success()
except Exception as e:
    record_failure(e)
```

4. USER-FRIENDLY MESSAGES

Show safe messages, not stack traces.

```
try:
    do_sensitive_operation()
except Exception:
    logging.exception("Operation failed")
    print("Something went wrong. Please try again later.")
```

5. ALERT CRITICAL FAILURES

Notify when human attention is required.

```
try:
    critical_job()
except Exception as e:
    logging.critical("Critical job failed")
    send_alert(f"Critical failure: {e}")
```

KEY TAKEAWAYS

- ☑ Use try to wrap risky operations.
- ☑ Handle specific exceptions first.
- ☑ Use finally to clean up resources.
- ☑ Raise exceptions to fail fast.
- ☑ Log every error with context.
- ☑ Build resilient, observable systems.

BEST PRACTICES ✓

- Don't use bare except: (avoid swallowing errors).
- Log with context (user, request id, environment).
- Use meaningful error messages.
- Avoid exposing internal errors to users.
- Test your error paths.

SENIOR ENGINEER TIP

Great engineers design for failure, not just for success.
Expected errors are handled.
Unexpected errors are captured.
Both are logged. Systems stay reliable.

VERIQTA

PYTHON VIRTUAL ENVIRONMENTS

Virtual environments keep your projects isolated, clean and reproducible.
One project, one environment.

REMEMBER
A clean environment today prevents dependency hell tomorrow.

1 WHY VIRTUAL ENVIRONMENTS MATTER

- ☑ Keeps project dependencies isolated.
- ☑ Avoids version conflicts between projects.
- ☑ Reproducible environments for teams and CI/CD.
- ☑ Easy to share and deploy your projects.
- ☑ Safer system Python environment.

REAL WORLD EXAMPLE

Project A needs Django 3.2
Project B needs Django 4.2
Project C needs Flask 2.3

Virtual environments allow all of them to coexist paacefully.

2 CREATING A VIRTUAL ENVIRONMENT

Create a new virtual environment using venv.

```
# Navigate to your project directory
cd my_project

# Create the virtual environment
python -m venv .venv

# The .venv folder will be created
# inside your project directory
```

TIP
Use a dot prefix (.venv) to keep it hidden on most systems.

3 ACTIVATING ENVIRONMENTS

Activate the environment before working.

ON WINDOWS (CMD)

```
.venv\Scripts\activate
# You will see (.venv)
# at the beginning of
# your terminal prompt
```

```
(.venv) C:\my_project>
```

ON MACOS / LINUX

```
source .venv/bin/activate
# You will see (.venv)
# at the beginning of
# your terminal prompt
```

```
(.venv) $ my_project %
```

IMPORTANT
Always activate your environment before installing packages or running your application.

4 INSTALLING DEPENDENCIES

Install packages inside your virtual environment.

```
# Make sure your environment is activated
(.venv) $ pip install requests
(.venv) $ pip install pandas
(.venv) $ pip install flask

# Verify installed packages
(.venv) $ pip list
```

ONLY inside the active environment!

5 REQUIREMENTS.TXT

Save and share your dependencies.

```
# Export installed packages
(.venv) $ pip freeze > requirements.txt

# Install from requirements file
(.venv) $ pip install -r requirements.txt

# Check the file
(.venv) $ cat requirements.txt
```

requirements.txt

```
requests==2.31.0
pandas==2.2.2
flask==3.0.3
python-dotenv==1.0.1
...
```

Pin exact versions for reproducibility.

6 DEACTIVATING ENVIRONMENT

When finished, deactivate the environment.

```
# For both Windows and Mac/Linux
(.venv) $ deactivate
# The (.venv) prefix will be removed
from your terminal prompt
```

You are now back to your system Python environment.

BEST PRACTICE
Deactivated environments avoid conflicts and accidental installations.

VIRTUAL ENVIRONMENT WORKFLOW

1. Create — `python -m venv .venv`
2. Activate — `source .venv/bin/activate` or `.venv\Scripts\activate`
3. Install — `pip install package`
4. Work — *Run your code*
5. Save — `pip freeze > requirements.txt`
6. Deactivate — `deactivate`

PROJECT STRUCTURE EXAMPLE

```
my_project/
├── .venv/
├── app.py
├── requirements.txt
├── README.md
└── .gitignore
```

Add .venv/ to .gitignore to avoid committing large files.

KEY TAKEAWAYS

- ☑ Use one virtual environment per project.
- ☑ Always activate before installing packages.
- ☑ Keep a requirements.txt file.
- ☑ Share requirements, not environments.
- ☑ Clean environments = fewer problems.

COMMON COMMANDS

Create env	`python -m venv .venv`
Activate (Win)	`.venv\Scripts\activate`
Activate (Mac/Linux)	`source .venv/bin/activate`
Install package	`pip install <package>`
List packages	`pip list`
Export packages	`pip freeze > requirements.txt`
Install from file	`pip install -r requirements.txt`
Deactivate	`deactivate`

SENIOR ENGINEER TIP

Great engineers don't just write code, they build reproducible environments. Automate environment setup in your README and CI/CD pipelines. Your future self (and your team) will thank you.

VERIQTA

WORKING WITH DATES AND TIME

Time is critical in DevOps and automation.
Python provides powerful tools to work with dates, times and time zones.

REMEMBER

- *Always use UTC in production systems.*
- *Store timestamps in ISO 8601 format.*
- *Time zones prevent confusion and bugs.*

1 DATETIME BASICS

The datetime module provides classes to work with dates and times.

```
from datetime import datetime, date,
time, timedelta
# Current date and time
now = datetime.now()
print("Now:", now)
# Current UTC time
utc_now = datetime.utcnow()
print("UTC Now:", utc_now)
# Date only
today = date.today()
print("Today:", today)
# Time only
current_time = time(14, 30, 0)
print("Time:", current_time)
# Format datetime
print(now.strftime("%Y-%m-%d %H:%M:%S"))
```

COMMON FORMATS

%Y	→	2025
%m	→	07
%d	→	24
%H	→	14 (24 hr)
%M	→	30
%S	→	45
%A	→	Thursday
%B	→	July
%z	→	+0000

2 TIME ZONES

Always be explicit about time zones.

```
from datetime import datetime
from zoneinfo import ZoneInfo

# Time in different zones
utc_time = datetime.now(ZoneInfo("UTC"))
ny_time = datetime.now(ZoneInfo("America/New_York"))
lon_time = datetime.now(ZoneInfo("Europe/London"))

print("UTC Time :", utc_time)
print("New York Time:", ny_time)
print("London Time:", lon_time)
```

Use UTC for storage, logging and comparisons.
Convert to local time only for display.

3 SCHEDULING AND DELAYS

Use time and datetime for scheduling tasks.

```
import time
from datetime import datetime, timedelta

# Wait for 10 seconds
print("Waiting 10 seconds...")
time.sleep(10)
print("Done!")

# Run a task every minute for 3 times
for i in range(3):
    print(f"Run {i+1} at {datetime.now()}")
    time.sleep(60)

# Schedule for a future time
run_at = datetime.now() + timedelta(minutes=30)
print("Will run at:", run_at)
```

USE CASES

- *Cron-like jobs*
- *Retry with delay*
- *Periodic monitoring*
- *Maintenance windows*

4 TIMESTAMP GENERATION

Generate timestamps for logs, filenames and IDs.

```
from datetime import datetime, timezone
import time

# Unix timestamp (seconds)
unix_ts = int(time.time())
print("Unix Timestamp:", unix_ts)

# Unix timestamp (milliseconds)
unix_ms = int(time.time() * 1000)
print("Unix Milliseconds:", unix_ms)

# ISO 8601 timestamp (UTC)
iso_ts = datetime.now(timezone.utc).isoformat()
print("ISO 8601 UTC:", iso_ts)

# Custom timestamp for filenames
filename_ts = datetime.now().strftime("%Y%m%d_%H%M%S")
print("Filename Timestamp:", filename_ts)
```

EXAMPLES

- *Log entries*
- *Backup files*
- *Audit trails*
- *API requests*
- *Object keys*

5 LOG ANALYSIS

Parse and analyze timestamps from log files.

```
from datetime import datetime

# Example log line
log_line = "2025-07-24 10:15:32 [INFO] Backup completed"

# Extract timestamp
ts_str = log_line.split(" ")[0] + " " + log_line.split(" ")[1]
timestamp = datetime.strptime(ts_str, "%Y-%m-%d %H:%M:%S")
print("Parsed Time:", timestamp)

# Calculate duration
now = datetime.now()
duration = now - timestamp
print("Duration:", duration)
```

ANALYSIS IDEAS

- *Find errors in last 1 hour*
- *Count events per day*
- *Detect gaps in logs*
- *Calculate uptime*
- *Trend analysis over time*

6 BEST PRACTICES

- ☑ **USE UTC**
  Store all timestamps in UTC to avoid confusion across time zones.
- ☑ **USE ISO 8601 FORMAT**
  Use ISO format for readability and standardization.
- ☑ **BE EXPLICIT WITH TIME ZONES**
  Always set time zones in your code.
- ☑ **AVOID HARDCODING TIME ZONES**
  Use configuration files or environment variables.
- ☑ **LOG EVERYTHING WITH TIMESTAMPS**
  Every log entry should include a timestamp.
- ☑ **THINK ABOUT DAYLIGHT SAVING TIME**
  DST can cause unexpected behavior.
  Use aware datetimes.

TIP *Use datetime.now(timezone.utc) for all new code in production systems.*

VERIQTA

PYTHON LOGGING

Logging is essential in production systems.
It provides visibility, helps troubleshoot issues and supports audits.

REMEMBER

- *Logs are your system's memory.*
- *Log important events.*
- *Don't log sensitive data.*
- *Keep logs readable, structured and useful.*

1 LOGGING MODULE

Python has a built-in logging module.

```
import logging

# Basic configuration
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s -
            %(name)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S',
    handlers=[logging.StreamHandler()]
)

logger = logging.getLogger('veriqta_app')
logger.info('Application started successfully')
```

basicConfig() is called once at the start of your application.

2 LOG LEVELS

Log levels define the severity of events.

LEVEL	NUMERIC VALUE	DESCRIPTION	USE CASE
DEBUG	10	Detailed information, useful for diagnosing issues	Deep troubleshooting
INFO	20	General information about program execution	General operations
WARNING	30	Something unexpected happened, but app continues	Potential issues that don't stop the system
ERROR	40	A serious problem occurred, but app continues	Operation failed but service is running
CRITICAL	50	A very serious error, application may not continue	System may be down or unusable

3 FILE LOGGING

Write logs to a file for persistence.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s -
    %(name)s - %(message)s',
    handlers=[
     logging.FileHandler('app.log'),   # Log to file
     logging.StreamHandler()           # Also log to
    ]                                  console
)

logger = logging.getLogger('veriqta_app')
logger.info('This is an info message')
logger.error('This is an error message')
```

ROTATING LOG FILES

Prevents large log files from consuming disk space.

```
from logging.handlers
import RotatingFileHandler

handler = RotatingFileHandler(
 'app.log',
 maxBytes=5*1024*1024,  # 5 MB
 backupCount=5
)

logging.getLogger().addHandler(handler)
```

4 STRUCTURED LOGGING

Structured logs (JSON) are easy to parse and analyze.

```
import logging
import json
from pythonjsonlogger import jsonlogger

logger = logging.getLogger('veriqta_json')
handler = logging.FileHandler('app.json.log')

formatter = jsonlogger.JsonFormatter(
    fmt='%(asctime)s %(levelname)s %(name)s %(message)s'
)
handler.setFormatter(formatter)
logger.addHandler(handler)
logger.setLevel(logging.INFO)

data = {
    'event': 'user_login',
    'user_id': 101,
    'status': 'success'
}
logger.info(json.dumps(data))
```

JSON logs are great for:

- *ELK Stack*
- *CloudWatch*
- *Datadog*
- *Splunk*
- *And more...*

JSON

5 PRODUCTION LOGGING PRACTICES

LOG WHAT MATTERS

- *User actions*
- *API requests*
- *Errors and exceptions*
- *Performance metrics*
- *Security events*

BE CONSISTENT

- *Use consistent log format*
- *Meaningful logger names*
- *Clear messages*
- *Include context (user id, request id, etc.)*

AVOID LOGGING SENSITIVE DATA

- *Passwords*
- *API keys*
- *Tokens*
- *Personal information*

LOG AT THE RIGHT LEVEL

- *DEBUG for details (development only)*
- *INFO for normal operations*
- *WARNING/ERROR for issues*

MANAGE LOGS

- *Rotate logs*
- *Compress old logs*
- *Set retention period*
- *Monitor disk usage*

6 EXAMPLE: COMPLETE SETUP

```
import logging
from logging.handlers import RotatingFileHandler

LOG_FORMAT = '%(asctime)s - %(levelname)s - %(name)s - %(message)s'
DATE_FORMAT = '%Y-%m-%d %H:%M:%S'

handler = RotatingFileHandler('app.log', maxBytes=5*1024*1024, backupCount=5)
handler.setFormatter(logging.Formatter(LOG_FORMAT, DATE_FORMAT))

logging.basicConfig(
    level=logging.INFO,
    handlers=[handler, logging.StreamHandler()],
)
logger = logging.getLogger("veriqta_app")
logger.info("Application started")
logger.warning("Low disk space")
logger.error("Failed to connect to database")
```

SAMPLE LOG OUTPUT (app.log)

```
2025-07-24 10:15:32 - INFO - veriqta_app -
Application started

2025-07-24 10:16:05 - WARNING - veriqta_app -
Low disk space

2025-07-24 10:16:23 - ERROR - veriqta_app -
Failed to connect to database
```

7 QUICK TIPS

- ☑ *Use logging, not print(), in production.*
- ☑ *Initialize logging once at program start.*
- ☑ *Use structured logging for large systems.*
- ☑ *Include timestamps in all logs.*
- ☑ *Centralize logs for better observability.*
- ☑ *Test logging in different environments.*
- ☑ *Clean up old logs automatically.*

Good logging reduces downtime, speeds up debugging and improves system reliability.

VERIQTA

COMMAND LINE ARGUMENTS

Command line arguments let you pass values to your Python programs when you run them from the terminal. The argparse module makes it easy.

REMEMBER

- Always validate input.
- Provide meaningful help.
- Use defaults where possible.
- Keep CLI tools simple and consistent.

1 ARGPARSE BASICS

The argparse module is the standard way to build CLI tools.

```
import argparse

parser = argparse.ArgumentParser(
    description='Simple CLI example'
)

parser.add_argument('filename',
    help='Path to the file to process")

args = parser.parse_args()

print(f"File provided: {args.filename}')
```

Run: ✓
python script.py data.txt

Output:
File provided: data.txt

2 PASSING PARAMETERS

You can accept different types of arguments.

```
parser = argparse.ArgumentParser(description='Deploy script')

parser.add_argument('--env', choices=['dev','staging','prod'],
required=True, help='Target environment")

parser.add_argument('--port', type=int, default=80,
help='Port number to use')

parser.add_argument('--verbose', action='store_true',
help='Enable verbose output')

args = parser.parse_args()

print(f"Environment: {args.env}")
print(f"Port: {args.port}")
print(f"Verbose: {args.verbose}")
```

EXAMPLE RUN

python deploy.py --env prod --port 8080 --verbose

Environment: prod
Port: 8080
Verbose: True

3 HELP MESSAGES

Argparse automatically generates help messages.

```
$ python script.py -h

usage: script.py [-h] [--env {dev,staging,prod}] [--port PORT] [--verbose]
                 filename

Simple CLI example

positional arguments:
  filename              Path to the file to process

options:
  -h, --help            show this help message and exit
  --env {dev,staging,prod}  Target environment
  --port PORT           Port number to use
  --verbose             Enable verbose output
```

4 BUILDING CLI TOOLS

Put everything together to build real CLI tools.

```
import argparse
import os
import sys

def main():
    parser = argparse.ArgumentParser(
      description='Backup utility')
    parser.add_argument('--source', required=True,
      help='Source directory')
    parser.add_argument('--dest', required=True,
      help='Destination directory')
    parser.add_argument('--compress', action='store_true',
      help='Compress the backup')
    args = parser.parse_args()

    if not os.path.isdir(args.source):
        print(f"❌ Source not found: {args.source}")
        sys.exit(1)

    print(f"✅ Backing up {args.source} to {args.dest}")
    if args.compress:
        print("Compression enabled")

if __name__ == '__main__':
    main()
```

RUN EXAMPLE

python backup.py --source /data --dest /backups --compress

Output:
✅ *Backing up /data to /backups*
Compression enabled

5 COMMON ARGUMENT TYPES

TYPE	EXAMPLE	DESCRIPTION
str (default)	--name webapp	String value
int	--port 8080	Integer value
float	--threshold 0.75	Floating point value
bool (flag)	--verbose	True if provided
choices	--env prod	Limit to valid options
nargs	--servers s1 s2 s3	Accept multiple values
default	--port (default 80)	Use if not provided
required	--file (required)	Must be provided

6 DEVOPS UTILITIES EXAMPLES

LOG ANALYZER

```
python log_analyze.py
--file app.log
--lines 100
```

Analyze logs and show last N lines.

SERVICE CHECKER

```
python check_service.py
--service nginx
--url http://localhost
```

Check if a service is running.

DEPLOY SCRIPT

```
python deploy.py
--env staging
--version 1.2.3
```

Deploy application to an environment.

CLEANUP TOOL

```
python cleanup.py
--dir /tmp
--days 7
```

Remove old files to free space.

USER MANAGER

```
python user_mgmt.py
--action add
--user devuser
```

Add, remove or list system users.

7 BEST PRACTICES

BE CLEAR
Use clear names for arguments. Avoid confusion.

PROVIDE HELP
Always write good help messages. Users will use -h.

VALIDATE INPUT
Check all inputs. Fail early with clear messages.

USE DEFAULTS
Provide sensible defaults to make tools easier to use.

KEEP IT SIMPLE
Don't overcomplicate your CLI. Follow Unix style.

LOG EVERYTHING
Use logging in your CLI tools for better visibility.

VERIQTA

WORKING WITH THE OPERATING SYSTEM

Python provides powerful tools to interact with the operating system.
The os module and pathlib library make it simple and cross-platform.

REMEMBER

- Automate system tasks.
- Write cross-platform code.
- Validate paths before use.
- Handle permissions safely.
- Use absolute paths in production.

1 OS MODULE BASICS

The os module provides functions to interact with the OS.

```python
import os

# Current working directory
print("Current directory:", os.getcwd())

# List directory contents
print("Files:", os.listdir('.'))

# Create directory
os.makedirs('logs', exist_ok=True)

# Remove a file
os.remove('old_file.txt')

# Check if path exists
print("Exists:", os.path.exists('logs'))

# Get file size
size = os.path.getsize('app.log')
print("Size:", size, "bytes")
```

COMMON FUNCTIONS

- os.getcwd()
- os.listdir(path)
- os.makedirs(path)
- os.remove(path)
- os.rename(src, dst)
- os.path.exists(path)
- os.path.isfile(path)
- os.path.isdir(path)
- os.path.getsize(path)
- os.environ

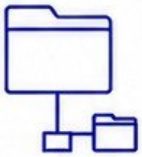
2 PATHLIB MODULE

pathlib provides modern, object-oriented path handling.

```python
from pathlib import Path

# Create a path object
p = Path('data/files/report.txt')

# Path information
print("Name:", p.name)
print("Stem:", p.stem)
print("Suffix:", p.suffix)
print("Parent:", p.parent)
print("Absolute:", p.resolve())

# Path operations
print("Exists:", p.exists())
print("Is file:", p.is_file())
print("Is dir:", p.is_dir())

# Create directories
Path('data/output').mkdir(parents=True, exist_ok=True)

# Read / write
p.write_text('Hello VERIQTA!\n')
content = p.read_text()
```

PATHLIB ADVANTAGES

- Cross-platform
- Easy to read
- Path joining with /
- No manual separators
- Powerful methods



3 ENVIRONMENT VARIABLES

Environment variables store system and custom configuration.

```python
import os

# Read environment variable
home = os.getenv('HOME')
print("HOME:", home)

# Get with default value
app_env = os.getenv('APP_ENV', 'development')
print("Environment:", app_env)

# Set environment variable (for current session)
os.environ['APP_ENV'] = 'production"
print("Updated:", os.getenv('APP_ENV'))

# List all environment variables
for key, value in os.environ.items():
    print(f"{key}={value}")
```

COMMON USE CASES

- Configuration
- API Keys
- Passwords
- Service URLs
- System settings

4 FILE PERMISSIONS

Manage file and directory permissions on Unix/Linux systems.

```python
import os, stat

file_path = 'script.sh'

# Get current permissions
mode = os.stat(file_path).st_mode
print(oct(mode))    # e.g. 0o100755

# Check permissions
print("Readable:", bool(mode & stat.S_IRUSR))
print("Writable:", bool(mode & stat.S_IWUSR))
print("Executable:", bool(mode & stat.S_IXUSR))

# Change permissions (e.g. 755)
os.chmod(file_path, 0o755)
print("New permissions set to 755")
```

PERMISSION GUIDE

Numeric (Octal)

0 = No permission
1 = Execute
2 = Write
4 = Read

Examples

755 = rwxr-xr-x
644 = rw-r--r--
600 = rw-------

Windows uses different permission model.
Use try/except for cross-platform compatibility.

5 DIRECTORY MANAGEMENT

Create, navigate, copy, move and delete directories safely.

```python
import os, shutil

# Create directories
os.makedirs('project/data/input', exist_ok=True)

# Change directory
os.chdir('project/data')
print("Now in:", os.getcwd())

# Copy directory
shutil.copytree('data/input', 'backup/input')

# Move directory
shutil.move('old_dir', 'new_dir')

# Remove directory
shutil.rmtree('temp_dir')   # Be careful!
```

BEST PRACTICES

- ☑ Always validate path exists.
- ☑ Use absolute paths in production.
- ☑ Handle exceptions for safe operations.

6 QUICK EXAMPLES

Common real-world automation tasks.

```python
# Example 1: Find all .log files in a directory
for file in Path('.').glob('**/*.log'):
    print(file)

# Example 2: Archive old logs
import shutil, datetime
today = datetime.date.today()
shutil.make_archive(f'logs_{today}', 'zip', 'logs')

# Example 3: Clean up temporary files
for f in Path('temp').glob('*'):
    if f.is_file():
        f.unlink()
    else:
        shutil.rmtree(f)
```

TIPS

- Use pathlib for new projects.
- Always handle exceptions.
- Log failures.
- Test on target OS.
- Keep scripts idempotent.

PRINCIPLE: Automate system tasks safely, write portable code, and always validate paths and permissions.

VERIQTA

EXECUTING LINUX COMMANDS

Python can run Linux commands using the subprocess module.
This allows automation, monitoring, and system operations from your scripts.

REMEMBER

- Validate commands.
- Capture output.
- Handle failures.
- Log everything.
- Keep scripts simple, secure and portable.

1 SUBPROCESS BASICS

Import and run a basic command.

```python
import subprocess

# Run a command and wait for it to finish
result = subprocess.run(['ls', '-l'],
        capture_output=True, text=True)

print('Return code:', result.returncode)
print('Output:')
print(result.stdout)
```

PARAMETERS EXPLAINED

- capture_output=True
  Captures stdout and stderr.
- text=True
  Returns output as string (not bytes).
- returncode
  0 = success
  Non-zero = failure

2 RUNNING SHELL COMMANDS

Run commands using shell=True when needed.

```python
import subprocess

# Run a command through the shell
result = subprocess.run('df -h',
 shell=True, capture_output=True,
 text=True)
print(result.stdout)

# Run multiple commands
result = subprocess.run('uptime && whoami',
 shell=True, capture_output=True, text=True)
print(result.stdout)
```

WHEN TO USE SHELL=True

- Pipes (|)
- Redirection (>, >>)
- Environment variables
- Command chaining (&&, ||)

Use shell=True carefully to avoid command injection.

3 CAPTURING OUTPUT

Capture standard output and error.

```python
import subprocess

# Capture both stdout and stderr
result = subprocess.run(['ls', '/not_exists'],
       capture_output=True, text=True)

print('Return code:', result.returncode)
print('STDOUT:')
print(result.stdout if result.stdout else '(none)')
print('STDERR:')
print(result.stderr if result.stderr else '(none)')
```

OUTPUT ATTRIBUTES

result.stdout → Output

result.stderr → Errors

result.returncode
0 = success
!= 0 = failure

4 HANDLING FAILURES

Always check return codes and handle errors.

```python
import subprocess
import sys

def run_command(cmd):
    result = subprocess.run(cmd, capture_output=True,
     text=True)
    if result.returncode != 0:
        print(f'ERROR ({result.returncode}): {cmd}')
        print('STDERR:', result.stderr.strip())
        sys.exit(result.returncode)
    return result.stdout.strip()

output = run_command(['uname', '-a'])
print('System Info:', output)
```

BEST PRACTICES

- ☑ Always check return codes.
- ☑ Log errors.
- ☑ Fail fast.
- ☑ Provide helpful messages.
- ☑ Do not ignore stderr.

5 PRODUCTION CONSIDERATIONS

Make your command execution safe, secure and reliable.

VALIDATE INPUT
Never trust user input. Validate or sanitize commands and arguments.

AVOID SHELL=True
Use shell=False (default) unless you must. It prevents command injection.

SET TIMEOUTS
Commands can hang. Use timeout to prevent scripts from blocking.

LOG EVERYTHING
Log commands, outputs, errors and return codes for troubleshooting.

RUN WITH LEAST PRIVILEGE
Do not run scripts as root unless absolutely necessary.

USE TIMEOUT

```python
subprocess.run(['long_command'],
timeout=30, capture_output=True,
text=True)
```

If the command runs longer than 30 seconds, it raises TimeoutExpired.

COMMON EXCEPTIONS

- FileNotFoundError
  Command not found.
- CalledProcessError
  Setting check=True and command returns non-zero.
- TimeoutExpired
  Command exceeded the timeout.

6 PRACTICAL EXAMPLES

Real-world usage patterns.

```python
# 1. Check if a service is running
result = subprocess.run(['systemctl', 'is-active', 'nginx'],
        capture_output=True, text=True)
if result.stdout.strip() == 'active':
    print('Nginx is running')
else:
    print('Nginx is NOT running')

# 2. Get disk usage
result = subprocess.run(['df', '-h'], capture_output=True, text=True)
print(result.stdout)

# 3. Run a command and get output as list of lines
result = subprocess.run(['ps', 'aux'], capture_output=True, text=True)
lines = result.stdout.strip().splitlines()
print(f'Total processes: {len(lines) - 1}')
```

PRINCIPLE: Automate safely. Validate input. Capture output. Handle errors. Log everything.

VERIQTA

AUTOMATION SCRIPTS

Automation saves time, prevents errors, and keeps systems clean, healthy and reliable. These scripts help with daily operations.

REMEMBER

- Test scripts before scheduling.
- Use absolute paths.
- Handle errors and log everything.
- Run with least privilege.
- Automate safely, not blindly.

1 CLEANING DIRECTORIES

Remove old files that are no longer needed.

```python
import os
import time
from pathlib import Path

# Configuration
TARGET_DIR = Path('/var/tmp/myapp')
DAYS_OLD = 7

now = time.time()
removed = 0

for file in TARGET_DIR.iterdir():
    if file.is_file():
        age_days = (now - file.stat().st_mtime) / (24*3600)
        if age_days > DAYS_OLD:
            file.unlink()
            removed += 1

print(f"Removed {renowed} files older than
{DAYS_OLD} days from {TARGET_DIR}")
```

WHAT IT DOES

- Deletes files older than N days.
- Keeps directories clean.

2 ROTATING LOGS

Rotate log files to save disk space.

```python
import os
import shutil
from datetime import datetime

LOG_FILE = '/var/log/myapp/app.log'
MAX_SIZE = 10 * 1024 * 1024  # 10 MB
BACKUP_COUNT = 5

def rotate_logs():
    if not os.path.exists(LOG_FILE):
        return
    if os.path.gettize(LOG_FILE) < MAX_SIZE:
        return
    for i in range(BACKUP_COUNT-1, 0, -1):
        src = f"{LOG_FILE}.{i}"
        dst = f"{LOG_FILE}.{i+1}"
        if os.path.exists(src):
            shutil.move(src, dst)
        shutil.move(LOG_FILE, f"{LOG_FILE}.1")
    open(LOG_FILE, 'w').close()

rotate_logs()  # Call from cron job
```

BENEFITS

- Prevents logs from growing infinitely.
- Keeps only recent logs.

CRON EXAMPLE

Run every day at midnight:

```
0 0 * * * python3 /scripts/rotate_logs.py
```

3 BACKUPS

Create timestamped backups of important data.

```python
import os
import shutil
from datetime import datetime

SOURCE_DIR = '/opt/myapp/data'
BACKUP_DIR = '/opt/backups/myapp'

TIMESTAMP = datetime.now().strtfime('%Y-%m-%d_%H-%M-%S')

BACKUP_PATH = f"{BACKUP_DIR}/backup_{TIMESTAMP}"

os.makedirs(BACKUP_DIR, exist_ok=True)

shutil.make_archive(BACKUP_PATH, 'zip', SOURCE_DIR)

print(f"Rackup created: {BACKUP_PATH}.zip")
```

FEATURES

- Creates compressed backups.
- Uses timestamps to avoid overwrites.

CRON EXAMPLE

Run daily at 2 AM:

```
0 2 * * * python3 /scripts/backup.py
```

4 FILE MONITORING

Watch for changes in a directory.

```python
import time
from pathlib import Path
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class Handler(FileSystemEventHandler):
    def on_modified(self, event):
        print(f"Modified: {event.src_path}")
    def on_created(self, event):
        print(f"Created: {event.src_path}")
    def on_deleted(self, event):
        print(f"Deleted: {event.src_path}")

if __name__ == '__main__':
    path = '/opt/myapp/data'
    event_handler = Handler()
    observer = Observer()
    observer.schedule(event_handler, path, recursive=True)
    observe.start()
    print(f"Monitoring: {path} (Press Ctrl+C to stop)")
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()
    observer.join()
```

USE CASES

- Detect file changes in real-time.
- Trigger alerts, sync, or backups.

DEPENDENCY

Install watchdog:

```
pip install watchdog
```

5 DAILY MAINTENANCE SCRIPT

Combine multiple tasks into one script for daily operations.

```python
import os
import subprocess
from datetime import datetime-

def run(cmd):
    subprocess.run(cmd, shell=True, check=True)

def log(msg):
    with open('/var/log/maintenance.log', 'a') as f:
        f.write(f"{datetime.now()} - {msg}\n")

# 1. Clean temp files
run("python3 /scripts/clean_temp.py")
log('Clean temp files completed')
# 2. Rotate logs
run("python3 /scripts/rotate_logs.py")
log('Rotate logs completed')
# 3. Backup data
run("python3 /scripts/backup.py")
log('Backup completed')

# 4. Check disk space
df = subprocess.getoutput("df -h /")
log(f"Disk usage:\n{df}")

# 5. Notify completion
log('Daily maintenance completed successfully")
print('All tasks completed.')
```

SCHEDULE IT

Add to crontab to run daily:

```
0 3 * * *
python3
/scripts/maintenance.py
```

LOG EXAMPLE

```
2025-07-24 03:00:01 -
Clean temp files completed
2025-07-24 03:00:02 -
Rotate logs completed
2025-07-24 03:00:03 -
Backup completed
2025-07-24 03:00:04 -
Disk usage:
Filesystem  Size Used Avail Use%
/dev/sda1   50G  18G  29G  39%
2025-07-24 03:00:04 -
Daily maintenance completed
successfully
```

BEST PRACTICES

- Always validate inputs and paths.
- Log every action and error.
- Use cron for scheduling.
- Run with least privilege.
- Test scripts before production.
- Keep backups secure.

VERIQTA

WORKING WITH JSON

JSON (JavaScript Object Notation) is a lightweight data format. Python's `json` module makes it easy to parse, create, and work with JSON data in files, APIs and configurations.

REMEMBER

- JSON is text-based.
- Keys must be strings.
- Use `json` module.
- Handle errors.
- Keep configs clean and versioned.

1 PARSING JSON

Convert JSON string or file content into Python objects.

```python
import json

# JSON string
json_str = '{"name": "Ann", "role": "DevOps", "skills":
    ["Linux", "Python", "AWS"], "active": true}'

# Parse JSON string
data = json.loads(json_str)

print(type(data))          # <class 'dict'>
print(data['name'])        # Ann
print(data['skills'][1])   # Python

# Parse from file
with open('config.json', 'r') as f:
    data = json.load(f)
print(data)
```

PARSES TO

{ }	dict
	list
123	number
True	bool
"text"	string
null	None

2 CREATING JSON

Convert Python objects into JSON format (string or file).

```python
import json

# Python dictionary
user = {
    "name": "Ann",
    "role": "DevOps Engineer",
    "skills": ["Linux", "Python", "Kubernetes"],
    "active": True,
    "projects": 5
}

# Convert to JSON string
json_str = json.dumps(user, indent=4)
print(json_str)

# Write JSON to file
with open('user.json', 'w') as f:
    json.dump(user, f, indent=4)  # Pretty format
```

dumps() OPTIONS

- `indent=4` Pretty print
- `sort_keys=True` Sort keys
- `ensure_asicii=False` Keep Unicode

3 READING API RESPONSES

Fetch data from APIs and parse JSON responses.

```python
import requests
import json

url = 'https://api.github.com/users/octocat'

response = requests.get(url, timeout=10)
response.raise_for_status()  # Raise error for bad status
data = response.json()       # Parse JSON response

print('User:', data['login'])
print('Name:', data['name'])
print('Public Repos:', data['public_repos'])
print('Followers:', data['followers'])
```

GOOD PRACTICES

- Always use timeouts
- Check status codes
- Handle errors
- Log API responses

Handle errors!

```python
try:
    data = response.json()
except ValueError:
    print('Invalid JSON response')
```

4 CONFIGURATION MANAGEMENT

Store application settings in JSON configuration files.

```json
# config.json
{
  "app": {
    "name": "VERIQTA Automation",
    "version": "1.0.0",
    "debug": false,
    "log_level": "INFO"
  },
  "paths": {
    "log_dir": "/var/log/veriqta",
    "backup_dir": "/backup/veriqta"
  },
  "database": {
    "host": "localhost",
    "port": 5432,
    "user": "veriqta",
    "password": "secret123"
  }
}
```

```python
# Load config
import json

with open('config.json') as f:
    config = json.load(f)

log_dir = config['paths']['log_dir']
db_host = config['database']['host']
debug = config['app']['debug']

print('Log directory:', log_dir)
print('DB Host:', db_host)
print("Debug Mode:', debug)
```

CONFIG TIPS

- Keep configs in version control.
- Use separate configs per environment.
- Do not store secrets in plain text.
- Use environment variables for secrets.

5 PRACTICAL EXAMPLES

1. Pretty Print JSON

```python
import json

with open('data.json') as f:
    data = json.load(f)

print(json.dumps(data, indent=4))
```

2. Filter JSON Data

```python
data = json.loads(json_str)

active_users = [u for u in
 data['users'] if u['active']]

print(active_users)
```

3. Merge JSON Files

```python
import json
with open('a.json') as f:
    a = json.load(f)
with open('b.json') as f:
    b = json.load(f)
merged = {**a, **b}  # Merge
print(json.dumps(merged, indent=2))
```

4. Save API Data

```python
import requests, json

res = requests.get(url)
data = res.json()

with open('api_data.json', 'w') as f:
    json.dump(data, f, indent=2)
```

5. Update JSON Config

```python
with open('config.json') as f:
    cfg = json.load(f)

cfg['app']['version'] = '1.1.0'

with open('config.json', 'w') as f:
    json.dump(cfg, f, indent=4)
```

BEST PRACTICES

Validate JSON
Use `jsonschema` or manual checks to validate data.

Handle Exceptions
Always catch JSON parsing errors.

Use Indent
Readable JSON is easier to maintain.

Avoid Hardcoding
Store data in JSON files or environment variables.

Security First
Never commit secrets or credentials.

VERIQTA

WORKING WITH YAML

YAML (YAML Ain't Markup Language) is a human-readable data serialization format. Python's PyYAML library helps you read, write, and manage YAML files—perfect for DevOps, Kubernetes, and configuration automation.

REMEMBER

- YAML uses indentation (not tabs) for structure.
- Use spaces (2 or 4).
- Consistent indentation is critical.
- YAML is widely used in DevOps and Kubernetes.

1 PYTHON YAML BASICS (PyYAML)

Install PyYAML:

```
pip install pyyaml
```

Import the library:

```
import yaml
```

COMMON FUNCTIONS

- `yaml.safe_load()` # Load YAML
- `yaml.safe_dump()` # Dump YAML
- `yaml.dump()` # Dump with full YAML features

2 READING YAML FILES

Load YAML content from a file.

```
import yaml

with open('config.yaml', 'r') as f:
    data = yaml.safe_load(f)

print(type(data))          # <class 'dict'>
print(data['app']['name']) # VERIQTA
print(data['database']['port']) # 5432
```

EXAMPLE YAML (config.yaml)

```
app:
    name: VERIQTA
    version: 1.0.0
    debug: true
database:
    host: localhost
    port: 5432
    user: veriqta
    password: secret123
features:
    - logging
    - monitoring
    - backups
```

3 WRITING YAML FILES

Convert Python objects into YAML and write to a file.

```
import yaml

data = {
    'app': {
        'name': 'VERIQTA',
        'version': '1.0.0',
        'debug': False
    },
    'database': {
        'host': 'localhost',
        'port': 5432,
        'user': 'veriqta',
        'password': 'secret123'
    },
    'features': ['logging', 'monitoring', 'backups']
}

with open('output.yaml', 'w') as f:
    yaml.safe_dump(data, f, sort_keys=False, indent=2)

print("YAML file written successfully!")
```

DUMP OPTIONS

- `default_flow_style=False` → Block style (preferred)
- `indent=2 or 4` → Number of spaces
- `sort_keys=True/False` → Keep original order
- `allow_unicode=True` → Support Unicode

TIP

Use `safe_dump` for security. Avoid `yaml.dump` with untrusted data.

4 READING KUBERNETES YAML

Kubernetes uses YAML for all resource definitions.

```
import yaml

with open('deployment.yaml') as f:
    k8s_obj = yaml.safe_load(f)

kind = k8s_obj['kind']       # Deployment
metadata = k8s_obj['metadata']
name = metadata['name']      # my-app
replicas = k8s_obj['spec']['replicas']  # 3
image = (k8s_obj['spec']['template']
    ['spec']['containers'][0]['image'])

print(f"Kind: {kind}")
print(f"Name: {name}")
print(f"Replicas: {replicas}")
print(f"Image: {image}")
```

K8S YAML EXAMPLE

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: app
        image: nginx:1.25
```

KUBERNETES TIP

Use `kubectl apply -f file.yaml` to apply YAML configs.

5 CONFIGURATION AUTOMATION

Use YAML to manage environments, applications, and infrastructure.

```
import yaml, os
# Load environment config
with open('env.yaml') as f:
    config = yaml.safe_load(f)
# Use config values in automation
host = config['database']['host']
port = config['database']['port']
debug = config['app']['debug']

print(f"Connecting to {host}:{port}")
print(f"Debug mode: {debug}")
```

WHY YAML?

- ☑ Human-readable
- ☑ Easy version control
- ☑ Widely supported
- ☑ Ideal for environment configs
- ☑ Perfect for infrastructure as code

6 PRACTICAL EXAMPLE: UPDATE YAML VALUE

Read YAML, update a value, and write it back.

```
import yaml

with open('config.yaml') as f:
    data = yaml.safe_load(f)

# Update a value
data['app']['version'] = '1.1.0'
data['debug'] = True

# Write back to file
with open('config.yaml', 'w') as f:
    yaml.safe_dump(data, f, sort_keys=False, indent=2)
print("Configuration updated successfully!")
```

USE CASES

- Environment configs
- Application settings
- Kubernetes manifests
- CI/CD pipelines
- Infrastructure as Code

BEST PRACTICES

- Use `safe_load` to avoid execution risks.
- Always use spaces for indentation.
- Validate YAML before applying to production.
- Store YAML files in version control (Git).
- Keep configs modular and reusable.
- Never expose secrets in YAML files.

VERIQTA

WORKING WITH CSV

CSV (Comma Separated Values) is a simple, widely used file format for storing tabular data. Python's `csv` module makes it easy to read, write, and automate CSV files for daily operations.

REMEMBER

- CSV stores plain text data in rows and columns.
- Always handle headers carefully.
- Validate data before processing.
- Use CSV for reports, inventories and exports.

1 CSV BASICS

Import the CSV module.

```python
import csv

# Open a CSV file for reading
with open('data.csv', 'r', newline='') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)      # Each row is a list
```

COMMON FUNCTIONS

- `csv.reader()` → Read CSV files
- `csv.writer()` → Write CSV files
- `csv.DictReader()` → Read as dictionaries
- `csv.DictWriter()` → Write dictionaries

2 READING CSV FILES

Read CSV data using reader or DictReader.

```python
# Read with header using DictReader
import csv
with open('inventory.csv', 'r', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"Item: {row['name']}, Qty: {row['quantity']},
            Price: {row['price']}")

# Read without header
with open('data.csv', 'r', newline='') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

BEST PRACTICES

- ☑ Use `newline=''` when opening files.
- ☑ Strip spaces from data.
- ☑ Validate required columns.
- ☑ Handle encoding (utf-8).

3 WRITING CSV FILES

Write data to CSV using writer or DictWriter.

```python
# Write simple rows
import csv
data = [
    ['name', 'role', 'department'],
    ['Ann', 'DevOps Engineer', 'IT'],
    ['John', 'SysAdmin', 'Operations']
]
with open('employees.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(data)

# Write dictionaries
fieldnames = ['name', 'role', 'department']
rows = [
    {'name': 'Ann', 'role': 'DevOps Engineer', 'department': 'IT'},
    {'name': 'John', 'role': 'SysAdmin', 'department': 'Operations'}
]
with open('employees_dict.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()    # Write header
    writer.writerows(rows)
```

WRITE OPTIONS

No Header
Use writeheader() for DictWriter.

Append Mode
Use 'a' to append data.

Clean Data
Remove newlines and extra spaces.

4 INVENTORY MANAGEMENT EXAMPLE

Track inventory items in a CSV file.

```python
import csv, os
INVENTORY_FILE = 'inventory.csv'

# Add new item
def add_item(name, quantity, price):
    file_exists = os.path.isfile(INVENTORY_FILE)
    with open(INVENTORY_FILE, 'a', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=['name', 'quantity',
        'price'])
        if not file_exists:
            writer.writeheader()
        writer.writerow({'name': name, 'quantity': quantity,
                         'price': price})

# List all items
def list_items():
    with open(INVENTORY_FILE, 'r', newline='') as f:
        reader = csv.DictReader(f)
        for row in reader:
            print(f"{row['name']} - Qty: {row['quantity']} - ₹{row['price']}")
```

INVENTORY COLUMNS

- ☑ name
- ☑ quantity
- ☑ price
- ☑ date_added
- ☑ supplier

5 REPORTING AUTOMATION

Generate CSV reports for analysis and sharing.

```python
import csv
from datetime import datetime

def generate_report(sales_data, filename):
    fieldnames = ['date', 'product', 'quantity', 'amount']
    with open(filename, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for row in sales_data:
            writer.writerow(row)

# Example sales data
sales = [
    {'date': '2025-07-22', 'product': 'Laptop', 'quantity': 2, 'amount': 2500},
    {'date': '2025-07-22', 'product': 'Mouse', 'quantity': 5, 'amount': 100},
    {'date': '2025-07-22', 'product': 'Keyboard', 'quantity': 3, 'amount': 300}
]

generate_report(sales, f'sales_report_{datetime.now().date()}.csv')
```

REPORT USE CASES

- Daily sales reports
- Inventory summaries
- System exports
- Audit and compliance

6 PRACTICAL EXAMPLES

Quick scripts for common CSV tasks.

```python
# Count total items in inventory
import csv
def count_items():
    with open('inventory.csv', 'r', newline='') as f:
        reader = csv.DictReader(f)
        return sum(int(row['quantity']) for row in reader)
print(f"Total Items: {count_items()}")

# Find items below stock level
def low_stock(threshold):
    with open('inventory.csv', 'r', newline='') as f:
        reader = csv.DictReader(f)
        return [row for row in reader if int(row['quantity']) < threshold]

print("Low Stock Items:")
for item in low_stock(5):
    print(f"{item['name']} - Qty: {item['quantity']}")
```

TIP

Use CSV for simple, portable data that can be opened in Excel, Google Sheets, or any spreadsheet application.

BEST PRACTICES

- Always validate CSV data before processing.
- Handle missing or empty values safely.
- Back up CSV files before modifying.
- Control access to sensitive CSV data.
- Automate reports and exports for consistency.

VERIQTA

REST APIs

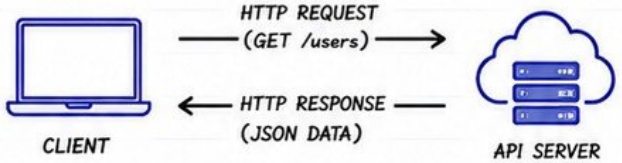
REST (Representational State Transfer) APIs allow systems to communicate over HTTP using standard methods, status codes, and structured data formats like JSON.

REMEMBER

- REST is stateless.
- Resources are identified by URLs.
- Use correct HTTP methods.
- Handle errors gracefully.
- Always check status codes.

1 API FUNDAMENTALS

REST APIs expose resources via endpoints (URLs).



KEY PRINCIPLES

- Client-Server Architecture
- Stateless (each request is independent)
- Cacheable
- Layered System
- Uniform Interface

2 HTTP METHODS

Use the right method for the right action.

METHOD	PURPOSE	EXAMPLE
GET	Retrieve resources	GET /users
POST	Create a new resource	POST /users
PUT	Update entire resource	PUT /users/1
PATCH	Update part of resource	PATCH /users/1
DELETE	Delete a resource	DELETE /users/1

3 AUTHENTICATION

APIs usually require authentication to protect resources.

1) API KEY (Simple and common)

Headers:

```
{ "x-api-key": "YOUR_API_KEY" }
```

2) BEARER TOKEN (JWT or OAuth)

Headers:

```
{ "Authorization": "Bearer YOUR_TOKEN" }
```

Always keep tokens and keys secret!

4 STATUS CODES

APIs return status codes to indicate the result.

CODE	MEANING	WHEN IT HAPPENS
200	OK	Request successful
201	Created	Resource created successfully
400	Bad Request	Invalid request or parameters
401	Unauthorized	Authentication required/failed
403	Forbidden	No permission to access resource
404	Not Found	Resource not found
500	Internal Server Error	Server-side error occurred

5 JSON RESPONSES

APIs commonly return data in JSON format.

```python
import requests

url = 'https://jsonplaceholder.typicode.com/users/1'
response = requests.get(url)

if response.status_code == 200:
    data = response.json()  # Parse JSON response
    print('Name:', data['name'])
    print('Email:', data['email'])

else:
    print("Error:", response.status_code, response.text)
```

SAMPLE JSON RESPONSE

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "leanne@example.com",
  "phone": "1-770-736-8031",
  "website": "hildegard.org"
}
```

6 MAKE API REQUESTS (EXAMPLE)

Perform GET, POST, PUT, DELETE requests.

```python
import requests
base_url = 'https://jsonplaceholder.typicode.com'

# GET
requests.get(f'{base_url}/posts/1')

# POST
requests.post(f'{base_url}/posts', json={'title':'Hello','body': 'API','userId':1})

# PUT
requests.put (f'{base_url}/posts/1', json={'id':1,'title':'Update','body':'Updated','userId':1})

# DELETE
requests.delete(f'{base_url}/posts/1')
```

BEST PRACTICES

- Always use HTTPS for API calls.
- Store API keys and tokens securely.
- Validate inputs before sending requests.
- Implement retries for transient failures.
- Log API requests and responses for monitoring.
- Handle errors and exceptions properly.

VERIQTA

PYTHON REQUESTS LIBRARY

The requests library makes it easy to send HTTP requests and interact with REST APIs. It is simple, powerful, and perfect for automation, integrations, and DevOps workflows.

REMEMBER

- Install: pip install requests
- Always check status codes.
- Handle exceptions.
- Use timeouts in production.
- Never hardcode secrets.

1 GET – Retrieve Data

Use GET to fetch resources from an API.

```python
import requests

url = 'https://jsonplaceholder.typicode.com/users/1'
response = requests.get(url)

print('Status Code :', response.status_code)
print('Response JSON :', response.json())
print('Headers :', response.headers['Content-Type'])
```

NOTES

- GET has no body.
- Used for reading resources.

2 POST – Create Data

Use POST to create a new resource.

```python
import requests
url = 'https://jsonplaceholder.typicode.com/posts'
payload = {
    'title': 'Hello',
    'body': 'This is a new post',
    'userId': 1
}
response = requests.post(url, json=payload)
print('Status Code :', response.status_code)
print('Response JSON :', response.json())
```

TIP

- Use json=payload to send JSON data.
- POST is not idempotent.

3 PUT – Update Entire Resource

Use PUT to update an existing resource completely.

```python
import requests
url = 'https://jsonplaceholder.typicode.com/posts/1'
payload = {
    'id': 1,
    'title': 'Updated Title',
    'body': 'Updated content',
    'userId': 1
}
response = requests.put(url, json=payload)
print('Status Code :', response.status_code)
print('Response JSON :', response.json())
```

NOTE

- PUT replaces the entire resource.
- Send all fields even if only one changes.

4 DELETE – Remove Resource

Use DELETE to remove a resource.

```python
import requests
url = 'https://jsonplaceholder.typicode.com/posts/1'

response = requests.delete(url)

print('Status Code :', response.status_code)
if response.status_code == 200:
    print('Resource deleted successfully')
else:
    print('Failed to delete resource')
```

NOTE

- DELETE should be idempotent.
- Check status code.

5 HEADERS – Send Custom Headers

Headers provide metadata with your request.

```python
import requests
url = 'https://jsonplaceholder.typicode.com/posts'
headers = {
    'User-Agent': 'VERIQTA-Client/1.0',
    'Accept': 'application/json'
}
response = requests.get(url, headers=headers)
print('Status Code :', response.status_code)
print('Content-Type :', response.headers['Content-Type'])
```

COMMON HEADERS

- User-Agent
- Accept
- Content-Type
- Authorization

6 AUTHENTICATION – Secure Your Requests

Add authentication to access protected APIs.

1) BASIC AUTH

```python
from requests.auth import HTTPBasicAuth
url = 'https://api.example.com/protected'
auth = HTTPBasicAuth('user', 'password')
response = requests.get(url, auth=auth)
```

2) BEARER TOKEN (JWT)

```python
headers = {
    'Authorication': 'Bearer YOUR_TOKEN_HERE'
}
response = requests.get(url, headers=headers)
```

SECURITY TIPS

- Store tokens in environment variables.
- Never commit secrets to code.

7 TIMEOUTS – Prevent Hanging Requests

Always set timeouts for reliability.

```python
import requests
url = 'https://api.example.com/data'

# 5 seconds connect timeout, 10 seconds read timeout
response = requests.get(url, timeout=(5, 10))

print('Status Code :', response.status_code)
```

NOTE

- timeout=(connect, read)
- Prevents your script from hanging.

HTTP METHOD SUMMARY

METHOD	PURPOSE	IDEMPOTENT?
GET	Retrieve data	Yes
POST	Create new resource	No
PUT	Update entire resource	Yes
DELETE	Remove resource	Yes

Choose the right method for the right action.

BEST PRACTICES

- Always check status codes.
- Handle exceptions with try/except.
- Use timeouts in production.
- Keep your code clean and modular.
- Log requests and responses.
- Secure credentials and API keys.

VERIQTA

WORKING WITH GIT USING PYTHON

GitPython is a Python library that allows you to interact with **Git** repositories programmatically. You can automate repository tasks, manage branches, read history, and integrate Git into DevOps workflows.

REMEMBER

- GitPython is a wrapper around Git.
- It uses your Git installation.
- Perfect for automation and DevOps scripts.

1 INSTALL AND IMPORT

Install GitPython and import the library.

```
# Install
pip install GitPython

# Import
from git import Repo

# Check GitPython version
import git
print(git.__version__) # e.g., 3.1.43
```

TIP

GitPython requires Git to be installed on your system and available in your PATH.

2 INITIALIZE OR OPEN A REPOSITORY

Create a new repository or open an existing one.

```
from git import Repo

# Initialize a new repository
repo = Repo.init('my-project')
print(f'Initialized repo: {repo.working_dir}')

# Open an existing repository
repo = Repo('my-project')
print(f'Opened repo: {repo.working_dir}')
```

NOTE

The path can be absolute or relative.

3 STAGE AND COMMIT CHANGES

Add files and create commits.

```
repo = Repo('my-project')

# Stage all changes
repo.git.add(A=True)

# Commit
commit = repo.index.commit('Add feature X')
print(f'Committed: {commit.hexsha[:7]} -
  {commit.message.strip()}')
```

NOTE

Use repo.index to stage and commit changes.

4 VIEW COMMIT HISTORY

Read the commit history of a repository.

```
repo = Repo('my-project')

# Iterate over commits
for commit in repo.iter_commits(max_count=5):
  print(f'{commit.hexsha[:7]} {commit.committed_datetime}')
  print(f'Author : {commit.author}')
  print(f'Message: {commit.message.strip()}')
  print('-' * 40)
```

TIP

Use max_count to limit the number of results.

5 BRANCH MANAGEMENT

Create, list, switch, and delete branches.

```
repo = Repo('my-project')

# List branches
print('Branches:')
for branch in repo.branches:
  print(f' - {branch.name}')

# Create new branch
new_branch = repo.create_head('feature/login')
print(f'Created branch: {new_branch.name}')

# Checkout / switch branch
new_branch.checkout())
print(f'Switched to: {repo.active_branch.name}')

# Delete branch
repo.delete_head('feature/login', force=True)
print('Deleted branch: feature/login')
```

QUICK REFERENCE

`repo.branches` → List all branches

`repo.active_branch` → Current branch

`repo.create_head(name)` → Create branch

`branch.checkout()` → Switch branch

`repo.delete_head(name)` → Delete branch

6 MORE USEFUL OPERATIONS

Common Git automation tasks.

```
repo = Repo('my-project')

# Remote operations
origin = repo.remote(name='origin')
origin.pull()        # Pull latest changes
origin.push()        # Push changes

# Get repository status
print(repo.git.status())

# Get current branch
print(f'Current branch: {repo.active_branch.name}')

# Get last commit
head_commit = repo.head.commit
print(f'Last commit: {head_commit.hexsha[:7]}
      {head_commit.message.strip()}')
```

NOTE

Remote operations require proper permissions and authentication.

BEST PRACTICES

- Always validate paths before opening repositories.
- Commit meaningful messages for better traceability.
- Automate branch workflows for features and releases.
- Avoid hardcoding credentials. Use SSH keys or tokens.
- Pull before push to avoid conflicts in automation.

VERIQTA

LINUX ADMINISTRATION AUTOMATION

Python can automate everyday Linux administration tasks. You can manage users, permissions, packages, and services programmatically for consistency, speed, and reliability.

REMEMBER

- Run scripts with appropriate permissions.
- Test on non-production systems first.
- Log every action.
- Automate safely.

1 USER MANAGEMENT

Create, delete, and manage system users.

```python
import subprocess

def create_user(username, shell="/bin/bash"):
    subprocess.run(["useradd", "-m", "-s", shell, username],
                   check=True)
    print(f"User '{username}' created successfully.")

def delete_user(username):
    subprocess.run(["userdel", "-r", username], check=True)
    print(f"User '{username}' deleted.")

def list_users():
    result = subprocess.run(["cut", "-d:", "-f1", "/etc/passwd"],
             capture_output=True, text=True, check=True)
    return result.stdout.strip().split('\n')

# Example
print(list_users())
```

TIP

Use sudo when running scripts that modify users.

2 FILE PERMISSIONS

Change ownership and permissions of files and directories.

```python
import os
import pwd, grp
def set_permission(path, mode):
    os.chmod(path, mode)
    print(f"Permission for {path} set to {oct(mode)}")

def change_owner(path, user, group):
    uid = pwd.getpwnam(user).pw_uid
    gid = grp.getgrnam(group).gr_gid
    os.chown(path, uid, gid)
    print(f"Ownership of {path} changed to {user}:{group}")

def get_permission(path):
    mode = oct(os.stat(path).st_mode)[-3:]
    print(f"Permissions for {path}: {mode}")

# Example
set_permission("/opt/app.log", 0o640)
change_owner("/opt/app.log", "appuser", "appgroup")
```

TIP

Permissions are critical for security.

3 PACKAGE MANAGEMENT

Install, remove, and update packages.

```python
import subprocess

def install_package(package):
    subprocess.run(["apt", "update"], check=True)
    subprocess.run(["apt", "install", "-y", package], check=True)
    print(f"Installed {package}")

def remove_package(package):
    subprocess.run(["apt", "remove", "-y", package], check=True)
    print(f"Removed {package}")

def update_system():
    subprocess.run(["apt", "update"], check=True)
    subprocess.run(["apt", "upgrade", "-y"], check=True)
    print("System updated successfully.")

# Example
install_package("nginx")
update_system()
```

TIP

Supports apt (Debian/Ubuntu). For yum/dnf, adjust the commands.

4 SERVICE MANAGEMENT

Start, stop, restart, and check services.

```python
import subprocess
def start_service(service):
    subprocess.run(["systemctl", "start", service], check=True)
    print(f"{service} started.")

def stop_service(service):
    subprocess.run(["systemctl", "stop", service], check=True)
    print(f"{service} stopped.")

def restart_service(service):
    subprocess.run(["systemctl", "restart", service], check=True)
    print(f"{service} restarted.")

def service_status(service):
    result = subprocess.run(["systemctl", "is-active", service],
                            capture_output=True, text=True)
    status = result.stdout.strip()
    print(f"{service} status: {status}")
    return status

# Example
restart_service("nginx")
service_status("nginx")
```

TIP

Use systemctl for modern Linux distributions.

EXAMPLE: DAILY ADMIN MAINTENANCE SCRIPT

```python
def daily_maintenance():
    print("[1] Updating system...")
    update_system()
    print("[2] Restarting nginx...")
    restart_service("nginx")
    print("[3] Checking disk space...")
    subprocess.run(["df", "-h"])
    print("[4] Completed maintenance.")

if __name__ == "__main__":
    daily_maintenance()
```

WHAT THIS SCRIPT DOES

- Updates packages
- Restarts services
- Checks disk space
- Provides consistent daily operations.

SCHEDULE WITH CRON

Run your script automatically.

```
0 2 * * * /usr/bin/python3 /opt/scripts/maint.py >>
/var/log/maint.log 2>&1
```

BEST PRACTICES

- Always validate inputs and paths.
- Use absolute paths in scripts.
- Log all actions and errors.
- Test scripts in a safe environment before production.
- Run with the least privilege needed.
- Keep backups before making changes.

VERIQTA

AWS AUTOMATION

boto3 is the **AWS SDK for Python**. It allows you to programmatically interact with AWS services to automate infrastructure, manage resources, and build powerful DevOps solutions.

REMEMBER

- Never hardcode AWS credentials. Use IAM roles in production.
- Handle exceptions gracefully.
- Use regions explicitly.
- Tag your resources.

Automate with safety!

1 boto3 OVERVIEW

boto3 provides a high-level interface to AWS services.

```python
# Install boto3
pip install boto3

# Import boto3
import boto3

# Create a session
session = boto3.Session(region_name='us-east-1')

# Create a client for EC2
ec2 = session.client('ec2')

# Create a resource for S3
s3 = session.resource('s3')
```

TIP

Use clients for low-level APIs.

Use resources for high-level operations.

2 AUTHENTICATION

boto3 uses AWS credentials to authenticate requests.

```python
# Method 1: AWS CLI Credentials (recommended)
# Configure once
aws configure

# Boto3 automatically picks up the config
import boto3
s3 = boto3.client('s3')

# Method 2: Explicit credentials
session = boto3.Session(
    aws_access_key_id="AKIAEXAMPLE",
    aws_secret_access_key="SECRETKEYEXAMPLE",
    region_name='us-east-1'
)
ec2 = session.client('ec2')

# Method 3: IAM Role (EC2/ECS/Lambda)
# No code needed, boto3 uses the role automatically
```

BEST PRACTICE

- Use IAM roles in production.
- Avoid long lived keys.
- Rotate keys regularly.

3 AWS SDK (BOTO3) BASICS

Core concepts you will use every day.

Session	Entry point to AWS. Holds config and credentials.
Client	Low-level service interface. One-to-one with APIs.
Resource	High-level interface. Object-oriented abstractions.
Paginator	Iterate through large result sets.
Waiter	Wait for a resource state (e.g., instance running).
Exceptions	Handle errors gracefully with try/except.

4 COMMON PATTERN

A typical boto3 workflow.

Authenticate — Use IAM user, role, or CLI credentials

→ **Create Client** — Initialize the service client or resource

→ **Call API** — Perform the operation using boto3 methods

→ **Process Response** — Handle the response and data

→ **Handle Errors** — Use try/except and log failures

5 COMMON AWS SERVICES WITH BOTO3

Popular services you will automate.

SERVICE	PURPOSE	COMMON OPERATIONS (EXAMPLES)	EXAMPLE CODE (SNIPPET)
EC2	Compute instances	• Run instances • Describe instances • Stop / Start instances	`ec2 = boto3.client('ec2')` `response = ec2.run_instances(ImageId='ami-12345678', InstanceType='t2.micro', MinCount=1, MaxCount=1)`
S3	Object storage	• List buckets • Upload files • Download files	`s3 = boto3.client('s3')` `s3.upload_file('file.txt', 'my-bucket', 'file.txt')`
RDS	Managed databases	• List DB instances • Create DB instance • Modify instance	`rds = boto3.client('rds')` `rds.describe_db_instances()`
IAM	Identity & access management	• Create users • Attach policies • List roles	`iam = boto3.client('iam')` `iam.create_user(UserName='newuser')`
CloudWatch	Monitoring & logs	• Put metrics • Create alarms • Get metrics	`cw = boto3.client('cloudwatch')` `cw.put_metric_data(Namespace='MyApp', MetricData=[...])`

NOTE

This is not a complete list. Boto3 supports 200+ AWS services!

6 BEST PRACTICES

- Use IAM roles instead of long term keys.
- Always specify the AWS region explicitly.
- Tag resources for cost tracking and organization.
- Handle exceptions and retry with exponential backoff.
- Log all actions and monitor CloudWatch.
- Keep scripts idempotent and safe.

VERIQTA

MANAGING EC2 WITH PYTHON

With boto3, you can automate the full lifecycle of EC2 instances. This page covers launching, describing, stopping instances, using tags, and security best practices.

REMEMBER

- Protect your credentials.
- Use least privilege IAM roles.
- Tag everything.
- Stop or terminate unused instances.
- Automate safely!

1 LAUNCH EC2 INSTANCE

Launch a new EC2 instance.

```python
import boto3
ec2 = boto3.client('ec2', region_name='us-east-1')
# Launch instance
response = ec2.run_instances(
    ImageId='ami-0c02fb55956c7d316',
    InstanceType='t2.micro',
    MinCount=1,
    MaxCount=1,
    KeyName='my-keypair',
    SecurityGroupIds=['sg-12345678'],
    SubnetId='subnet-12345678',
    TagSpecifications=[{
        'ResourceType': 'instance',
        'Tags': [{'Key': 'Name', 'Value': 'PythonAutoServer'}]
    }]
)
instance_id = response['Instances'][0]['InstanceId']
print(f"Launched Instance ID: {instance_id}")
```

TIP

Always use key pairs and security groups for secure access.

2 DESCRIBE INSTANCES

List and describe your EC2 instances.

```python
import boto3
ec2 = boto3.client('ec2')
# Describe all instances
response = ec2.describe_instances(
    Filters=[{'Name': 'instance-state-name',
              'Values': ['pending','running','stopping','stopped']}]
)
for reservation in response['Reservations']:
    for instance in reservation['Instances']:
        print(f" Instance ID: {instance['InstanceId']}")
        print(f" State      : {instance['State']['Name']}")
        print(f" Type       : {instance['InstanceType']}")
        print(f" Public IP  : {instance.get('PublicIpAddress','N/A')}")
        print(f" Name Tag
            : {next((t['Value'] for t in instance.get('Tags',[])
                     if t['Key']=='Name'), 'N/A')}")
    print('--' * 60)
```

TIP

Use filters to get specific instances or states.

3 STOP INSTANCES

Stop one or more running instances.

```python
import boto3
ec2 = boto3.client('ec2')

# Stop instances
instance_ids = ['i-0123456789abcdef0']

response = ec2.stop_instances(InstanceIds=instance_ids)
print('Stopping instances:', instance_ids)

for code in response['StoppingInstances']:
    print(f"- {code['InstanceId']} => {code['CurrentState']['Name']}")
```

TIP

Stopping saves cost while keeping data intact.

4 MANAGE TAGS

Add, update, or remove tags from instances.

```python
import boto3
ec2 = boto3.client('ec2')
# Add or update tags
ec2.create_tags(
    Resources=['i-0123456789abcdef0'],
    Tags=[{'Key': 'Environment', 'Value': 'Production'},
          {'Key': 'Owner', 'Value': 'DevOps'}]
)
# List tags
response = ec2.describe_tags(
    Filters=[{'Name': 'resource-id',
              'Values': ['i-0123456789abcdef0']}] )
for tag in response['Tags']:
    print(f"{tag['Key']}: {tag['Value']}")
```

TIP

Tag your resources for cost tracking and organization.

5 SECURITY CONSIDERATIONS

- Use IAM roles instead of access keys where possible.
- Grant least privilege permissions (only what is needed).
- Store credentials securely (AWS Secrets Manager or environment variables).
- Use security groups instead of opening all ports.

SECURE ARCHITECTURE WITH PYTHON + EC2

- Keep AMIs and software updated.
- Enable CloudWatch monitoring.
- Terminate unused resources.
- Never commit credentials to code repositories.

6 QUICK REFERENCE: COMMON EC2 ACTIONS WITH BOTO3

ACTION	BOTO3 METHOD	DESCRIPTION	EXAMPLE
Launch Instance	ec2.run_instances()	Create and launch a new EC2 instance	ec2.run_instances(ImageId='ami-xxxx', MinCount=1, MaxCount=1, ...)
Describe Instances	ec2.describe_instances()	Get details of instances	ec2.describe_instances()
Stop Instance(s)	ec2.stop_instances()	Stop running instances	ec2.stop_instances(InstanceIds=['i-123456'])
Start Instance(s)	ec2.start_instances()	Start stopped instances	ec2.start_instances(InstanceIds=['i-123456'])
Terminate Instance(s)	ec2.terminate_instances()	Terminate instances permanently	ec2.terminate_instances(InstanceIds=['i-123456'])
Add/Update Tags	ec2.create_tags()	Add or update tags	ec2.create_tags(Resources=['i-123456'], Tags=[{'Key':'Name','Value':'Web'}])
Describe Tags	ec2.describe_tags()	List tags for resources	ec2.describe_tags(Filters=[{'Name':'resource-id','Values':['i-123456']}])

VERIQTA

MANAGING S3 WITH PYTHON

With boto3, you can automate every common task in Amazon S3. This page covers uploading, downloading, deleting objects, managing buckets, and enabling versioning.

REMEMBER

- Protect your credentials.
- Use least privilege IAM roles.
- Version your important data.
- Encrypt sensitive objects.
- Clean up unused resources.
- Automate safely!

1 UPLOAD OBJECTS

Upload a file to an S3 bucket.

```python
import boto3
s3 = boto3.client('s3')

# Upload file
s3.upload_file(
    Filename='local-file.txt',
    Bucket='my-bucket',
    Key='folder/local-file.txt',
    ExtraArgs={'ContentType': 'text/plain'}
)

print('File uploaded successfully!')
```

TIP

Use upload_file for large files. It handles multipart uploads automatically.

2 DOWNLOAD OBJECTS

Download a file from S3 to your local machine.

```python
import boto3
s3 = boto3.client('s3')

# Download file
s3.download_file(
    Bucket='my-bucket',
    Key='folder/local-file.txt',
    Filename='downloaded-file.txt'
)

print('File downloaded successfully!')
```

TIP

Use download_file for large files. It supports resuming downloads.

3 DELETE OBJECTS

Delete a single object or multiple objects.

Delete a single object

```python
import boto3
s3 = boto3.client('s3')

s3.delete_object(
    Bucket='my-bucket',
    Key='folder/local-file.txt'
)

print('Object deleted.')
```

Delete multiple objects

```python
import boto3
s3 = boto3.client('s3')
objects = [
    {'Key': 'file1.txt'},
    {'Key': 'file2.txt'}
]
s3.delete_objects(
    Bucket='my-bucket',
    Delete={'Objects':objects}
)

print('Objects deleted.')
```

TIP

Use delete_objects to delete up to 1000 objects in a single request.

4 MANAGE BUCKETS

Create, list, and delete S3 buckets.

Create Bucket

```python
import boto3
s3 = boto3.client('s3')
s3.create_bucket(
    Bucket='my-new-bucket',
    CreateBucketConfiguration={
        'LocationConstraint':
        'us-east-1'
    }
)
print('Bucket created.')
```

List Buckets

```python
import boto3
s3 = boto3.client('s3')
response = s3.list_buckets()

for bucket in response["Buckets"]:
    print(bucket['Name'])
```

Delete Bucket

```python
import boto3
s3 = boto3.client('s3')
s3.delete_bucket(Bucket='my-bucket')
print('Bucket deleted.')
```

TIP

Bucket names must be unique globally. Choose simple and meaningful names.

5 ENABLE VERSIONING

Enable versioning on a bucket to protect your data.

```python
import boto3
s3 = boto3.client('s3')

s3.put_bucket_versioning(
    Bucket='my-bucket',
    VersioningConfiguration={
        'Status': 'Enabled'
    }
)

print('Versioning enabled.')
```

TIP

Versioning protects against accidental deletes and overwrites. You can restore older versions.

6 LIST OBJECTS IN A BUCKET

List objects with optional prefix.

```python
import boto3
s3 = boto3.client('s3')
response = s3.list_objects_v2(
    Bucket='my-bucket',
    Prefix='folder/'
)
if 'Contents' in response:
    for obj in response['Contents']:
        print(f"{obj['Key']} - {obj['Size']} bytes")
else:
    print('No objects found.')
```

TIP

Use Prefix to list objects in a folder-like structure.

S3 OPERATIONS QUICK REFERENCE

OPERATION	BOTO3 METHOD	DESCRIPTION	EXAMPLE
Upload Object	s3.upload_file()	Upload a file to S3	s3.upload_file('file.txt', 'my-bucket', 'folder/file.txt')
Download Object	s3.download_file()	Download a file from S3	s3.download_file('my-bucket', 'file.txt', 'file.txt')
Delete Object	s3.delete_object()	Delete a single object	s3.delete_object(Bucket='my-bucket', Key='file.txt')
Create Bucket	s3.create_bucket()	Create a new bucket	s3.create_bucket(Bucket='my-bucket')
List Buckets	s3.list_buckets()	List all buckets	s3.list_buckets()
Enable Versioning	s3.put_bucket_versioning()	Enable bucket versioning	s3.put_bucket_versioning(Bucket='my-bucket', VersioningConfiguration={'Status': 'Enabled'})

BEST PRACTICES

- Use IAM roles instead of access keys.
- Enable versioning for critical data.
- Encrypt data using SSE-S3 or SSE-KMS.
- Lifecycle policies to manage old objects.
- Monitor access with CloudTrail and S3 logs.
- Automate cleanup to reduce costs.

VERIQTA

CLOUDWATCH AUTOMATION

With boto3, you can read CloudWatch metrics, create alarms, manage log groups, and build automated monitoring solutions. This page covers core automation tasks.

REMEMBER

- Use least privilege IAM roles.
- Monitor what matters.
- Set meaningful thresholds.
- Test alarms regularly.
- Clean up old logs.
- Automate responsibly!

① READING METRICS

Retrieve CloudWatch metrics programmatically.

```python
import boto3
import datetime

cw = boto3.client('cloudwatch')

end = datetime.datetime.utcnow()
start = end - datetime.timedelta(minutes=30)

response = cw.get_metric_statistics(
  Namespace='AWS/EC2',
  MetricName='CPUUtilization',
  Dimensions=[{'Name': 'InstanceId',
               'Value': 'i-0123456789abcdef0'}],
  StartTime=start,
  EndTime=end,
  Period=300,
  Statistics=['Average']
)
datapoints = response["Datapoints"]
for d in sorted(datapoints, key=lambda x: x['Timestamp']):
  print(f"{d['Timestamp']} - {d['Average']} %")
```

TIP

Use appropriate Period and Statistics for accurate insights.

② CREATING ALARMS

Create CloudWatch alarms to alert on issues.

```python
import boto3
cw = boto3.client('cloudwatch')

response = cw.put_metric_alarm(
  AlarmName='HighCPUAlarm',
  AlarmDescription='Alarm when CPU > 80% for 5 minutes',
  Namespace='AWS/EC2',
  MetricName='CPUUtilization',
  Dimensions=[{'Name': 'InstanceId',
               'Value': 'i-0123456789abcdef0'}],
  Statistic='Average',
  Period=300,
  EvaluationPeriods=2,
  Threshold=80.0,
  ComparisonOperator='GreaterThanThreshold',
  AlarmActions=['arn:aws:sns:us-east-1:123456789012:MyTopic'],
  OKActions=['arn:aws:sns:us-east-1:123456789012:MyTopic'],
  TreatMissingData='notBreaching'
)

print(f"Alarm created: {response['AlarmArn']}")
```

TIP

Use SNS topics for notifications and test your alarms.

③ MANAGING LOG GROUPS

Create, describe, list, and delete log groups.

Create Log Group

```python
import boto3
logs = boto3.client('logs')
logs.create_log_group(
  logGroupName='/app/myapp',
  kmsKeyId='arn:aws:kms:us-east-1:
   123456789012:key/abcd-1234'
)
print('Log group created.')
```

Describe Log Groups

```python
response = logs.describe_log_groups(
  logGroupNamePrefix='/app/'
)
for lg in response['logGroups']:
  print(lg['logGroupName'],
    lg['creationTime'])
```

List Log Groups

```python
response = logs.describe_log_groups()
for lg in response['logGroups']:
  print(lg['logGroupName'])
```

Delete Log Group

```python
logs.delete_log_group(
  logGroupName='/app/myapp')
)
print('Log group deleted.')
```

TIP

Use meaningful names and retention policies for cost control.

④ EVENT MONITORING

Monitor CloudWatch Events (EventBridge) programmatically.

```python
import boto3
events = boto3.client('events')

# List rules
rules = events.list_rules()["Rules']
for rule in rules:
    print(f"{rule['Name']} - {rule['State']}")

# Describe a rule
name = 'DailyBackupRule'
rule = events.describe_rule(Name=name)['Rule']
print('Description:', rule['Description'])
print('Schedule:', rule.get('ScheduleExpression', 'N/A'))

# List targets for a rule
targets = events.list_targets_by_rule(Rule=name)['Targets']
for t in targets:
    print(f"  Target: {t['Id']} -> {t['Arn']}")
```

TIP

Use EventBridge rules for scheduling and real-time event-driven automation.

CLOUDWATCH AUTOMATION WORKFLOW

Your AWS Resources (EC2, RDS, Lambda, etc.) → Send Metrics to CloudWatch → Read Metrics with boto3 → Create Alarms for Thresholds → Notifications via SNS / Email → Automate Actions & Respond

CLOUDWATCH AUTOMATION BEST PRACTICES

- Use IAM roles with least privilege.
- Monitor key metrics only. Avoid noise.
- Set appropriate thresholds to reduce alerts.
- Review and test alarms regularly.
- Set log retention policies to control costs.
- Automate monitoring, not just alerts.

VERIQTA

DOCKER AUTOMATION

With the Docker SDK for Python, you can automate container, image, volume, and network management.
This page covers core Docker automation tasks.

REMEMBER

- Automate repetitive Docker tasks.
- Use least privilege.
- Handle errors gracefully.
- Clean up resources.
- Automate responsibly!

① DOCKER SDK OVERVIEW

Docker SDK (docker-py) allows you to control Docker Engine via Python.

```python
import docker

client = docker.from_env() # Connect to Docker Engine

# Test connection
print('Docker Version:', client.version()['Version'])
print('API Version:', client.version()['ApiVersion'])
```

TIP

from_env() connects using environment settings like DOCKER_HOST.

② WORKING WITH CONTAINERS

Create, start, stop, restart, list, and remove containers.

```python
import docker
client = docker.from_env()

# Run a container
container = client.containers.run(
    'nginx',
    name='web',
    detach=True,
    ports={'80/tcp': 8080}
)
print('Container started:', container.name)

# Stop a container
container.stop()

# Start a container
container.start()

# Remove a container
container.remove(force=True)
```

TIP

Use detach=True to run containers in the background. Use force=True to remove running containers safely.

③ WORKING WITH IMAGES

Pull, list, and remove Docker images.

```python
import docker
client = docker.from_env()

# Pull an image
image = client.images.pull('nginx:latest')
print('Image pulled:', image.tags[0])

# List images
images = client.images.list()
for img in images:
    print(img.id[:12], img.tags)

# Remove an image
client.images.remove('nginx:latest', force=True)
```

TIP

Always pull the specific image version you need.

④ WORKING WITH VOLUMES

Create, list, and remove Docker volumes.

```python
import docker
client = docker.from_env()

# Create a volume
volume = client.volumes.create(name='data-volume')
print('Volume created:', volume.name)

# List volumes
volumes = client.volumes.list()
for vol in volumes:
    print(vol.name)

# Remove a volume
volume.remove(force=True)
```

TIP

Volumes persist data outside containers. Use named volumes for production.

⑤ WORKING WITH NETWORKS

Create, list, and remove Docker networks.

```python
import docker
client = docker.from_env()

# Create a network
network = client.networks.create('app-network', driver='bridge')
print('Network created:', network.name)

# List networks
networks = client.networks.list()
for net in networks:
    print(net.name, net.driver)

# Remove a network
network.remove()
```

TIP

Custom networks enable container communication and isolation.

⑥ COMBINED EXAMPLE

Run a container with volume and network.

```python
import docker
client = docker.from_env()
# Create volume and network
volume = client.volumes.create('app-data')
network = client.networks.create('app-net')

# Run container with volume and network
container = client.containers.run(
    'nginx',
    name='web-app',
    detach=True,
    ports={'80/tcp': 8080},
    volumes={'app-data': {'bind': '/usr/share/nginx/html', 'mode': 'rw'} },
    network='app-net'
)

print('Container running:', container.name)
```

TIP

Combine volumes and networks for stateful and connected applications.

DOCKER SDK QUICK REFERENCE

TASK	METHOD	DESCRIPTION	EXAMPLE
Run Container	client.containers.run()	Create and start a container	client.containers.run('nginx', detach=True)
List Containers	client.containers.list()	List containers	client.containers.list(all=True)
Stop Container	container.stop()	Stop a running container	container.stop()
Remove Container	container.remove()	Remove a container	container.remove(force=True)
Pull Image	client.images.pull()	Pull an image from registry	client.images.pull('nginx:latest')
Create Volume	client.volumes.create()	Create a volume	client.volumes.create('data')
Create Network	client.networks.create()	Create a network	client.networks.create('app-net')

BEST PRACTICES

- Reuse containers; do not recreate unnecessarily.
- Clean up stopped containers, unused images, volumes, networks.
- Handle exceptions and timeouts.
- Use named volumes for data persistence.
- Use custom networks for application communication.

VERIQTA

CONTAINER MANAGEMENT SCRIPTS

Automate everyday container tasks using the Docker SDK for Python.
This page covers starting, stopping, restarting containers, and building cleanup automation.

REMEMBER

- Automate repetitive tasks.
- Handle errors gracefully.
- Add logging for visibility.
- Clean up unused resources.
- Test your scripts safely!

1 STARTING CONTAINERS

Start one or more stopped containers.

```python
import docker
client = docker.from_env()

# Start a single container
container = client.containers.get('web-app')
container.start()
print(f"Started container: {container.name}")

# Start multiple containers
names = ['web-app', 'db', 'redis']
for name in names:
  try:
      container = client.containers.get(name)
      container.start()
      print(f"Started: {name}")
  except docker.errors.NotFound:
      print(f"Container not found: {name}")
```

TIP

Use container names for readability instead of long IDs.

2 STOPPING CONTAINERS

Stop running containers gracefully.

```python
import docker
client = docker.from_env()

# Stop a single container
container = client.containers.get('web-app')
container.stop(timeout=10)  # wait max 10 seconds
print(f"Stopped container: {container.name}")

# Stop multiple containers
for container in client.containers.list(filters={'status':'running'}):
  try:
      container.stop(timeout=10)
      print(f"Stopped: {container.name}")
  except docker.errors.APIError as e:
      print(f"Error stopping {container.name}: {e}")
```

TIP

Stop uses SIGTERM by default. Containers have 10 seconds to shut down.

3 RESTARTING CONTAINERS

Restart containers without manual intervention.

```python
import docker
client = docker.from_env()

# Restart a single container
container = client.containers.get('web-app')
container.restart(timeout=10)
print(f"Restarted container: {container.name}")

# Restart all containers (optional)
for container in client.containers.list(filters={'status':'running'}):
  try:
      container.restart(timeout=10)
      print(f"Restarted: {container.name}")
  except docker.errors.APIError as e:
      print(f"Error restarting {container.name}: {e}")
```

TIP

Restart is a stop + start sequence. Useful for app updates. Keeps the ID unchanged.

4 GET CONTAINER STATUS

Check the status of containers programmatically.

```python
import docker
client = docker.from_env()

# Get status of a single container
container = client.containers.get('web-app')
print(f"{container.name}: {container.status}")

# List all containers with status
containers = client.containers.list(all=True)
for c in containers:
    status = c.status
    print(f"{c.name:20}      {status:10}    {c.id[:12]}")

# Status values: created, restarting, running,
# removing, paused, exited, dead
```

TIP

Status changes frequently. Perfect for monitoring scripts.

5 CLEANUP AUTOMATION

Remove stopped containers, unused images, unused volumes, and networks.

Cleanup stopped containers

```python
import docker
client = docker.from_env()

# Remove all stopped containers
stopped = client.containers.list(filters={'status':'exited'})
for c in stopped:
    print(f"Removing stopped container: {c.name}")
    c.remove()
```

Cleanup unused images

```python
images = client.images.prune()
print(f"Images removed: {images['ImagesDeleted']}")
```

Cleanup unused volumes

```python
volumes = client.volumes.prune()
print(f"Volumes removed: {volumes['VolumesDeleted']}")
```

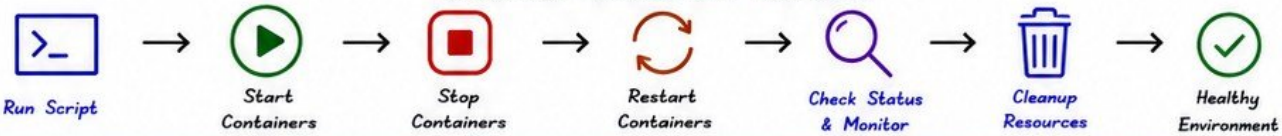
Cleanup unused networks

```python
networks = client.networks.prune()
print(f"Networks removed: {networks['NetworksDeleted']}")
```

TIP

Run cleanup during low traffic periods to avoid impact. Use care in production!

CONTAINER MANAGEMENT WORKFLOW



DOCKER COMMAND EQUIVALENTS

TASK	DOCKER CLI	PYTHON SDK (DOCKER SDK)
Start Container	`docker start <name>`	`container.start()`
Stop Container	`docker stop <name>`	`container.stop()`
Restart Container	`docker restart <name>`	`container.restart()`
List Containers	`docker ps -a`	`client.containers.list(all=True)`
Remove Container	`docker rm <name>`	`container.remove()`
Prune System	`docker system prune`	`client.containers.prune()` / `images.prune()`

BEST PRACTICES

- Always handle exceptions (NotFound, APIError, Timeout).
- Use timeouts for stop/restart to avoid hanging scripts.
- Log all actions to a file for troubleshooting.
- Do not hardcode container names; use variables or config files.
- Test scripts in a safe environment before using in production.

VERIQTA

KUBERNETES PYTHON CLIENT

The Kubernetes Python Client (kubernetes library) lets you interact with Kubernetes clusters programmatically. This page covers installation, authentication, cluster access, and basic operations.

REMEMBER

- Keep kubeconfig secure.
- Use service accounts in production.
- Handle exceptions gracefully.
- Close API clients after use.
- Test in a safe environment first!

1 INSTALLING THE CLIENT

Install the official Kubernetes Python client.

```
# Install using pip
pip install kubernetes

# Verify installation
python -c "import kubernetes; '
 print(kubernetes.__version__)"

# Example output:
# 29.0.0
```

TIP

Keep the client updated for new features and bug fixes.

2 AUTHENTICATION

Authenticate to Kubernetes using kubeconfig or in-cluster config.

```
from kubernetes import config

# Option 1: Use kubeconfig (default location)
config.load_kube_config()  # Uses ~/.kube/config

# Option 2: Use specific kubeconfig file
config.load_kube_config(config_file='/path/to/config')

# Option 3: In-cluster config (for Pods running in k8s)
config.load_incluster_config()
```

TIP

Use service accounts with least privilege in production environments.

3 ACCESSING CLUSTERS

Load configuration and create API clients.

```
from kubernetes import client, config

# Load configuration
config.load_kube_config()

# Create API clients
core_v1 = client.CoreV1Api()
apps_v1 = client.AppsV1Api()
net_v1 = client.NetworkingV1Api()
custom = client.CustomObjectsApi()
```

TIP

Create only the clients you need. Reuse clients instead of creating new ones repeatedly.

4 LIST NAMESPACES AND NODES

Retrieve namespaces and nodes in the cluster.

```
from kubernetes import client
core_v1 = client.CoreV1Api()

# List namespaces
ns_list = core_v1.list_namespace()
for ns in ns_list.items:
    print(ns.metadata.name)

# List nodes
node_list = core_v1.list_node()
for node in node_list.items:
    print(f"{node.metadata.name} - {node.status.node_info.os_image}")
```

TIP

Great for discovery and inventory automation.

5 GET AND DESCRIBE PODS

Get pods in a namespace and view details.

```
from kubernetes import client
core_v1 = client.CoreV1Api()
namespace = "default"
# List pods
pods = core_v1.list_namespaced_pod(namespace=namespace)
for pod in pods.items:
    print(f"{pod.metadata.name} - {pod.status.phase}")

# Get details of a specific pod
pod_name = "my-pod"
pod = core_v1.read_namespaced_pod(name=pod_name,namespace=namespace)
print(pod)
```

TIP

Pod phases: Pending, Running, Succeeded, Failed, Unknown.

6 CREATE A NAMESPACE

Create resources programmatically.

```
from kubernetes import client
core_v1 = client.CoreV1Api()
ns_body = client.V1Namespace(
    metadata=client.V1ObjectMeta(name='demo-ns')
)
try:
    core_v1.create_namespace(body=ns_body)
    print("Namespace 'demo-ns' created successfully")
except client.exceptions.ApiException as e:
    if e.status == 409:
        print("Namespace already exists")
    else:
        print(f"Error: {e}")
```

TIP

Use try/except for clean error handling and idempotent scripts.

COMMON BASIC OPERATIONS

- **List Resources** — Get list of pods, services, nodes, etc.
- **Get Resource** — Retrieve details of a specific resource.
- **Create Resource** — Create resources like pods, services, configmaps.
- **Update Resource** — Update labels, images, replicas, etc.
- **Delete Resource** — Delete resources gracefully.
- **Watch Resource** — Watch for changes in real time.

QUICK REFERENCE

TASK	API CLIENT	METHOD	DESCRIPTION
List Pods	CoreV1Api	list_namespaced_pod()	List all pods in a namespace
Get Pod	CoreV1Api	read_namespaced_pod()	Get details of a pod
Create Pod	CoreV1Api	create_namespaced_pod()	Create a pod
Delete Pod	CoreV1Api	delete_namespaced_pod()	Delete a pod
List Deployments	AppsV1Api	list_namespaced_deployment()	List deployments
Scale Deployment	AppsV1Api	patch_namespaced_deployment_scale()	Scale deployment replicas
List Services	CoreV1Api	list_namespaced_service()	List services
Get Logs	CoreV1Api	read_namespaced_pod_log()	Get pod logs

BEST PRACTICES

- Store kubeconfig securely and do not hardcode paths.
- Use in-cluster config for workloads running inside Kubernetes.
- Apply least privilege with RBAC and service accounts.
- Handle API exceptions and retries gracefully.
- Close API clients if creating many or long-running scripts.
- Log actions for observability and debugging.

VERIQTA

MANAGING KUBERNETES RESOURCES

With the Kubernetes Python Client, you can create, update, delete, and manage all core Kubernetes resources programmatically. This page covers Pods, Deployments, Services, Namespaces, ConfigMaps, and Secrets.

REMEMBER

- Keep kubeconfig secure.
- Use namespaces to isolate resources.
- Store sensitive data in Secrets, not in code.
- Handle exceptions and log all API actions.

① PODS

Create, list, and delete Pods.

```python
from kubernetes import client
v1 = client.CoreV1Api()

# Create a Pod
pod = client.V1Pod(
    metadata=client.V1ObjectMeta(name='demo-pod'),
    spec=client.V1PodSpec(
        containers=[client.V1Container(
            name='nginx',
            image='nginx:latest',
            ports=[client.V1ContainerPort(container_port=80)]
        )
    )
)
v1.create_namespaced_pod(namespace='default', body=pod)

# List Pods
pods = v1.list_namespaced_pod(namespace='default')
for p in pods.items:
    print(p.metadata.name, ' - ', p.status.phase)

# Delete Pod
v1.delete_namespaced_pod(name='demo-pod', namespace='default')
```

TIP

Pods are ephemeral. They can be replaced at any time.

② DEPLOYMENTS

Create, update, and scale Deployments.

```python
from kubernetes import client, apps
apps_v1 = client.AppsV1Api()

# Create Deployment
dep = client.V1Deployment(
    metadata=client.V1ObjectMeta(name='web-app'),
    spec=client.V1DeploymentSpec(
        replicas=3,
        selector=client.V1LabelSelector(match_labels={'app':'web'}),
        template=client.V1PodTemplateSpec(
            metadata=client.V1ObjectMeta(labels={'app':'web'}),
            spec=client.V1PodSpec(
                containers=[client.V1Container(
                    name='nginx',
                    image='nginx:latest',
                    ports=[client.V1ContainerPort(container_port=80)]
                )
            )
        )
    )
)
apps_v1.create_namespaced_deployment(namespace='default', body=dep)

# Scale Deployment
apps_v1.patch_namespaced_deployment_scale(
  name='web-app', namespace='default', body={'spec':{'replicas':5}})
```

TIP

Deployments ensure desired state and enable rolling updates.

③ SERVICES

Expose your application inside or outside the cluster.

```python
from kubernetes import client
v1 = client.CoreV1Api()

# Create Service (ClusterIP)
svc = client.V1Service(
  metadata=client.V1ObjectMeta(name='web-service'),
  spec=client.V1ServiceSpec(
    selector={'app':'web'},
    ports=[client.V1ServicePort(port=80, target_port=80)],
    type='ClusterIP'
)
v1.create_namespaced_service(namespace='default', body=svc)

# List Services
svcs = v1.list_namespaced_service(namespace='default')
for p in svcs.items:
  print(p.metadata.name, ' - ', s.spec.type, ' - ', s.spec.cluster_ip)
```

TIP

Use LoadBalancer or NodePort type to expose services externally.

④ NAMESPACES

Create, list, and delete Namespaces.

```python
from kubernetes import client
v1 = client.CoreV1Api()

# Create Namespace
ns = client.V1Namespace(
   metadata=client.V1ObjectMeta(name='dev')
)
v1.create_namespace(ns)

# List Namespaces
nss = v1.list_namespace()
for n in nss.items:
    print(n.metadata.name)

# Delete Namespace
v1.delete_namespace(name='dev')
```

TIP

Use namespaces to isolate teams and environments.

⑤ CONFIGMAPS

Store non-sensitive configuration as key-value pairs.

```python
from kubernetes import client
v1 = client.CoreV1Api()

# Create ConfigMap
cm = client.V1ConfigMap(
   metadata=client.V1ObjectMeta(name='app-config'),
   data={'APP_ENV':'production', 'DEBUG':'false', 'TIMEOUT':'30'}
)

v1.create_namespaced_config_map(namespace='default', body=cm)

# Get ConfigMap
cfg = v1.read_namespaced_config_map(name='app-config', namespace='default')
print(cfg.data)
```

TIP

ConfigMaps are ideal for application settings and feature flags.

⑥ SECRETS

Store sensitive data such as passwords, tokens, and keys.

```python
from kubernetes import client, base64
v1 = client.CoreV1Api()

# Create Secret (Opaque)
secret = client.V1Secret(
   metadata=client.V1ObjectMeta(name='db-secret'),
   type='Opaque',
   data={
      'username': base64.b64encode(b'admin').decode('utf-8'),
      'password': base64.b64encode(b'secretpass').decode('utf-8')
   }
)
v1.create_namespaced_secret(namespace='default', body=secret)
# Read Secret
sec = v1.read_namespaced_secret(name='db-secret', namespace='default')
decoded = {k: base64.b64decode(v).decode('utf-8') for k,v in sec.data.items()}
print(decoded)
```

TIP

Never store secrets in code or logs. Always use Secrets.

RESOURCE MANAGEMENT WORKFLOW

BEST PRACTICES

- ✓ Use namespaces to organize and isolate resources.
- ✓ Keep ConfigMaps for non-sensitive data only.
- ✓ Store credentials and tokens in Secrets.
- ✓ Use labels and selectors consistently.
- ✓ Handle API exceptions and retries gracefully.

VERIQTA

MONITORING KUBERNETES

Monitor cluster health, podstatus, events, and resource usage using the Kubernetes Python Client and automation.
This page covers key monitoring tasks and automation scripts.

REMEMBER

- Monitor early, act faster.
- Automate responses.
- Keep your cluster healthy.
- Use metrics and events for better insights.

1 CLUSTER HEALTH

Check overall cluster health using the API server.

```python
from kubernetes import client
v1 = client.CoreV1Api()

def cluster_health():
    nodes = v1.list_node().items
    ready = sum(1 for n in nodes if all(c.type == 'Ready' and c.status == 'True'
                                        for c in n.status.conditions))
    total = len(nodes)
    return {'ready': ready, 'total': total, 'healthy': ready == total}

print(cluster_health())
```

TIP

Checks if all nodes are Ready.

2 POD STATUS

Monitor status of all pods in the cluster.

```python
from kubernetes import client
v1 = client.CoreV1Api()

pods = v1.list_pod_for_all_namespaces().items
status_summary = {}
for pod in pods:
    phase = pod.status.phase
    status_summary[phase] = status_summary.get(phase, 0) + 1

print(status_summary)
```

TIP

Track pod phases like Running, Pending, Failed, etc.

3 EVENTS

View recent events across the cluster.

```python
from kubernetes import client
v1 = client.CoreV1Api()

events = v1.list_event_for_all_namespaces(limit=20)

for ev in events.items:
    print(f"[{ev.last_timestamp}] {ev.metadata.namespace}/
          f"{ev.involved_object.kind}/{ev.involved_object.name} - "
          f"{ev.reason}: {ev.message}")
```

TIP

Events help troubleshoot issues quickly.

4 RESOURCE USAGE

Monitor resource usage for nodes and pods.

```python
from kubernetes import client
from kubernetes.client.rest import ApiException
v1 = client.CoreV1Api()

nodes = v1.list_node().items
for node in nodes:
    alloc = node.status.allocatable
    usage = node.status.capacity
    print(f"Node: {node.metadata.name}")
    print(f"  CPU: {usage['cpu']} / {alloc['cpu']}")
    print(f"  Memory: {usage['memory']} / {alloc['memory']}")
```

TIP

Track CPU and memory usage to prevent resource exhaustion.

5 AUTOMATION SCRIPT – MONITOR AND ALERT

Run checks and alert on unhealthy conditions or high resource usage.

```python
from kubernetes import client
import smtplib
from email.mime.text import MIMEText
v1 = client.CoreV1Api()

def send_alert(subject, message):
    msg = MIMEText(message)
    msg['Subject'] = subject
    msg['From'] = 'k8s-monitor@example.com'
    msg['To'] = 'admin@example.com'
    with smtplib.SMTP('localhost') as server:
        server.send_message(msg)

def check_cluster():
    nodes = v1.list_node().items
    for node in nodes:
        for cond in node.status.conditions:
            if cond.type == 'Ready' and cond.status != 'True':
                send_alert('Node Not Ready', f'Node {node.metadata.name} is not Ready')
    pods = v1.list_pod_for_all_namespaces().items
    for pod in pods:
        if pod.status.phase == 'Failed':
            send_alert('Pod Failed', f'Pod {pod.metadata.name} is Failed')
check_cluster()
```

TIP

Automate monitoring and alerting to respond faster to issues.

COMMON ALERT CONDITIONS

- ☑ Node Not Ready
- ☑ Pod Failed
- ☑ High CPU Usage
- ☑ High Memory Usage
- ☑ Events with Errors
- ☑ Cluster Unreachable

BEST PRACTICES

- ✓ Monitor all critical cluster components.
- ✓ Set alerts for thresholds and failures.
- ✓ Use automation to reduce response time.
- ✓ Log and analyze events regularly.
- ✓ Keep your monitoring tools updated.

Proactive Monitoring
Stronger Clusters
Better Automation

MONITORING AUTOMATION WORKFLOW

Collect Metrics and Events → Analyze Conditions → Alert on Issues → Log and Report → Take Action Automatically

RESOURCE SUMMARY

Resource	What It Monitors	Key API	Useful For
Cluster Health	Overall cluster status	list_node()	High-level health check
Pods	Pod status and counts	list_pod_for_all_namespaces()	Workload monitoring
Events	Cluster events and warnings	list_event_for_all_namespaces()	Troubleshooting
Resource Usage	CPU and memory usage	list_node(), node.status	Capacity planning
Automation	Alerts and auto-actions	Custom scripts	Proactive operations

TIP: Tools to Integrate

- Prometheus (Metrics)
- Grafana (Dashboards)
- Alertmanager (Alerts)
- kubectl (CLI)

VERIQTA

JENKINS AUTOMATION

Automate Jenkins tasks using the Jenkins API.
This page covers the API, triggering builds, checking build status, and managing jobs programmatically.

REMEMBER

- Use API tokens, not passwords.
- Store credentials securely.
- Handle errors and retries.
- Log actions for audit.

1 JENKINS API OVERVIEW

Jenkins exposes a powerful REST API.

Base URL:
`http://<jenkins-host>:<port>/api/json`

Authentication Options:

- API Token (Recommended)
- Username + API Token
- Basic Auth (Avoid for automation)

TIP
Use API Token for all automation to avoid storing passwords.

2 JENKINS API AUTHENTICATION

Use API Token for authentication.

```python
import requests
JENKINS_URL = "http://jenkins.example.com:8080"
USERNAME = "automation-user"
API_TOKEN = "your_api_token"
auth = (USERNAME, API_TOKEN)
headers = { "Accept": "application/json" }
# Test connection
response = requests.get(f"{JENKINS_URL}/api/json",
      auth=auth, headers=headers, timeout=10)
print(response.status_code)  # 200 means success
```

TIP
Create an API Token from: Manage Jenkins → Security → API Tokens

3 TRIGGERING BUILDS

Trigger a job build programmatically.

```python
import requests
def trigger_build(job_name, parameters=None):
    url = f"{JENKINS_URL}/job/{job_name}/build"
    params = {}
    if parameters:
        params["json"] = json.dumps(parameters)
    response = requests.post(url, auth=auth, params=params)
    if response.status_code in [201, 200]:
        print(f"Build triggered for job: {job_name}")
    else:
        print(f"Failed to trigger build: {response.status_code}")
# Example
# With parameters
trigger_build("my-pipeline-job",
              {"env": "prod", "VERSION": "1.0"})
```

TIP
Status 201 means build queuud successfully.

4 BUILD STATUS

Check the status of a build.

```python
import requests
def get_build_status(job_name, build_number):
    url = f"{JENKINS_URL}/job/{job_name}/{build_number}/api/json"
    response = requests.get(url, auth=auth)
    if response.status_code == 200:
        data = response.json()
        status = data["result"]  # null, SUCCESS, FAILURE, etc.
        building = data['building"]
        print(f"Build #{build_number} status: {status}")
        print(f"Building: {building}")
    else:
        print(f"Failed to get status: {response.status_code}")

# Example
get_build_status("my-pipeline-job", 42)
```

TIP
result can be:
null (in progress)
SUCCESS
FAILURE
UNSTABLE
ABORTED

5 JOB MANAGEMENT - LIST JOBS

List all jobs in Jenkins.

```python
import requests
def list_jobs():
    url = f"{JENKINS_URL}/api/json?tree=jobs[name,color]"
    response = requests.get(url, auth=auth)
    data = response.json()

    for job in data['jobs']:
        print(f"{job['name']} ( {job['color']})")

# Example
list_jobs()
```

TIP
Use tree parameter to reduce payload size.

6 JOB MANAGEMENT - CREATE JOB

Create a new freestyle job.

```python
import requests, json
job_name = "auto-created-job"
config_xml = """
<project>
  <actions/>
  <description>Created via Jenkins API</description>
  <keepDependencies>false</keepDependencies>
  <properties/>
  <scm class="hudson.scm.NullSCM"/>
  <builders/>
</project>
"""
url = f"{JENKINS_URL}/createItem?name={job_name}"
headers = {"Content-Type": "application/xml"}
response = requests.post(url, auth=auth, data=config_xml, headers=headers)
if response.status_code in [200, 201]:
    print(f"Job '{job_name}' created successfully.")
else:
    print(f"Failed to create job: {response.status_code}")
```

TIP
Job config is XML.
You can export existing jobs and modify.

7 JOB MANAGEMENT - DELETE JOB

Delete a job.

```python
import requests
def delete_job(job_name):
    url = f"{JENKINS_URL}/job/{job_name}/doDelete"
    response = requests.post(url, auth=auth)
    if response.status_code in [200, 202]:
        print(f"Job '{job_name}' deleted.")
    else:
        print(f"Failed to delete job: {response.status_code}")
# Example
delete_job("auto-created-job")
```

TIP
Deletion is immediate.
No recycle bin!

8 USEFUL ENDPOINTS

Common Jenkins API endpoints.

PURPOSE		ENDPOINT
Get Jenkins Info	GET	/api/json
List All Jobs	GET	/api/json?tree=jobs[name]
Job Info	GET	/job/{job}/api/json
Trigger Build	POST	/job/{job}/build
Build Info	GET	/job/{job}/{build}/api/json
Console Output	GET	/job/{job}/{build}/consoleText
Delete Job	POST	/job/{job}/doDelete

TIP
Explore more endpoints at: /api/api-explorer/

JENKINS AUTOMATION WORKFLOW

Authenticate with API Token → Trigger Build → Monitor Build Status → Manage Jobs → Log & Report Results

BEST PRACTICES

- Use dedicated service accounts with API tokens.
- Never hardcode credentials; use environment variables or secret stores.
- Implement retries and timeouts in API calls.
- Log all API actions for traceability.
- Validate build results and send alerts on failures.

VERIQTA

GITHUB AUTOMATION

Automate GitHub tasks using the GitHub API.
This page covers the API, repository management, pull requests, and workflow automation.

REMEMBER

- Use Personal Access Tokens (PAT).
- Store tokens securely.
- Handle errors and rate limits.
- Keep automation idempotent.
- Log actions for audit.

① GITHUB API OVERVIEW

GitHub provides a powerful REST API.

Base URL:
`https://api.github.com`

Authentication Options:

- Personal Access Token (PAT) ← Recommended
- GitHub App
- OAuth App

TIP
Use PAT with minimum required permissions.

② GITHUB API AUTHENTICATION

Use a Personal Access Token (PAT) for authentication.

```python
import requests
TOKEN = "ghp_your_personal_access_token"
HEADERS = {
    "Authorication": f"token {TOKEN}",
    "Accept": "application/vnd.github+json"
}

# Test connection
response = requests.get("https://api.github.com/user",
                        headers=HEADERS)
print(response.status_code, response.json()["login"])
```

TIP
Store tokens in environment variables.

③ REPOSITORY MANAGEMENT

Create, list, update, and delete repositories.

```python
import requests
OWNER = "your-username"

# List repositories
url = f"https://api.github.com/users/{OWNER}/repos"
repos = requests.get(url, headers=HEADERS).json()

for repo in repos:
    print(repo["name"], repo["private"])

# Create repository
repo_data = {
   "name": "automated-repo",
   "private": False,
   "description": "Created via GitHub API"
}

response = requests.post(url, headers=HEADERS,
 json=repo_data)
print(response.status_code, response.json()["html_url"])
```

TIP
Check for existing repo before creating to avoid duplicates.

④ PULL REQUESTS

Create, list, and merge pull requests.

```python
# Create Pull Request
pr_data = {
    "title": "Automated PR",
    "head": "feature-branch",
    "base": "main",
    "body": "This PR is created automatically."
}
url = f"https://api.github.com/repos/{OWNER}/your-repo/pulls"
response = requests.post(url, headers=HEADERS, json=pr_data)
print(response.status_code, response.json()["html_url"])

# List Pull Requests
prs = requests.get(url, headers=HEADERS).json()
for pr in prs:
    print(pr["number"], pr["title"], pr["state"])

# Merge Pull Request (squash)
pr_number = 1
merge_url = f"https://api.github.com/repos/{OWNER}/your-repo/pulls/{pr_number}/merge"
merge_data = {"merge_method": "squash"}
merge_resp = requests.put(merge_url, headers=HEADERS, json=merge_data)
print(merge_resp.status_code, merge_resp.json()["merged"])
```

TIP
Ensure branch protection rules allow API merges.

⑤ WORKFLOW AUTOMATION

Trigger GitHub Actions workflows and monitor runs.

```python
# Trigger Workflow
workflow_id = "ci-cd.yml"
ref = "main"
url = f"https://api.github.com/repos/{OWNER}/your-
    repo/actions/workflows/{workflow_id}/dispatches"
payload = {"ref": ref}
response = requests.post(url, headers=HEADERS, json=payload)
print("Triggered:", response.status_code)

# List Workflow Runs
runs_url = f"https://api.github.com/repos/{OWNER}/your-repo/actions/runs"
runs = requests.get(runs_url, headers=HEADERS).json()
for run in runs["workflow_runs"][:5]:
   print(run["id"], run["name"], run["status"], run["conclusion"])
```

TIP
Use workflow dispatch to trigger manual runs.

⑥ COMMON ENDPOINTS

Frequently used GitHub REST API endpoints.

PURPOSE	METHOD	ENDPOINT
Get User Info	GET	/user
List Repositories	GET	/users/{owner}/repos
Create Repository	POST	/user/repos
Get Repository	GET	/repos/{owner}/{repo}
List Pull Requests	GET	/repos/{owner}/{repo}/pulls
Create Pull Request	POST	/repos/{owner}/{repo}/pulls
Merge Pull Request	PUT	/repos/{owner}/{repo}/pulls/{pull_number}/merge
Trigger Workflow	POST	/repos/{owner}/{repo}/actions/workflows/{id}/dispatches
List Workflow Runs	GET	/repos/{owner}/{repo}/actions/runs

TIP
https://docs.github.com/en/rest is your complete reference.

GITHUB AUTOMATION WORKFLOW

Authenticate with PAT → Manage Repositories → Create / Manage Pull Requests → Trigger Workflows → Monitor Runs → Log & Report

BEST PRACTICES

- ✓ Use minimal token permissions.
- ✓ Validate responses and handle errors.
- ✓ Respect rate limits and use pagination.
- ✓ Log all automation actions.
- ✓ Keep scripts idempotent and retry on failure.

VERIQTA

TERRAFORM AUTOMATION

Automate infrastructure provisioning using Terraform.
This page covers executing Terraform, plan, apply, destroy, and state management.

Terraform

REMEMBER

- Always plan before apply.
- Store state securely.
- Use remote backend for teams.
- Keep your state consistent.
- Automate responsibly.

1 EXECUTING TERRAFORM OVERVIEW

Terraform workflow follows a predictable sequence.

INIT	PLAN	APPLY	DESTROY	STATE
Initialize provider and backend	Preview changes	Apply changes	Remove infrastructure	Manage state file

TIP
Follow the workflow consistently to avoid drift.

2 INITIALIZE TERRAFORM

Initialize your working directory.

```
terraform init

# Downloads providers
# Initializes backend
# Prepares modules
```

TIP
Run init after changing providers, modules, or backend config.

3 PLAN: PREVIEW CHANGES

Preview what Terraform will create, update, or destroy.

```
terraform plan

# Save plan to file
terraform plan -out=tfplan

# Inspect plan file later
terraform show tfplan
```

TIP
Always run plan before apply.

4 APPLY: EXECUTE CHANGES

Apply the planned changes to provision infrastructure.

```
terraform apply

# Apply saved plan (recommended in automation)
terraform apply tfplan

# Auto-approve (use carefully)
terraform apply -auto-approve
```

TIP
Review the plan carefully before applying.

5 DESTROY: TEAR DOWN INFRASTRUCTURE

Destroy all resources managed by Terraform.

```
terraform destroy

# Destroy with saved plan
terraform destroy tfplan

# Auto-approve destroy (use carefully)
terraform destroy -auto-approve
```

TIP
Destroy is permanent. Use with caution.

6 STATE MANAGEMENT OVERVIEW

Terraform tracks real infrastructure in the state file.

TIP
If state is lost, Terraform loses track of your resources.

7 STATE MANAGEMENT COMMANDS

Common commands to manage Terraform state.

Command	Purpose	Example
terraform state list	List all resources in state	terraform state list
terraform state show <addr>	Show details of a resource	terraform state show aws_instance.web
terraform state mv <src> <dst>	Move resource in state	terraform state mv aws_instance.old aws_instance_new
terraform state rm <addr>	Remove resource from state	terraform state rm aws_s3_bucket.logs
terraform import <addr> <id>	Import existing resource	terraform import aws_s3_bucket.logs my-bucket
terraform refresh	Refresh state from real infra	terraform refresh

TIP: Use state commands carefully. Incorrect changes can cause drift or resource recreation.

8 BACKEND & REMOTE STATE

Store state remotely for team collaboration and safety.

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"   # for state locking
    encrypt        = true
  }
}
```

TIP
Remote state provides locking, backup, and team safety.

BEST PRACTICES

- Always run terraform plan before apply.
- Use remote backend (S3 + DynamoDB) for production.
- Enable state locking to prevent concurrent updates.
- Never edit terraform.tfstate manually.
- Run terraform destroy only when necessary.
- Use meaningful names and tags for all resources.

TERRAFORM AUTOMATION WORKFLOW

VERIQTA

ANSIBLE INTEGRATION

Automate infrastructure configuration and applications at scale using Ansible.
This page covers running playbooks, dynamic inventory, and automation pipelines.

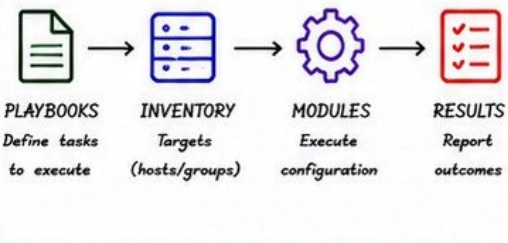
ANSIBLE

REMEMBER

- Idempotency is your friend.
- Test playbooks in staging first.
- Keep playbooks simple and modular.
- Use version control for all automation.
- Log everything. Automate responsibly.

① ANSIBLE INTEGRATION OVERVIEW

Ansible connects systems, inventory, and automation.



TIP
Ansible uses SSH. No agents required on managed nodes.

② RUNNING PLAYBOOKS

Execute automation with playbooks.

```
# Run a playbook
ansible-playbook -i inventory.ini site.yml

# With extra variables
ansible-playbook -i inventory.ini site.yml \
  -e "app_env=production"

# Check mode (dry run)
ansible-playbook -i inventory.ini site.yml --check

# Verbose output
ansible-playbook -i inventory.ini site.yml -vvv
```

TIP
Use --check before applying changes to preview what will happen.

③ EXAMPLE PLAYBOOK

Simple web server installation playbook.

```
---
- name: Install and start Nginx
  hosts: webservers
  become: yes
  vars:
    pkg: nginx
  tasks:
    - name: Install Nginx
      apt:
        name: "{{ pkg }}"
        state: present
        update_cache: yes
    - name: Start Nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

TIP
Use become: yes for privilege escalation. Ansible ensures idempotent execution.

④ DYNAMIC INVENTORY

Inventory that updates automatically.

```
# Example: AWS EC2 Dynamic Inventory

[webservers]
tag_Environment_Production
tag_Role_Web

[dbservers]
tag_Environment_Production
tag_Role_Database

# Use dynamic inventory script or plugin

ansible-inventory -i aws_ec2.yml --list
ansible-inventory -i aws_ec2.yml --graph
```

TIP
Dynamic inventory ensures accuracy as infrastructure changes.

⑤ INVENTORY FILE (STATIC)

Example static inventory file.

```
[webservers]
web01 ansible_host=10.0.1.11
web02 ansible_host=10.0.1.12

[dbservers]
db01 ansible_host=10.0.2.20

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

TIP
Group hosts by role. Use [all:vars] for global variables.

⑥ AUTOMATION PIPELINES

Integrate Ansible with CI/CD pipelines.

```
# Example GitHub Actions step
- name: Run Ansible Playbook
  uses: ansible/ansible-runner@v2
  with:
    playbook: site.yml
    inventory: inventory.ini
    extra_vars: "app_env=production"
```

TIP
Automate tests after deployment to ensure success.

⑦ BEST PRACTICES

- Keep playbooks short and focused.
- Use roles to organize complex playbooks.
- Store sensitive data in Ansible Vault.
- Test playbooks in staging first.
- Use check mode before applying changes.
- Version control everything.
- Log output for audit and troubleshooting.
- Keep inventory updated and accurate.

ANSIBLE PROJECT STRUCTURE

```
project/
├── inventory/
│   └── hosts.ini
├── group_vars/
├── host_vars/
├── roles/
├── playbooks/
│   └── site.yml
└── ansible.cfg
```

⑧ USEFUL COMMANDS

COMMAND	PURPOSE
ansible -i inventory all -m ping	Test connectivity to all hosts
ansible-playbook -i inventory site.yml	Run a playbook
ansible-playbook --check site.yml	Dry run (check mode)
ansible-inventory -i inventory --list	List inventory hosts
ansible-vault create secret.yml	Create encrypted file
ansible-vault edit secret.yml	Edit encrypted file
ansible-doc ping	View module documentation

TIP
ansible-doc <module> is your best friend.

VERIQTA

MONITORING INFRASTRUCTURE

Monitor your infrastructure in real time and stay ahead of issues.
This page covers CPU, memory, disk, network monitoring, and alert generation.

REMEMBER

- Monitor everything important.
- Set meaningful alert thresholds.
- Test alerts regularly.
- Act before users are affected.
- Logs + Metrics + Alerts = Visibility.

① CPU MONITORING

Track CPU usage, load, and spikes.

```
# CPU usage
top -bn1 | grep "Cpu(s)"
mpstat -P ALL 1 5

# Per-process CPU
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head -10

# Load average
uptime

# Record to file
sar -u 1 60 > cpu_report.txt
```

TIP

Set alert when CPU usage > 80% for 5 minutes.

② MEMORY MONITORING

Track memory usage, swap, and caching.

```
# Memory overview
free -h

# Detailed memory
vmstat -s

# Monitor memory usage
ps -eo pid,ppid,cmd,%mem --sort=-%mem | head -10

# Record to file
sar -r 1 60 > memory_report.txt
```

TIP

Alert when available memory < 15% of total.

③ DISK MONITORING

Track disk usage, I/O, and inode usage.

```
# Disk usage
df -h

# Inode usage
df -i

# Disk I/O stats
iostat -dx 1 5

# Top disk processes
iotop -o -b -n 1 | head -10

# Record to file
sar -d 1 60 > disk_report.txt
```

TIP

Alert when disk usage > 85% or inode usage > 90%.

④ NETWORK MONITORING

Track bandwidth, connections, and errors.

```
# Network usage
ifstat -i eth0 1 5

# Connections
ss -tulnp | head -20

# Packet errors
netstat -i | egrep "errors|drops"

# Latency check
ping -c 5 8.8.8.8

# Record to file
sar -n DEV 1 60 > network_report.txt
```

TIP

Alert when bandwidth usage > 80% or high packet loss detected.

⑤ ALERT GENERATION

Generate alerts based on thresholds.

```
# Example alert script (CPU)
#!/bin/bash
THRESHOLD=80
USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print $2 +$4}' |
 tr -d "%")
if [ "$USAGE" -gt "$THRESHOLD" ]; then
  echo "ALERT: CPU usage is ${USAGE}% on $(hostname)" |
  mail -s "CPU Alert" admin@example.com
fi

# Schedule with cron
# */5 * * * * /opt/monitor/alert_cpu.sh
```

TIP

Send alerts via email, Slack, Teams, or PagerDuty.

⑥ POPULAR MONITORING TOOLS

Choose tools that fit your infrastructure.

TOOL	TYPE	USE CASE
Prometheus	Metrics	Time-series monitoring & alerting
Grafana	Visualization	Dashboards & data visualization
Zabbix	Monitoring	Infrastructure & network monitoring
Nagios	Monitoring	System, service & network checks
Datadog	Cloud Monitoring	Metrics, logs, APM, alerts
New Relic	APM	Application & infrastructure insights
AWS CloudWatch	Cloud Monitoring	AWS resources monitoring & alerts

TIP

Start simple. Scale your monitoring as your system grows.

ALERTING WORKFLOW

Collect Metrics → Evaluate Thresholds → Trigger Alert → Notify Team → Investigate & Resolve

BEST PRACTICES

- Monitor what matters. Avoid noise.
- Set meaningful thresholds based on trends.
- Use dashboards to visualize key metrics.
- Test alerts regularly to avoid false alarms.
- Automate responses where possible.
- Document runbooks for common alerts.

VERIQTA

LOG ANALYSIS AUTOMATION

Automate the collection, parsing, analysis, and reporting of logs to detect issues faster and respond quickly. This page covers parsing logs, pattern matching, error detection, and reporting.

REMEMBER

- Logs tell the story.
- Automate the boring work.
- Detect early, fix fast.
- Good logs + Good analysis = Happy systems.

① PARSING LOGS

Extract structured data from unstructured logs.

```
# Example log line
2024-07-21 12:30:45 ERROR [api] User login failed
user=john ip=192.168.1.10

# Python: parse log line
import re
pattern = r'(?P<timestamp>\s+) (?P<level>\w+) \[(\w+)]
  (?P<message>.*?) user=(?P<user>\w+) ip=(?P<ip>\S+)'
line = "2024-07-21 12:30:45 ERROR [api] User login failed
user=john ip=192.168.1.10"
match = re.search(pattern, line)
if match:
    data = match.groupdict()
    print(data)  # Parsed dictionary
```

TIP

Parsing turns messy logs into usable data.

② PATTERN MATCHING

Search for specific patterns in logs.

```
# Find all ERROR or CRITICAL lines
import re
error_pattern = re.compile(r'\b(ERROR|CRITICAL)\b')

log_text = open('app.log').read()
matches = error_pattern.findall(log_text)

print(f"Found {len(matches)} error lines")

# Extract all IP addresses
ip_pattern = re.compile(r'(?:\d{1,3}\.){3}\d{1,3}')
ips = ip_pattern.findall(log_text)
print(f"IPs found: {set(ips)}")
```

TIP

Regex is your best friend for finding what matters.

③ ERROR DETECTION

Automatically detect and classify errors.

```
# Error detection with severity levels
import re
rules = {
   'CRITICAL': re.compile(r'CRITICAL|FATAL|PANIC'),
   'ERROR':    re.compile(r'ERROR|Exception|Traceback'),
   'WARNING':  re.compile(r'WARNING'),
   'INFO':     re.compile(r'INFO')
}
def detect_level(line):
    for level, pattern in rules.items():
        if pattern.search(line):
            return level
    return 'UNKNOWN'

# Example usage
level = detect_level(line)
print('Detected level:', level)
```

TIP

Classify logs by severity to focus on what needs urgent attention.

④ REPORTING

Generate reports and summaries automatically.

```
# Simple error report
from collections import Counter

# Assume 'levels' is a list of detected levels
levels = ['INFO','ERROR','ERROR','WARNING','CRITICAL','ERROR"]
summary = Counter(levels)

with open('log_report.txt', 'w') as f:
   f.write('LOG ANALYSIS REPORT\n')
   f.write('='*30 + '\n')
   for level, count in summary.items():
      f.write(f"{level:<10} : {count:<5}\n")

print("Report generated: log_report.txt")
```

TIP

Clear reports help teams act fast and track trends.

SAMPLE REPORT (log_report.txt)

```
LOG ANALYSIS REPORT
==================================
CRITICAL   :     2
ERROR      :    15
WARNING    :     8
INFO       :   120
```

⑤ END-TO-END LOG ANALYSIS SCRIPT

Put everything together in one Python script.

```
import re
from collections import Counter
import smtplib

log_file = 'app.log'
levels = []
errors = []
pattern = re.compile(r'(?P<ts>\S+) (?P<level>\w+) \[.*?\] (?P<msg>.*)')
with open(log_file) as f:
    for line in f:
        match = pattern.search(line)
        if match:
            level = match.group('level')
            levels.append(level)
            if level in ['ERROR', 'CRITICAL']:
                errors.append(line.strip())
# Summary
summary = Counter(levels)
# Send alert if critical errors found
if summary['CRITICAL'] > 0:
    subject = f"ALERT: {summary['CRITICAL']} critical errors detected"
    body = '\n'.join(errors[:20]) + f"\n?..and {len(errors)-20}?"
    # send_email(subject, body)  # Implement as needed
print('Analysis complete. Check report and alerts.')
```

TIP

Automate end-to-end: collect, parse, detect, report, and alert.

USEFUL TOOLS

TOOL	USE CASE
grep / awk	Quick search & extract on Linux
sed	Stream editing & parsing
jq	Parse JSON logs
logstash	Collect, parse & ship logs
fluentd	Log collection & forwarding
ELK Stack	Search, visualize, alert (Elasticsearch)
Python (re)	Custom parsing & automation
SPLUNK	Enterprise log analysis & alerting

TIP Choose the right tool for your scale and environment.

LOG ANALYSIS AUTOMATION WORKFLOW

Collect Logs (Sources) → Parse Logs (Structure Data) → Pattern Match (Find Issues) → Detect Errors (Classify) → Generate Report (Summary) → Send Alerts (Notify Team)

BEST PRACTICES

- Centralize logs from all systems.
- Use consistent log format (JSON recommended).
- Monitor critical errors in real time.
- Keep alerts actionable and noise-free.
- Review and improve log rules regularly.

VERIQTA

REGULAR EXPRESSIONS

Regular Expressions (Regex) are powerful patterns used for matching, searching, validating, and extracting data. This page covers pattern matching, log parsing, input validation, and real production use cases.

REMEMBER

- Regex makes searching smarter.
- Master the basics first.
- Test before production.
- Small patterns = Big power.
- Regex is logic, not magic.

1 WHAT IS REGULAR EXPRESSION?

A sequence of characters that defines a search pattern.

String:

User: admin@example.com logged in at 2024-07-21 12:30:45

Regex Pattern:

```
\b[\w.-]+@[\w.-]+\.\w+\b
```

Matches:

admin@example.com

TIP

Regex helps you find exact data even in noisy logs.

2 COMMON REGEX SYMBOLS

SYMBOL	MEANING	EXAMPLE	MATCHES
.	Any single character	a.c	abc, acc, a1c
*	Zero or more	a*	a, aa, aaa
+	One or more	a+	a, aa, aaa
?	Zero or one	colou?r	color, colour
[]	Character set	[abc]	a, b, or c
[^]	Not in set	[^0-9]	Not digits
()	Capture group	(error)	Capture "error"
|	OR operator	error|fail	error or fail
^	Start of string	^User	Starts with User
$	End of string	\.com$	Ends with .com

TIP

Use () to capture data and extract important fields.

3 PATTERN MATCHING EXAMPLES

```
# Email validation
^[\w.-]+@[\w.-]+\.\w+$
Matches: user@example.com

# IP Address
^(\d{1,3}\.){3}\d{1,3}$
Matches: 192.168.1.10

# Phone Number (11 digits)
^\+?\d{1,3}?\d{10,11}$
Matches: +2348123456789

# Date Format (YYYY-MM-DD)
^\d{4}-\d{2}-\d{2}$
Matches: 2024-07-21
```

TIP

Test your patterns in regex101.com before production.

4 LOG PARSING EXAMPLES

Extract useful data from logs using regex.

Log Line:

2024-07-21 12:30:45 ERROR [Auth] User login failed IP: 192.168.1.10 User: admin

Timestamp	Level	Module	Message	IP	User
2024-07-21 12:30:45	ERROR	[Auth]	User login failed	192.168.1.10	admin

Regex Pattern:

```
^(\d{4}-\d{2}-\d{2})\s+(\d{2}:\d{2}:\d{2})\s+(ERROR)\s+\[([^\]]+)\]\s+(.+?)\s+IP:\s*([\d\.]+)\s+User:\s*(\w+)$
```

Tools: grep, awk, sed, Python (re), Logstash, Go, Rust

TIP

Capture groups help extract each field separately.

5 INPUT VALIDATION EXAMPLES

VALIDATION TYPE	REGEX PATTERN	EXAMPLE MATCHES
Email	^[\w.-]+@[\w.-]+\.\w+$	user@mail.com
Password (8+ chars)	^(?=.*[A-Za-z])(?=.*\d).{8,}$	Password123
Phone (10–15 digits)	^\+?\d{10,15}$	+2348123456789
URL	^https?://[^\s/$.?#].[^\s]*$	https://veriqta.com
Username	^[a-zA-Z0-9_]{3,20}$	Veriqta_01

6 PRODUCTION USE CASES

- Log analysis and monitoring
- Security incident detection
- Data extraction from reports
- Input validation in applications
- Web scraping and text mining

TIP

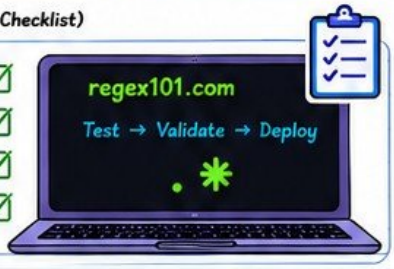
Regex is used everywhere: CI/CD, Security, Monitoring, APIs, Databases, and more.

7 REGEX WITH LINUX COMMANDS

COMMAND	USE CASE	EXAMPLE
grep	Search pattern	grep "ERROR" app.log
grep -E	Extended regex	grep -E "ERR|WARN" app.log
sed -E	Replace with regex	sed -E 's/(\d{4}-\d{2}-\d{2})/DATE/' file.log
awk	Extract fields	awk '/ERROR/ {print $0}' app.log
awk -F	Split with regex	awk -F':' '{print $1,$3}' file.log
cut	Cut matched parts	cut -d' ' -f1-3 log.txt

8 TEST YOUR REGEX (Quick Checklist)

- Does it match what you expect?
- Does it avoid unwanted matches?
- Is it efficient and readable?
- Does it handle edge cases?



REGEX WORKFLOW

Identify Need → Define Pattern → Test on Sample → Validate Results → Use in Production

BEST PRACTICES

- Keep patterns simple and clear.
- Use anchors (^ and $) for exact matches.
- Always test with real data.
- Comment complex patterns.
- Document your regex for your team.

VERIQTA

NETWORK AUTOMATION

Automate network tasks to test, monitor, and validate connectivity, ports, and DNS. This saves time, reduces manual work, and improves reliability.

REMEMBER

- Automate repetitive tasks.
- Log everything.
- Handle errors gracefully.
- Validate before you deploy.
- Automation = speed + accuracy.

1 PING AUTOMATION

Check host availability automatically.

```python
import subprocess, csv, datetime
hosts = ["8.8.8.8", "1.1.1.1", "google.com"]
results = []
for host in hosts:
    response = subprocess.run(["ping", "-c", "4", host],
                              capture_output=True, text=True)
    status = "UP" if response.returncode == 0 else "DOWN"
    results.append([host, status, datetime.datetime.now()])

with open("ping_report.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Host", "Status", "Checked At"])
    writer.writerows(results)

# Output: ping_report.csv
```

TIP

Use -c for count and -W for timeout.

2 PORT SCANNING

Scan ports on a host automatically.

```python
import socket, csv, datetime
target = "192.168.1.10"
ports = [22, 80, 443, 3306, 5432, 8080]
open_ports = []
for port in ports:
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(1)
    result = sock.connect_ex((target, port))
    if result == 0:
        open_ports.append(port)
    sock.close()
with open("port_scan.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Target", "Open Ports", "Scanned At"])
    writer.writerow([target, ",".join(map(str, open_ports)),
                     datetime.datetime.now()])
# Output: port_scan.csv
```

TIP

Scan only authorized systems. Respect security policies.

3 DNS LOOKUP AUTOMATION

Resolve DNS records for multiple domains.

```python
import dns.resolver, csv, datetime
domains = ["google.com", "veriqta.com", "github.com"]
records = ["A", "AAAA", "MX", "NS", "TXT"]
rows = []
for domain in domains:
    for record in records:
        try:
            answers = dns.resolver.resolve(domain, record)
            for r in answers:
                rows.append([domain, record, r.to_text()])
        except Exception as e:
            rows.append([domain, record, f"ERROR: {e}"])

with open("dns_lookup.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Domain", "Record", "Value"])
    writer.writerows(rows)

# Output: dns_lookup.csv
```

TIP

Useful records: A, AAAA, MX, NS, TXT.

4 CONNECTIVITY TESTING

Test connectivity to multiple endpoints and log results.

```python
import subprocess, csv, datetime
endpoints = [
    {"name": "Google DNS", "host": "8.8.8.8"},
    {"name": "Cloudflare DNS", "host": "1.1.1.1"},
    {"name": "VERIQTA Website", "host": "veriqta.com"}
]
results = []
for ep in endpoints:
    response = subprocess.run(["ping", "-c", "4", ep["host"]],
                              capture_output=True, text=True)
    status = "UP" if response.returncode == 0 else "DOWN"
    results.append([ep["name"], ep["host"], status,
                    datetime.datetime.now()])
with open("connectivity_report.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Host", "Status", "Checked At"])
    writer.writerows(results)
# Output: connectivity_report.csv
```

TIP

Run regularly using cron or scheduler.

5 NETWORK AUTOMATION WORKFLOW

Automate → Collect → Analyze → Alert

Define Targets → Run Automation → Collect Results → Analyze Data → Alert & Report

USEFUL TOOLS

TOOL	USE CASE
ping	Test host availability
nmap	Port scanning & network discovery
dig / nslookup	DNS queries & lookups
python (socket)	Custom network automation
cron / task scheduler	Schedule automation tasks
csv / json	Store and analyze results

TIP

Combine tools for powerful automation pipelines.

6 SAMPLE AUTOMATION OUTPUT

ping_report.csv

Host	Status	Checked At
8.8.8.8	UP	2024-07-21 12:30
1.1.1.1	UP	2024-07-21 12:30
google.com	UP	2024-07-21 12:30

port_scan.csv

Target	Open Ports	Scanned At
192.168.1.10	22,80,443	2024-07-21 12:30

dns_lookup.csv

Domain	Record	Value
google.com	A	142.250.72.14
google.com	MX	10 alt1.aspmx.l.google.com.
veriqta.com	A	104.21.19.12

connectivity_report.csv

Name	Host	Status	Checked At
Google DNS	8.8.8.8	UP	2024-07-21 12:31
Cloudflare DNS	1.1.1.1	UP	2024-07-21 12:31
VERIQTA Website	veriqta.com	UP	2024-07-21 12:31

7 BEST PRACTICES

- Automate safely and ethically.
- Log everything for audit & troubleshooting.
- Handle timeouts and errors.
- Use meaningful reports and alerts.
- Keep scripts modular and reusable.
- Test automation before production.

VERIQTA

SSH AUTOMATION

Automate remote access, command execution, and file transfer using SSH. This page covers Paramiko, remote execution, file transfer, and secure automation best practices.

REMEMBER

- Automate with security.
- Use keys, not passwords.
- Handle errors and timeouts.
- Log every action.
- Test safely before production.

1 PARAMIKO OVERVIEW

Paramiko is a Python library for SSH automation.

- Pure Python SSHv2 library
- Supports authentication, commands, SFTP
- Works on Linux, macOS, Windows

TIP

Always prefer key-based authentication over passwords.

2 SSH CONNECTION WITH PARAMIKO

```python
import paramiko

# Connect to remote server
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect(
    hostname="192.168.1.10",
    username="admin",
    key_filename="~/.ssh/id_rsa",
    port=22,
    timeout=10
)
print("SSH connection established")
```

TIP

Use key_filename for secure key-based auth.

3 REMOTE COMMAND EXECUTION

Run commands on remote servers.

```python
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect(hostname="192.168.1.10",
               username="admin",
               key_filename="~/.ssh/id_rsa")
# Execute command
stdin, stdout, stderr = client.exec_command("uptime")
output = stdout.read().decode()
error = stderr.read().decode()
print("OUTPUT:\n", output)
if error:
    print("ERROR:\n", error)
client.close()
```

TIP

Always check stderr for errors. Do not ignore it.

4 FILE TRANSFER WITH SFTP

Secure file upload and download.

```python
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect(hostname="192.168.1.10",
               username="admin",
               key_filename="~/.ssh/id_rsa")

sftp = client.open_sftp()
# Upload file
sftp.put("/home/ann/local_file.txt", "/home/admin/remote_file.txt")
# Download file
sftp.get("/var/log/syslog", "./syslog_backup")
sftp.close()
client.close()
```

TIP

SFTP uses the same SSH connection.

5 SECURE AUTOMATION BEST PRACTICES

- Use SSH keys, not passwords.
- Store keys securely with proper permissions.
- Limit user permissions on remote servers.
- Validate host keys (Avoid AutoAddPolicy in production).
- Log every action and failure.
- Use timeouts to avoid hanging scripts.
- Handle exceptions and reconnect safely.

TIP

Security is not optional. It is the foundation of automation.

6 ERROR HANDLING EXAMPLE

```python
import paramiko
from paramiko.ssh_exception import SSHException
try:
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(hostname="192.168.1.10",
                   username="admin",
                   key_filename="~/.ssh/id_rsa",
                   timeout=10)
    stdin, stdout, stderr = client.exec_command("df -h")
    print(stdout.read().decode())
except SSHException as e:
    print(f"SSH Error: {e}")
except Exception as e:
    print(f"Error: {e}")
finally:
    client.close()
```

TIP

Always handle exceptions gracefully.

7 COMMON SSH AUTOMATION TASKS

TASK	DESCRIPTION
Check Server Uptime	Run uptime or whoami to verify server status.
Deploy Application	Transfer files and restart services.
Backup Files	Download logs or important files.
Log Collection	Fetch logs for monitoring and analysis.
System Updates	Run update commands on multiple servers.
Health Checks	Automate checks and alert on failures.

8 SSH AUTOMATION WORKFLOW

Authenticate Securely → Execute Commands → Transfer Files → Log Results → Automate Safely

USEFUL COMMANDS

```
ssh -i ~/.ssh/id_rsa user@host              # Connect using key
scp -i ~/.ssh/id_rsa file user@host:/path   # Copy file
sftp -i ~/.ssh/id_rsa user@host             # Open SFTP session
ssh-keygen -t rsa -b 4096                   # Generate SSH key
```

FINAL NOTE

SSH automation is powerful, but with great power comes great responsibility. Secure your keys, validate your hosts, and test your automation before it touches production.

VERIQTA

SCHEDULING AUTOMATION

Scheduling automation allows tasks to run at the right time without manual intervention.

This page covers cron, Windows Task Scheduler, Python sched, and background jobs.

REMEMBER

- Automate repetitive tasks.
- Schedule wisely.
- Monitor scheduled jobs.
- Log all task executions.
- A good schedule = efficiency.

1 CRON (LINUX / MACOS)

Cron is a time-based job scheduler in Unix-like systems.

CRON SYNTAX (5 FIELDS)

```
*     *     *     *     command_to_execute
```

Minute	Hour	Day	Month	DayOfWeek
0-59	0-23	1-31	1-12	0-6

EXAMPLE CRON JOBS

```
0 2 * * *      /usr/bin/backup.sh       # Run daily at 2:00 AM
*/15 * * * *   /usr/bin/monitor.sh      # Every 15 minutes
0 9 * * 1-5    /usr/bin/report.sh       # Weekdays at 9:00 AM
0 0 1 * *      /usr/bin/monthly.sh      # 1st of every month
0 */6 * * *    /usr/bin/healthcheck.sh  # Every 6 hours
```

COMMON CRON COMMANDS

`crontab -e`	Edit crontab
`crontab -l`	List all jobs
`crontab -r`	Remove all jobs
`service cron status`	Check cron service

TIP

Edit your crontab with: `crontab -e`

List jobs: `crontab -l`

2 WINDOWS TASK SCHEDULER

Windows Task Scheduler runs tasks at scheduled times.

STEPS TO CREATE A TASK

1. Open Task Scheduler
2. Click Create Basic Task
3. Give the task a name and description
4. Choose the trigger (time/event)
5. Set the action (start a program/script)
6. Review and finish

EXAMPLE ACTIONS

- Run a Python script daily at 7 AM
- Backup files every Sunday night
- Generate reports on system startup
- Clean temporary files every hour

TIP

Use highest privilege for system tasks.

3 PYTHON SCHED MODULE

Python's sched module allows timing events in a script.

```python
import sched
import time

scheduler = sched.scheduler(time.time, time.sleep)

def job():
    print("Scheduled job is running...")

# Schedule 'job' to run after 10 seconds
scheduler.enter(10, 1, job)

print("Scheduler started...")
scheduler.run()
```

TIP

Uses delay + priority for job execution.

4 BACKGROUND JOBS (LINUX)

Run tasks in the background without blocking your terminal.

RUN COMMAND IN BACKGROUND

```
command &          # Runs in background
```

DISOWN JOB (KEEP RUNNING AFTER LOGOUT)

```
disown -h %job_id      # Example: disown -h %1
```

CHECK JOBS

```
jobs -l            # List running background jobs
```

BRING JOB TO FOREGROUND

```
fg %job_id         # Example: fg %1
```

TIP

Use `nohup` for long running processes.

`nohup cmd &`

SCHEDULING AUTOMATION WORKFLOW

Define Task → Set Schedule → Run Task → Log Output → Monitor & Analyze

BEST PRACTICES

- Schedule jobs during off-peak hours.
- Always log outputs and errors.
- Set alerts for job failures.
- Test your schedules before production.
- Document all scheduled jobs.
- Review and optimize regularly.

CRON PATTERN EXAMPLES

`@reboot`	Run once at system startup
`@daily`	Run once every day at midnight
`@weekly`	Run once every week
`@monthly`	Run once every month
`0 3 * * 0`	At 3:00 AM every Sunday
`*/10 * * * *`	Every 10 minutes

PYTHON SCHED EXAMPLE (MULTIPLE JOBS)

```python
import sched, time
s = sched.scheduler(time.time, time.sleep)
def task1(): print("Task 1 executed")
def task2(): print("Task 2 executed")
s.enter(5, 1, task1)
s.enter(10, 1, task2)
s.run()
```

WINDOWS TASK TYPES

- Daily
- Weekly
- Monthly
- On Startup
- On Logon
- On Event
- One Time
- On Idle

VERIQTA

CONFIGURATION MANAGEMENT

Configuration management ensures your applications behave the same across environments. This page covers reading configs, environment files, secrets handling, and production recommendations.

REMEMBER

- Keep configs separate from code.
- Never commit secrets.
- Use environment-specific values.
- Validate configuration at startup.
- Secure configs = secure apps.

1 READING CONFIGURATION FILES (PYTHON)

Use configuration files to store settings outside your code.

Using configparser (.ini file)

```python
import configparser
config = configparser.ConfigParser()
config.read('app.ini')

host = config["database"]["host"]
port = config.getint('database', 'port')
debug = config.getboolean("app", 'debug')

print(host, port, debug)
```

Example app.ini

```
[database]
host = localhost
port = 5432
user = appuser
password = secret123

[app]
debug = true
log_level = INFO
```

TIP

Use config files for structured settings.

2 ENVIRONMENT FILES (.ENV)

Store environment-specific values in .env files.

Using python-dotenv

```python
from dotenv import load_dotenv
import os

load_dotenv()  # Loads .env file

db_host = os.getenv("DB_HOST")
db_user = os.getenv("DB_USER")
db_pass = os.getenv("DB_PASS")
debug = os.getenv("DEBUG", "false")

print(db_host, db_user, debug)
```

Example .env

```
DB_HOST=localhost
DB_USER=appuser
DB_PASS=SuperSecret123
DEBUG=true
LOG_LEVEL=INFO
```

Common Environments

```
# .env.development
# .env.staging
# .env.production
```

TIP

.env files make it easy to switch environments.

.ENV

3 SECRETS HANDLING

Never hardcode secrets. Use secure methods.

METHOD	DESCRIPTION	EXAMPLE
Environment Variables	Store secrets in env vars or .env files	DB_PASS=secret
AWS Secrets Manager	Managed secrets service	Get secret via boto3
HashiCorp Vault	Centralized secret management	vault kv get secret/app
Kubernetes Secrets	Store secrets in K8s securely	kubectl create secret generic db-secret ...

Example: AWS Secrets Manager (Python)

```python
import boto3, json
client = boto3.client('secretsmanager', region_name="us-east-1")
response = client.get_secret_value(SecretId='my/secret')
secret = json.loads(response["SecretString"])
db_password = secret['db_password"]
```

TIP

Use managed secret stores for production applications.

4 PRODUCTION RECOMMENDATIONS

Follow these practices for secure and reliable configuration management.

- Never commit .env or secret files to version control.
- Use different configs for dev, staging, and production.
- Validate all required configs at application startup.
- Fail fast if a required config is missing.
- Use least privilege for accessing secrets.
- Rotate secrets regularly.
- Use IAM roles instead of long-lived keys.
- Encrypt sensitive data in transit and at rest.
- Document all configuration variables.

TIP

Good config management prevents outages and security breaches.

5 VALIDATE CONFIGURATION AT STARTUP

Ensure all required configs are present and valid.

```python
import os, sys
required_vars = ["DB_HOST", "DB_USER", "DB_PASS"]
missing = [var for var in required_vars if not os.getenv(var)]
if missing:
    print(f"Missing required configs: {', '.join(missing)}")
    sys.exit(1)

print("All configurations loaded successfully.")
```

TIP

Validate early to avoid runtime failures.

6 CONFIGURATION BEST PRACTICES WORKFLOW

A simple workflow for production-ready configuration management.

Define Config Requirements → Store in Secure Config Files /.env/Secrets → Validate at Application Startup → Load & Apply Configurations Safely → Monitor & Rotate Secrets Regularly

USEFUL COMMANDS

```
# Load .env manually          source .env
# Check env variables         printenv | grep DB_
# List AWS secrets            aws secretsmanager list-secrets
# Get secret value            aws secretsmanager get-secret-value --secret-id my/secret
```

FINAL NOTE

Configuration management is the backbone of reliable systems.
Keep configurations clean, secure, and environment-specific.
A well-managed config today prevents incidents tomorrow.

VERIQTA

SECRETS MANAGEMENT

Secrets management keeps sensitive information secure, accessible to authorized applications, and protected from exposure.

REMEMBER

- Never hardcode secrets.
- Least privilege always.
- Encrypt in transit and at rest.
- Rotate secrets regularly.
- Monitor and audit access.

① ENVIRONMENT VARIABLES

Store non-sensitive or low-sensitivity configuration.

Example (.env file)

```
DB_HOST=Localhost
DB_USER=appuser
DB_PASS=SuperSecret123
API_KEY=abcd1234
LOG_LEVEL=INFO
```

Load in Application (Python)

```
from dotenv import load_dotenv
import os
load_dotenv()          # Loads .env file
db_pass = os.getenv("DB_PASS")
print(db_pass)
```

TIP

Use .env for local dev only. Never commit .env files.

② AWS SECRETS MANAGER

Fully managed service to store, retrieve, and rotate secrets.

Store a Secret (CLI)

```
aws secretsmanager create-secret \
  --name myapp/db-credentials \
  --secret-string '{"username":"admin",
                    "password":"SuperSecret123"}'
```

Retrieve a Secret (CLI)

```
aws secretsmanager get-secret-value \
  --secret-id myapp/db-credentials \
  --query SecretString --output text
```

```
{"username":"admin","password":"SuperSecret123"}
```

TIP

Enable automatic rotation for databases, API keys, and other critical secrets.

Key Benefits

- Automatic rotation
- Encryption at rest (KMS)
- Fine-grained IAM access
- Audit with CloudTrail

③ HASHICORP VAULT CONCEPTS

Open-source tool for managing secrets, encryption, and sensitive data.

Core Concepts

- Secrets Engine – Stores and manages secrets
- Vault Server – Central server for secrets
- Authentication – Validates identity (user, app, etc.)
- Policies – Control access to secrets
- Lease & TTL – Secrets are time-bound
- Audit Logging – Track all secret access

Basic Workflow

User / App → Authenticate → Vault Server → Retrieve Secret

TIP

Vault issues temporary tokens. No static credentials.

Example: Get Secret from Vault (CLI)

```
vault kv get secret/myapp/db
```

====== Metadata ======	
Key	Value
created_time	2024-07-21T10:00:02Z
version	1

====== Data ======	
Key	Value
username	admin
password	SuperSecret123

④ SECURITY BEST PRACTICES

- Never hardcode secrets in code or images.
- Use IAM roles / policies with least privilege.
- Encrypt secrets in transit (TLS) and at rest.
- Rotate secrets regularly and automate where possible.
- Use environment-specific secrets (dev, staging, prod).
- Audit and monitor access to all secrets.
- Restrict secret access to only required services.
- Use short-lived credentials and tokens.
- Avoid storing secrets in plaintext files.
- Enable versioning and backup for secrets.
- Disable unused, old, or expired secrets.

TIP

Good secrets management prevents breaches and protects your customers.

⑤ SECRETS MANAGEMENT WORKFLOW

1. Identify Secrets → 2. Store Securely → 3. Control Access → 4. Use in Applications → 5. Rotate & Monitor → 6. Audit & Review

COMMON TOOLS

- AWS Secrets Manager
- HashiCorp Vault
- Azure Key Vault
- Google Secret Manager
- Doppler / 1Password / Bitwarden

```
# AWS: List secrets
  aws secretsmanager list-secrets
# AWS: Describe secret
  aws secretsmanager describe-secret --secret-id mysecret
# AWS: Get secret value
  aws secretsmanager get-secret-value --secret-id mysecret
```

```
#Vault: List secrets
vault kv list secret/
#Vault: Read secret
vault kv get secret/myapp/db
#Vault: Revoke token
vault token revoke <token>
```

FINAL NOTE

Secrets are the keys to your kingdom. Manage them with care, automate everything, and never expose what you can protect.

VERIQTA

BUILDING A HEALTH CHECK FRAMEWORK

A health check framework automatically validates your systems, services, and endpoints, reports the status, and alerts you when something goes wrong.

REMEMBER

- Detect issues early.
- Automate checks.
- Reduce downtime.
- Alert the right people.
- Measure and improve.

① SERVICE CHECKS

Verify that required services are running and healthy.

```python
import subprocess
services = ["nginx", "docker", "postgresql"]

def check_service(service):
    result = subprocess.run(
        ["systemctl", "is-active", service],
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )
    return result.stdout.strip() == "active"

for svc in services:
    status = "UP" if check_service(svc) else "DOWN"
    print(f"{svc}: {status}")
```

TIP

Customize the service list based on your environment.

Common Services to Check

- nginx / apache
- docker
- postgresql / mysql
- redis
- kubelet
- application processes

② ENDPOINT VALIDATION

Validate HTTP/HTTPS endpoints, APIs, and custom URLs.

```python
import requests
endpoints = {
    "Home Page": "https://example.com",
    "API Health": "https://api.example.com/health",
    "Login Page": "https://example.com/login"
}
def check_endpoint(name, url, timeout=5):
    try:
        r = requests.get(url, timeout=timeout)
        if r.status_code == 200:
            return "UP", r.status_code
        else:
            return "DOWN", r.status_code
    except Exception as e:
        return "DOWN", str(e)
for name, url in endpoints.items():
    status, info = check_endpoint(name, url)
    print(f"{name}: {status} - {info}")
```

TIP

Use proper timeouts and retry logic for reliability.

Validation Types

- HTTP Status
- Response Time
- Content Check
- SSL Certificate
- JSON Schema

③ REPORTING

Generate clear reports for visibility and auditing.

```python
import json
from datetime import datetime

report = {
    "timestamp": datetime.now().isoformat(),
    "summary": {"total": 5, "up": 4, "down": 1},
    "results": [
      {"name": "nginx", "status": "UP", "detail": "active"},
      {"name": "api", "status": "DOWN", "detail": "503"}
    ]
}
with open("health_report.json", "w") as f:
    json.dump(report, f, indent=4)
print("Report saved to health_report.json")
```

TIP

Store reports in files, databases, or monitoring systems.

Report Formats

- JSON
- CSV
- HTML
- Slack / Email
- Dashboard

④ ALERTING

Alert the right people when checks fail.

```python
import smtplib
def send_email_alert(subject, message):
    server = smtplib.SMTP("smtp.gmail.com", 587)
    server.starttls()
    server.login("you@gmail.com", "your_password")
    msg = f"Subject: {subject}\n\n{message}"
    server.sendmail("you@gmail.com", "ops@example.com", msg)
    server.quit()

# Example usage
send_email_alert(
    "Health Check Alert: API is DOWN",
    "The API endpoint returned 503 Service Unavailable."
)
```

TIP

Integrate with Slack, Telegram, PagerDuty, OpsGenie, or email.

Alert Channels

- Email
- Slack
- Telegram
- PagerDuty
- OpsGenie

⑤ HEALTH CHECK FRAMEWORK WORKFLOW

1. Define Checks
2. Run Checks
3. Collect Results
4. Generate Report
5. Send Alerts
6. Monitor & Improve

BEST PRACTICES

- Keep checks lightweight and fast.
- Set appropriate timeouts.
- Avoid false alerts.
- Monitor from multiple locations.
- Review and update checks regularly.

USEFUL COMMANDS

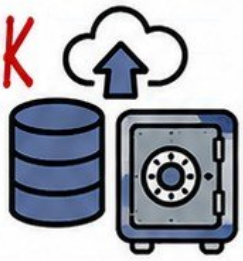
```
systemctl is-active <service>                    # Check service status
curl -Is https://example.com                     # Check HTTP endpoint
ping -c 4 example.com                            # Test connectivity
dig example.com                                  # DNS lookup
openssl s_client -connect example.com:443 -brief # SSL check
```

FINAL NOTE

A solid health check framework is the foundation of reliable systems. Automate checks, alert fast, and never let small issues become big outages.

VERIQTA

BUILDING A BACKUP FRAMEWORK



A backup framework ensures your data and systems can be recovered quickly and reliably. This page covers automation, validation, scheduling, and retention policies.

REMEMBER

- Automate backups.
- Validate restores regularly.
- Encrypt backups.
- Follow retention policies.
- Test your restores.

① BACKUP AUTOMATION

Automate backups to avoid human errors and ensure consistency.

Python Backup Script (Example)

```python
import subprocess, datetime, os
BACKUP_DIR = "/backups/db"
DB_NAME = "appdb"
timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
filename = f"{DB_NAME}_{timestamp}.sql.gz"
filepath = f"{BACKUP_DIR}/{filename}"
cmd = [
    "pg_dump", "-U", "postgres", "-d", DB_NAME,
    "-F", "c", "-f", "-"
]
with open(filepath, "wb") as f:
    subprocess.run(cmd, stdout=f, check=True)
print(f"Backup completed: {filepath}")
```

TIP

Encrypt backups at rest and in transit.

What to Backup

- Databases
- Configurations
- Application Data
- Logs
- Certificates
- Secrets
- System Files
- Snapshots

② RESTORE VALIDATION

Backups are useless if you cannot restore them.

Restore Validation Script (Example)

```python
import subprocess, os, tempfile
BACKUP_FILE = "/backups/db/appdb_latest.sql.gz"
with tempfile.TemporaryDirectory() as tmpdir:
    restore_path = f"{tmpdir}/restore.sql"
    cmd = [
        "pg_restore", "-U", "postgres", "-d", "postgres",
        BACKUP_FILE
    ]
    result = subprocess.run(cmd, stdout=subprocess.PIPE,
                            stderr=subprocess.PIPE)
    if result.returncode == 0:
        print("Restore validation successful ✅")
    else:
        print("Restore validation failed ❌")
        print(result.stderr.decode())
```

TIP

Validate restores in a separate environment. Never skip this step.

Validation Checklist

- Backup integrity check
- Data consistency verification
- Restore to isolated environment
- No missing or corrupted data
- Application starts successfully
- Performance check after restore

③ SCHEDULING

Schedule backups during low-traffic hours.

Cron Example (Linux)

```
# Daily full backup at 2:00 AM
0 2 * * * /usr/bin/python3 /scripts/backup.py

# Weekly backup on Sunday at 3:00 AM
0 3 * * 0 /usr/bin/python3 /scripts/weekly_backup.py

# Monthly backup on 1st day at 4:00 AM
0 4 1 * * /usr/bin/python3 /scripts/monthly_backup.py
```

Windows Task Scheduler

1. Open Task Scheduler
2. Create Basic Task
3. Set Trigger (Daily/Weekly)
4. Set Action (Start a Program)
5. Add your backup script path
6. Save and test

Best Practices

- Run backups during off-peak hours.
- Avoid overlapping backup jobs.
- Monitor scheduled jobs.
- Retry on failure.
- Notify on failure.

④ RETENTION POLICIES

Define how long backups are kept and automatically remove old data.

Retention Policy Script (example)

```python
import os, time, shutil
BACKUP_DIR = "/backups/db"
RETENTION_DAYS = 30

now = time.time()
for file in os.listdir(BACKUP_DIR):
    filepath = os.path.join(BACKUP_DIR, file)
    file_time = os.path.getmtime(filepath)
    age_days = (now - file_time) / (3600 * 24)
    if age_days > RETENTION_DAYS:
        print(f"Deleting old backup: {file}")
        os.remove(filepath)
```

TIP

Follow the 3-2-1 rule: 3 copies, 2 media types, 1 offsite copy.

Retention Strategies

- Daily backups: keep 7–14 days
- Weekly backups: keep 4–8 weeks
- Monthly backups: keep 6–12 months
- Yearly backups: keep 1–7 years

⑤ BACKUP FRAMEWORK WORKFLOW

1. Identify What to Backup
2. Automate Backup
3. Schedule Backup
4. Validate Restore
5. Apply Retention
6. Alert & Monitor

COMMON TOOLS

- AWS Backup / AWS Data Lifecycle Manager
- Velero (Kubernetes Backups)
- Restic / Borg
- Rclone
- Cron / Task Scheduler
- Cloud Storage (S3, GCS, Azure Blob)

USEFUL COMMANDS

```
pg_dump -U postgres -d appdb -F c -f backup.dump      # Backup PostgreSQL
pg_restore -U postgres -d appdb backup.dump           # Restore PostgreSQL
aws s3 cp backup.tar.gz s3://my-backup-bucket/        # Upload to S3
aws s3 sync s3://my-backup-bucket /backups/           # Download from S3
find /backups -type f -mtime +30 -delete              # Delete files older than 30 days
```

FINAL NOTE

A backup is only as good as its restore. Test regularly, automate everything, and document your recovery process.

VERIQTA

INCIDENT RESPONSE AUTOMATION

Incident response automation helps you **detect**, **investigate**, **collect evidence**, **notify teams**, and **recover** systems faster with consistency and accuracy.

REMEMBER

- Automate the repeatable.
- Document everything.
- Reduce response time.
- Preserve evidence.
- Recover safely.

① DETECT

Automatically detect anomalies and trigger alerts.

Example: Detect High CPU Usage (Linux)

```
#!/bin/bash
THRESHOLD=80
CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' |
    cut -d. -f1)
if [ "$CPU" -gt "$THRESHOLD" ]; then
   echo "ALERT: High CPU - ${CPU}%"
   ./notify.sh "High CPU" "CPU is ${CPU}% on $(hostname)"
fi
```

TIP

Use monitoring tools like Prometheus, CloudWatch, or Datadog for alerts.

② INVESTIGATE

Run automated checks and gather context.

Example: Investigate Disk Usage (Linux)

```
#!/bin/bash
echo "===== Disk Usage ====="
df -h
echo -e "\n====== Top 10 Large Directories ====="
du -ahx / | sort -hr | head -n 10
echo -e "\n====== Failed Login Attempts ====="
grep "Failed password" /var/log/auth.log | tail -n 20

echo -e "\n====== Running Processes (Top 10) ====="
ps aux --sort=-%cpu | head -n 11
```

TIP

Investigate before taking action to avoid wrong remediation.

③ COLLECT EVIDENCE

Collect logs, configs, and system state automatically.

Example: Collect Evidence Bundle (Linux)

```
#!/bin/bash
THEIDEHOP_D=$(date +%Y%m%d_%H%M%S)
OUT_DIR="/incident_evidence/$INCIDENT_ID"
mkdir -p $OUT_DIR

cp -r /var/log $OUT_DIR/
cp /etc/passwd $OUT_DIR/
ss -tulnp > $OUT_DIR/network.txt
ps aux > $OUT_DIR/processes.txt
df -h > $OUT_DIR/disk.txt
tar -czf $OUT_DIR.tar.gz $OUT_DIR
echo "Evidence collected: $OUT_DIR.tar.gz"
```

TIP

Always store evidence in a secure location with proper access control.

What to Collect

- Logs
- Configurations
- Network info
- Processes
- System info
- Screenshots (if possible)

④ NOTIFY TEAMS

Send automated alerts to the right people.

Example: Send Alert via Slack Webhook (Python)

```
import requests, json
def send_slack_alert(title, message):
    url = "https://hooks.slack.com/services/T000/B000/XXXXXXXX"
    payload = {
      "text": f"*{title}*\n{message}",
      "channel": "#incidents"
    }
    requests.post(url, data=json.dumps(payload),
                  headers={'Content-Type': 'application/json'})

# Example usage
send_slack_alert("High CPU Alert",
            "CPU usage is above 80% on server01")
```

TIP

Notify quickly and clearly. Include context and impact.

Notification Channels

Slack · Email · Telegram · PagerDuty · OpsGenie

⑤ RECOVERY WORKFLOW

Automate safe recovery and validation steps.

Example: Automated Recovery Workflow (Bash)

```
#!/bin/bash
SERVICE="nginx"
echo "1. Stopping $SERVICE"
sudo systemctl stop $SERVICE
echo "2. Restoring from backup"
sudo cp /backups/nginx.conf /etc/nginx/nginx.conf
echo "3. Starting $SERVICE"
sudo systemctl start $SERVICE
echo "4. Validating service"
curl -f http://localhost/health
if [ $? -eq 0 ]; then
      echo "Recovery successful ✅"
      ./notify.sh "Recovery Success" "$SERVICE is back online"
else
      echo "Recovery failed ❌"
      ./notify.sh "Recovery Failed" "$SERVICE failed health check"
fi
```

TIP

Always validate after recovery. Never assume systems are fully restored.

Recovery Steps

Stop Service → Restore Backup → Start Service → Validate Health → Confirm Recovery

⑥ INCIDENT RESPONSE AUTOMATION WORKFLOW

1. Detect Alert Triggered → 2. Investigate Collect Context → 3. Collect Evidence Preserve Data → 4. Notify Teams Alert People → 5. Recover Restore Systems → 6. Report Document & Review

BEST PRACTICES

- Automate repeatable tasks and scripts.
- Document every step and decision.
- Keep evidence tamper-proof and time-stamped.
- Use secure channels for notifications.
- Test your incident response workflow regularly.
- Continuously improve based on lessons learned.

COMMON TOOLS

- Prometheus / Alertmanager
- ELK / OpenSearch
- AWS CloudWatch
- TheHive / Shuffle
- PagerDuty
- Slack / Microsoft Teams
- Ansible / Rundeck / StackStorm

USEFUL COMMANDS

```
journalctl -xe
dmesg | tail -n 50
ss -tulnp
lsof -i
```

```
sudo systemctl status <service>   # Service status
curl -I <url>                     # Check endpoint
nc -zv <host> <port>              # Port connectivity
tcpdump -i any                    # Packet capture
```

FINAL NOTE

Automation does not replace judgment.
It strengthens your response, reduces human error, and ensures your systems recover faster and safer.

VERIQTA

PYTHON DESIGN BEST PRACTICES

Good Python design makes your code easier to read, maintain, test, and scale. Follow these best practices to write professional, production-ready code.

REMEMBER

- Write clean, readable code.
- Keep it simple and consistent.
- Build reusable components.
- Document your code.
- Test, refactor, and improve.

① MODULAR CODE

Break your code into small, focused modules.

Example: Using Modules

```
project/
|-- main.py
|-- config.py
|-- utils.py
|-- db.py
```

```python
# utils.py
def get_timestamp():
    import datetime
    return datetime.datetime.now()

# main.py
from utils import get_timestamp

def main():
    print(f"Started at {get_timestamp()}")

if __name__ == "__main__":
    main()
```

TIP

One module should have one clear responsibility.

Benefits

- Better organization
- Reusability
- Easier testing
- Maintainability

② REUSABLE LIBRARIES

Create reusable functions and packages.

Example: Reusable Library

```python
# mylib/helpers.py
def load_json(path):
    import json
    with open(path, 'r') as f:
        return json.load(f)

def save_json(data, path):
    import json
    with open(path, 'w') as f:
        json.dump(data, f, indent=2)

# Usage in another module
from mylib.helpers import load_json, save_json
```

TIP

Build your own utility library for common tasks.

Good Library Practices

- Small, single-purpose functions
- Write unit tests
- Avoid global variables
- Version your libraries

③ NAMING CONVENTIONS

Use clear and consistent names.

Naming Rules

- Variables/functions: snake_case
- Classes: PascalCase
- Constants: UPPER_SNAKE_CASE
- Modules: snake_case.py
- Packages: shortname/

Good Examples

- user_name
- calculate_total()
- MAX_RETRIES
- UserManager
- data_processor.py

TIP

Use names that clearly describe purpose.

Aa

Avoid

- x, y, z
- myVar1
- CONFIG
- getData()
- doStuff()
- tempFile.py

④ DOCUMENTATION

Document your code for humans and future you.

Example: Docstrings and Comments

```python
def divide(a: float, b. float) -> float:
    """
    Divide two numbers.

    Args:
        a (float): Numerator
        b (float): Denominator

    Returns:
        float: Result of division

    Raises:
        ValueError: If b is zero
    """
    if b == 0:
        raise ValueError("Denominator cannot be zero")
    return a / b
```

TIP

Docstrings make your code self-explanatory.

What to Document

- Modules
- Functions
- Parameters
- Exceptions
- Classes
- Complex logic
- Return values
- Usage examples

⑤ PYTHON DESIGN BEST PRACTICES SUMMARY

Code Structure
- Keep it simple
- One function, one task
- Avoid deep nesting

Reusability
- Write reusable functions
- Build libraries
- Avoid code duplication

Readability
- Use meaningful names
- Follow naming conventions
- Keep lines short

Maintainability
- Write clean code
- Add comments and docstrings
- Refactor regularly

Testability
- Write unit tests
- Use modular design
- Keep dependencies minimal

Scalability
- Design for change
- Follow SOLID principles
- Avoid premature optimization

USEFUL COMMANDS

```
# Run a Python file
python script.py
# Check code style (flake8)
flake8 .
# Format code (black)
black .
# Lint code (pylint)
pylint script.py
```

FINAL NOTE

Good design is not about writing more code. It is about writing better code that is clean, reusable, documented, and easy to maintain.

VERIQTA

PERFORMANCE OPTIMIZATION

Optimize your Python scripts to run faster, use less memory, and scale efficiently. Small improvements can lead to big performance gains.

REMEMBER

- Measure before you optimize.
- Optimize the right bottlenecks.
- Write efficient and clean code.
- Use built-in tools.
- Automate and scale wisely.

1 EFFICIENT LOOPS

Write loops that are fast and Pythonic.

Example: Loop Optimization

```python
# Inefficient
result = []
for i in range(len(items)):
    result.append(items[i] * 2)

# Efficient
result = [x * 2 for x in items]

# Use enumerate instead of range(len())
for idx, value in enumerate(items):
    print(idx, value)

# Use zip for parallel iteration
for name, score in zip(names, scores):
    process(name, score)
```

TIP

Use list/dict/set comprehensions and built-in functions.

{ ... }

Loop Best Practices

- Avoid unnecessary work inside loops
- Prefer built-in functions
- Use local variables for frequently accessed values
- Break early when possible

2 MEMORY USAGE

Manage memory efficiently to avoid high usage.

Example: Memory Optimization

```python
# Inefficient: loading huge file into memory
data = open("bigfile.txt").readlines()

# Efficient: process line by line
with open("bigfile.txt") as f:
    for line in f:
        process(line)

# Use generators for large datasets
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line

for line in read_large_file("bigfile.txt"):
    process(line)
```

TIP

Stream data, use generators, and avoid loading everything into memory.

Memory Best Practices

- Use generators and iterators
- Use __slots__ in classes
- Delete unused variables (del var)
- Monitor memory usage

3 PROFILING

Profile your code to find bottlenecks.

Example: Profiling with cProfile

```python
import cProfile
import pstats

def main():
    # your code here
    pass

if __name__ == "__main__":
    profiler = cProfile.Profile()
    profiler.enable()
    main()
    profiler.disable()
    stats = pstats.Stats(profiler)
    stats.sort_stats('cumulative').print_stats(10)
```

TIP

Profile first, then optimize the slowest functions.

Popular Profiling Tools

cProfile · timeit · line_profiler · memory_profiler · py-spy

4 SCALING SCRIPTS

Make your scripts handle more work and larger data.

Example: Scaling with Multiprocessing

```python
from multiprocessing import Pool, cpu_count

def work(item):
    # heavy computation
    return item * item

if __name__ == "__main__":
    items = list(range(100000))
    with Pool(processes=cpu_count()) as pool:
        results = pool.map(work, items)
    print(len(results))

# For I/O bound tasks, use ThreadPoolExecutor
# For async I/O, use asyncio
```

TIP

Use the right approach for your workload: CPU bound vs I/O bound.

Scaling Strategies

- Use multiprocessing for CPU-bound tasks
- Batch and parallelize work
- Use threading/asyncio for I/O-bound tasks
- Optimize algorithms and data structures

5 PERFORMANCE OPTIMIZATION CHECKLIST

Code Efficiency

- Use built-ins
- Avoid nested loops
- Use comprehensions
- Minimize function calls

Memory Management

- Use generators
- Stream large data
- Delete unused vars
- Monitor memory

Profiling

- Measure performance
- Find bottlenecks
- Profile regularly
- Focus on top N

Scaling

- Use concurrency
- Batch operations
- Distribute workload
- Optimize I/O

General

- Write clean code
- Document and test
- Refactor regularly
- Review performance

COMMON TOOLS

- cProfile / pstats
- timeit
- line_profiler
- memory_profiler
- py-spy
- multiprocessing
- asyncio
- psutil

USEFUL COMMANDS

```
python -m cProfile -o profile.out script.py   # Profile a script
python -m pstats profile.out                  # Analyze profile
python -m timeit -s "setup code" "stmt"       # Time small code snippets
mprof run script.py                           # Memory profile
mprof plot                                    # Plot memory usage
```

FINAL NOTE

Performance is an ongoing process.
Measure, optimize, test, and repeat.
Small improvements lead to massive impact.

VERIQTA

SECURITY BEST PRACTICES

Secure Python code protects your systems, data, and users. Follow these best practices to reduce risk, prevent vulnerabilities, and build production-ready applications.

REMEMBER

- Security is everyone's responsibility.
- Trust nothing, validate everything.
- Principle of least privilege.
- Keep secrets secret.
- Stay updated and patch regularly.

1 SECURE CODING </>

Write code that is safe, predictable, and easy to audit.

Example: Avoid Dangerous Patterns

```
# ✗ Vulnerable: Using eval()
user_input = input("Enter value: ")
result = eval(user_input)

# ☑ Safe: Use literal_eval() for literals only
import ast
user_input = input("Enter value: ")
result = ast.literal_eval(user_input)

# ☑ Use parameterized queries, not string formatting
query = "SELECT * FROM users WHERE id = %s"
cursor.execute(query, (user_id,))
```

TIP

Avoid eval(), exec(), and shell=True unless absolutely necessary.

Secure Coding Checklist

- ☑ Validate all inputs
- ☑ Use safe APIs
- ☑ Handle exceptions
- ☑ Avoid hardcoded secrets
- ☑ Log safely (no sensitive data)
- ☑ Follow coding standards

2 INPUT VALIDATION

Validate and sanitize all inputs from users and systems.

Example: Input Validation

```
import re

def is_valid_email(email):
    pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"
    return re.match(pattern, email) is not None

def is_valid_id(user_id):
    return user_id.isdigit() and len(user_id) <= 10

# Usage
email = input("Email: ")
if not is_valid_email(email):
    raise ValueError("Invalid email format")
```

TIP

Validate early, validate often, and never trust user input.

Validation Checklist

- ☑ Validate type
- ☑ Validate length
- ☑ Validate format
- ☑ Whitelist > blacklist
- ☑ Sanitize before use
- ☑ Reject unexpected input

3 DEPENDENCY MANAGEMENT

Keep dependencies updated and minimize risks.

Example: Using requirements.txt

```
# requirements.txt
requests==2.31.0
boto3==1.34.0
pydantic==2.6.1
cryptography==42.0.5

# Install
pip install -r requirements.txt

# Check vulnerabilities
pip install safety
safety check
```

TIP

Scan dependencies regularly and update promptly.

Dependency Checklist

- ☑ Pin versions
- ☑ Scan for vulnerabilities
- ☑ Remove unused packages
- ☑ Use trusted sources (PyPI)
- ☑ Review changelogs
- ☑ Update regularly

4 LEAST PRIVILEGE

Grant only the minimum permissions required.

Example: AWS IAM Least Privilege (Boto3)

```
import boto3

# Create session with limited IAM role
session = boto3.Session(profile_name="readonly_role")
s3 = session.client('s3')

# This role can only list buckets, not delete or modify
response = s3.list_buckets()
for bucket in response['Buckets']:
    print(bucket['Name'])
```

TIP

More access is more risk. Limit it.

Least Privilege Checklist

- ☑ Use least privilege IAM roles
- ☑ Limit file permissions
- ☑ Restrict network access
- ☑ Remove unnecessary access
- ☑ Review permissions regularly
- ☑ Separate duties

5 SECRETS PROTECTION

Protect secrets at every stage: storage, use, and transmission.

Example: Using Environment Variables

```
import os

db_password = os.environ.get("DB_PASSWORD")
if not db_password:
    raise RuntimeError("DB_PASSWORD not set")
```

TIP

Never hardcode secrets. Use environment variables or a secrets manager.

Secrets Protection Checklist

- ☑ Never hardcode secrets
- ☑ Use environment variables
- ☑ Use secrets manager
- ☑ Encrypt secrets at rest
- ☑ Restrict access
- ☑ Rotate secrets regularly

SECURITY BEST PRACTICES SUMMARY

Secure Coding	Input Validation	Dependencies	Least Privilege	Secrets Protection	Layered Defense
Write safe, readable, and auditable code.	Validate all inputs to prevent attacks.	Manage and update dependencies safely.	Grant minimum access required to perform tasks.	Protect secrets at rest, in transit, and in use.	Combine all practices for strong security posture.

COMMON TOOLS

- bandit (Security Linter)
- safety (Dependency Scan)
- pip-audit (Vulnerability Scan)
- AWS Secrets Manager
- HashiCorp Vault

```
pip install bandit
bandit -r .

pip install safety
safety check

pip install pip-audit
pip-audit
pip freeze > requirements.txt
```

FINAL NOTE

Security is not a feature — it is a mindset. Build securely, test continuously, and never stop improving.

VERIQTA

TESTING PYTHON SCRIPTS

Testing ensures your Python code works as expected, handles errors, and remains reliable as it grows. Good tests build confidence and prevent regressions.

REMEMBER

- Write tests before bugs.
- Test small, test often.
- Keep tests independent.
- Mock external systems.
- Automate everything.

① UNITTEST (PYTHON BUILT-IN)

The standard testing framework that comes with Python.

Example: Using unittest

```python
# mymodule.py
def add(a, b):
    return a + b

# test_mymodule.py
import unittest
from mymodule import add

class TestAdd(unittest.TestCase):
    def test_add_positive(self):
        self.assertEqual(add(2, 3), 5)
    def test_add_negative(self):
        self.assertEqual(add(-1, -1), -2)
    def test_add_zero(self):
        self.assertEqual(add(0, 0), 0)

if __name__ == '__main__':
    unittest.main()
```

TIP

Name test files as test_*.py so tools can discover them easily.

unittest Best Practices

- Use descriptive test names
- Test one thing at a time
- Keep tests independent
- Use setUp and tearDown
- Cover normal and edge cases
- Avoid hardcoding values

② PYTEST OVERVIEW

pytest

A powerful and developer-friendly testing framework.

Example: pytest basics

```python
# test_math.py
def test_add():
    assert 2 + 3 == 5

def test_sub():
    assert 5 - 2 == 3

def test_mul():
    assert 4 * 3 == 12

# Run tests
# pytest -v

# With fixtures
import pytest

@pytest.fixture
def sample_data():
    return {'name': 'VERIQTA', 'active': True}

def test_data(sample_data):
    assert sample_data['active'] is True
```

TIP

pytest auto-discovers tests, shows better errors, and supports fixtures.

pytest Best Practices

- Use simple and readable tests
- Use fixtures for setup/teardown
- Parametrize repeated tests
- Use meaningful assertions
- Keep tests fast and isolated
- Run with -v for detailed output

pytest

③ MOCKING EXTERNAL DEPENDENCIES

Mocking isolates your tests from external systems.

Example: Mocking with unittest.mock

```python
from unittest.mock import patch, Mock

def get_user_data(user_id):
    # Imagine this calls an external API
    pass

@patch('requests.get')
def test_get_user_data(mock_get):
    mock_response = Mock()
    mock_response.json.return_value = {'id':1,'name':'Ann'}
    mock_get.return_value = mock_response

    data = get_user_data(1)
    assert data['name'] == 'Ann'
```

TIP

Mock external APIs, databases, S3, or services to keep tests fast, reliable, and offline.

Mocking Best Practices

- Mock at the boundary
- Return realistic data
- Avoid over-mocking
- Reset mocks between tests
- Test behavior, not implementation
- Verify calls and interactions

④ AUTOMATION TESTING

Automate tests and run them as part of your workflow.

Example: Simple automation with pytest

```
# Run tests and generate report
pytest -v --junitxml=report.xml

# Fail the pipeline on test failure
pytest -v --maxfail=1

# Run tests in parallel
pytest -n auto

# Run specific test
pytest -k "test_add" -v
```

TIP

Automate tests in CI/CD so you catch bugs before they reach users.

Automation Best Practices

- Run tests on every commit
- Use CI/CD pipelines
- Keep tests fast and reliable
- Generate test reports
- Notify on failures
- Track test coverage

⑤ TESTING WORKFLOW

1. Write Code → 2. Write Tests → 3. Run Tests → 4. Fix Issues → 5. Re-run Tests → 6. Commit with Confidence

TOOLS IN PYTHON ECOSYSTEM

- unittest – Built-in testing framework
- pytest – Advanced testing framework
- unittest.mock – Mocking library
- coverage.py – Code coverage tool
- tox – Test across multiple environments
- nose2 – Alternative test runner

pytest coverage.py TOX

USEFUL COMMANDS

```
python -m unittest discover
pytest
pytest -v
pytest -k "test_name"
pytest --maxfail=1
pytest -n auto
pytest --cov=.
coverage html
```

TESTING CHECKLIST

- Tests are independent
- All edge cases covered
- No external calls without mocks
- Tests run locally
- CI/CD pipeline configured
- Coverage is acceptable

FINAL NOTE

Tests are not a luxury. They are the safety net that allows you to move fast without breaking things.

VERIQTA

BUILDING A DEVOPS UTILITY LIBRARY

A DevOps utility library helps you **standardize**, **reuse**, and **accelerate** automation across projects and teams.

Build once, use everywhere.

REMEMBER

- Write once, use everywhere.
- Keep it simple and reliable.
- Document every function.
- Test everything.
- Version and share safely.

1 SHARED FUNCTIONS

Common functions used across scripts and services.

Example: utils/common.py

```python
import os
import logging
from datetime import datetime

def now():
    return datetime.utcnow().isoformat() + "Z"

def env(key, default=None):
    return os.getenv(key, default)

def run(cmd):
    import subprocess
    result = subprocess.run(cmd, shell=True,
                            capture_output=True, text=True)
    if result.returncode != 0:
        raise RuntimeError(f"Command failed: {cmd}")
    return result.stdout.strip()
```

TIP

Keep shared functions small, focused, and well tested.

Shared Functions Best Practices

- Single responsibility
- Avoid global state
- Add docstrings
- Clear inputs and outputs
- Handle errors properly
- Write unit tests

2 HELPER MODULES

Group related utilities into modules.

Example: utils/aws_helpers.py

```python
import boto3
from botocore.exceptions import ClientError

session = boto3.Session()
ec2 = session.client("ec2")

def get_instance_state(instance_id):
  try:
      res = ec2.describe_instances(InstanceIds=[instance_id])
      return res["Reservations"][0]["Instances"][0]["State"]["Name"]
  except ClientError as e:
      raise RuntimeError(f"AWS error: {e}")

def list_s3_buckets():
    res = session.client('s3').list_buckets()
    return [b["Name"] for b in res["Buckets"]]
```

TIP

Organize by domain:

aws_helpers
k8s_helpers
net_helpers
fs_helpers
...

Helper Modules Best Practices

- Group by functionality
- Keep dependencies minimal
- Add logging
- Use consistent naming
- Return predictable results
- Write tests

3 REUSABLE CLASSES

Use classes when you need state, behavior, or structure.

Example: utils/logger.py

```python
import logging

class Logger:
  def __init__(self, name='veriqta', level='INFO'):
      self.logger = logging.getLogger(name)
      self.logger.setLevel(getattr(logging, level))
      if not self.logger.handlers:
          handler = logging.StreamHandler()
          formatter = logging.Formatter(
              '%(asctime)s - %(levelname)s - %(message)s'
          )
          handler.setFormatter(formatter)
          self.logger.addHandler(handler)

  def info(self, message):
      self.logger.info(message)

  def error(self, message):
      self.logger.error(message)
```

TIP

Classes help you manage complexity and maintain clean interfaces.

Class Design Best Practices

- Clear __init__()
- Keep public methods small
- Add docstrings
- Encapsulate behavior
- Use type hints
- Write tests

4 INTERNAL LIBRARIES

Package and share your library across projects.

Example: Packaging your library

Project structure:

```
devops_utils/
  |-- __init__.py
  |-- common.py
  |-- aws_helpers.py
  |-- k8s_helpers.py
  |-- logger.py
  |-- exceptions.py
  └-- setup.py
```

setup.py

```python
from setuptools import setup, find_packages

setup(
  name="devops_utils",
  version="1.0.0",
  packages=find_packages(),
  install_requires=["boto3", "kubernetes", "requests"],
  author="VERIQTA"
)
```

TIP

Version your library and publish to private PyPI or internal Git.

Internal Library Best Practices

- Semantic versioning
- Use CI/CD for testing
- Publish securely
- Automate builds
- Generate documentation
- Communicate changes

5 BUILDING & USING YOUR LIBRARY

Plan — Identify common functions and patterns.

→ **Build** — Write, test, and document your modules.

→ **Package** — Structure your library and create setup.py.

→ **Test** — Run unit tests and integration tests.

→ **Publish** — Publish to private repository or internal PyPI.

→ **Maintain** — Improve, fix, version, and your projects.

LIBRARY EXAMPLE USAGE

```python
from devops_utils.common import now, run
from devops_utils.aws_helpers import list_s3_buckets
from devops_utils.logger import Logger

log = Logger()
log.info(f"Script started at {now()}")
buckets = list_s3_buckets()
log.info(f"Found {len(buckets)} buckets")
output = run("echo VERIQTA ROCKS")
log.info(output)
```

USEFUL COMMANDS

```
# Build package
python setup.py sdist bdist_wheel
# Install locally
pip install -e .
```

```
# Install from private repo
pip install --index-url
https://pypi.mycompany.com/simple
devops_utils
```

LIBRARY CHECKLIST

- Modular and organized
- Versioned
- Well documented
- Secure dependencies
- Tested and reliable
- Easy to install and update

FINAL NOTE

A good utility library is a force multiplier. It saves time, reduces errors, and brings consistency to your automation.

VERIQTA

PRODUCTION PYTHON PROJECT STRUCTURE

A well-structured Python project is easier to maintain, test, scale, and deploy. Good structure is engineering discipline. Build it right from the beginning.

REMEMBER

- Structure for clarity.
- Configure for environments.
- Log everything.
- Document your code.
- Package for reuse.

① FOLDER ORGANIZATION

Standard project layout for production applications.

```
my_project/
|-- my_project/
    |-- __init__.py
    |-- main.py
    |-- config/
    |-- core/
    |-- services/
    |-- utils/
    |-- models/
|-- tests/
|-- scripts/
|-- config/
|-- logs/
|-- docs/
|-- .env.example
|-- .gitignore
|-- requirements.txt
|-- setup.py
|-- README.md
```

WHAT EACH FOLDER DOES

my_project/	Main Python package
config/	Configuration modules
core/	Core business logic
services/	External integrations
utils/	Helper utilities
models/	Data models / schemas
tests/	Automated tests
scripts/	Operational scripts
config/	Environment configs
logs/	Application logs
docs/	Documentation
.env.example	Environment template

TIP

Keep source code in a package, not in the root. It prevents import issues and keeps things clean.

② CONFIGURATION LAYOUT

Separate configuration by environment.

```
config/
|-- base.yaml          # default settings
|-- development.yaml   # dev overrides
|-- staging.yaml       # staging overrides
|-- production.yaml    # production overrides
|-- logging.yaml       # logging configuration

.env.example
.env.development
.env.staging
.env.production
```

Example: config/base.yaml

```yaml
app:
  name: my_project
  version: 1.0.0

server:
  host: 0.0.0.0
  port: 8000

database:
  host: localhost
  port: 5432
  pool_size: 10

logging:
  level: INFO
```

TIP

Use a config file + .env for sensitive values.

Configuration Best Practices

- Use one base config + environment overrides
- Never commit real secrets
- Load config at startup
- Validate configuration

③ LOGGING

Log consistently and with context.

Example: logging setup (logging.yaml)

```yaml
version: 1
formatters:
  standard:
    format: "%(asctime)s [%(levelname)s] %(name)s - %(message)s"
handlers:
  console:
    class: logging.StreamHandler
    level: INFO
    formatter: standard
loggers:
  my_project:
    level: INFO
    handlers: [console]
    propagate: no
)
```

TIP

Logs are your visibility in production. Make them structured, consistent, and useful.

Logging Best Practices

- Use structured logs (JSON)
- Log levels: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Include timestamps
- Log request IDs / correlation IDs
- Avoid sensitive data in logs
- Rotate and archive logs

④ DOCUMENTATION

Good docs help your future self and your team.

Example: README.md

```
# My Project
A production-ready Python application.

## Features
- Feature A
- Feature B
- Feature C

## Installation
  pip install -r requirements.txt

## Configuration
  Copy .env.example to .env and update values.

## Run
  python -m my_project.main

## Tests
  pytest
```

TIP

Document how to install, configure, run, and test your project.

Documentation Best Practices

- Keep README simple and clear
- Document environment variables
- Explain architecture and modules
- Add examples
- Keep docs updated

⑤ PACKAGING

Package your project for reuse and easy deployment.

Example: setup.py

```python
from setuptools import setup, find_packages

setup(
    name="my_project",
    version="1.0.0",
    packages=find_packages(),
    install_requires=[
        "requests",
        "pyyaml",
    ],
    entry_points={
        'console_scripts': [
            'my-project = my_project.main:main"
        ]
    },
)
```

TIP

Packaging makes your project installable, shareable, and easy to deploy.

Packaging Best Practices

- Use setup.py or pyproject.toml
- Pin dependencies
- Version your releases
- Publish to private index

USEFUL COMMANDS

```
# Install dependencies
pip install -r requirements.txt
# Run application
python -m my_project.main
# Run tests
pytest
# Build package
pip install -e .
```

⑥ PRODUCTION PROJECT CHECKLIST

- Clear folder structure
- Configuration separated
- Secrets in .env (not committed)
- Logging configured
- Error handling in place
- Tests written
- README complete
- Dependencies pinned
- Virtual environment used
- Code quality tools applied
- Documentation updated
- Package installable
- Deployment script ready
- Monitoring integrated

FINAL NOTE

A strong project structure is not just about folders. It reflects engineering discipline, maintainability, and production readiness.

VERIQTA

PRODUCTION CASE STUDY

Automating Linux Server Administration

Real world project: building a Python automation framework to manage, monitor, and maintain Linux servers at scale.

Goal: Reduce manual work, prevent issues, and improve reliability.

REMEMBER

- Automate repeatable tasks.
- Validate before changes.
- Log everything.
- Alert early.
- Keep scripts idempotent.

① REQUIREMENTS

What the automation framework must do.

- Manage multiple Linux servers via SSH
- Gather system information (CPU, RAM, Disk, Uptime)
- Check service status (nginx, sshd, docker, etc.)
- Deploy configuration updates
- Create and manage users
- Automate backups
- Health checks and alerting
- Centralized logging and reporting
- Easy to extend with new modules
- Run on schedule with minimal human intervention

INFO

Target Environment:

Ubuntu 22.04 LTS

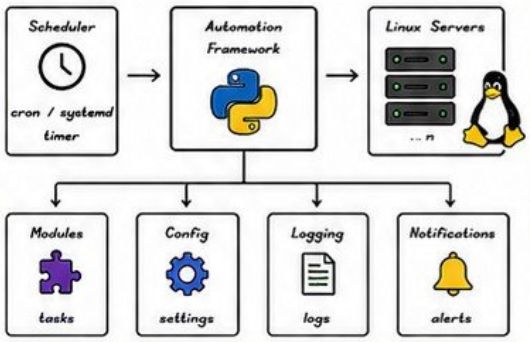
Python 3.11+

SSH Key Auth

10+ Servers

② DESIGN

High level architecture of the solution.



KEY POINTS

Modular design

Extensible

Idempotent

Observable

Secure

③ IMPLEMENTATION

Step by step implementation summary.

a) Project Structure

```
linux_admin_automation/
|-- config/
|   |-- servers.yaml
|-- modules/
|   |-- system_info.py
|   |-- service_check.py
|   |-- user_manage.py
|   |-- backup.py
|-- utils/
|   |-- ssh.py
|   |-- logger.py
|   |-- helpers.py
|-- main.py
|-- requirements.txt
```

b) Connect to Servers (SSH)

```
import paramiko

def run_command(host, user, cmd):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy
    (paramiko.AutoAddPolicy())
    client.connect(hostname=host,
    username=user, key_filename=
    '~/.ssh/id_rsa')
    stdin, stdout, stderr = client.exec_
    command(cmd)
    output = stdout.read().decode()
    client.close()
    return output
```

c) Example Module: System Info

```
from utils.ssh import run_command
def get_system_info(host, user):
cpu = run_command(host, user, "top -bn1 | grep 'Cpu(s)'")
mem = run_command(host, user, "free -h | grep Mem")
disk = run_command(host, user, "df -hT")
uptime = run_command(host, user, "uptime -p")
return {"cpu": cpu, "mem": mem, "disk": disk, "uptime": uptime}
```

d) Run Tasks in Main

```
from modules.system_info import get_system_info
import yaml, logging
with open('config/servers.yaml') as f:
    servers = yaml.safe_load(f)
for server in servers['servers']:
    info = get_system_info(server['host'],
                           server['user'])
    logging.info(f"{server['host']} -> {info}")
```

IMPLEMENTATION BEST PRACTICES

- Use SSH key authentication
- Handle exceptions and timeouts
- Keep modules single-purpose
- Validate inputs
- Log all actions
- Return structured data (JSON)

④ LESSONS LEARNED

What we learned from building and running this in production.

- **Automation saves hours every day.** Routine tasks like user creation, backups, and health checks became one-click operations.
- **Idempotent scripts prevent disasters.** Scripts can run repeatedly without causing unwanted side effects.
- **Logging and alerts are critical.** Without good logs, troubleshoting becomes very difficult.
- **Start small, then scale.** We started with 3 tasks and grew to 15+ modules over time.
- **Security first.** Keys, secrets, and least privilege keep the system safe. Never hardcode credentials.
- **Version control everything.** Code, configs, and docs in Git ensure consistency and rollback when needed.

OUTCOME

- 80% reduction in manual tasks
- Zero missed backup window
- Faster incident response
- Better visibility and reporting

SAMPLE AUTOMATION TASKS

- Collect system metrics
- df -h
- free -h
- top -bn1
- Rotate logs
- Automate backups
- Update packages
- Clean old files
- Send health reports
- Alert on failures

SAMPLE servers.yaml

```
servers:
  - host: 192.168.1.10
    user: ubuntu
  - host: 192.168.1.11
    user: ubuntu
  - host: 192.168.1.12
    user: ubuntu
```

SAMPLE CRON JOB

```
# Run automation every hour
0 * * * * /usr/bin/python3 /opt/linux_admin_automation/
main.py >> /var/log/linux_admin_automation.log 2>&1
```

USEFUL COMMANDS

```
ssh user@host
df -h
free -h
top -bn1
systemctl status nginx
journalctl -u nginx -n 50
sudo useradd -m newuser
crontab -l
```

CHECKLIST

- SSH access works
- Modules tested
- Logging enabled
- Error handling in place
- Config validated
- Cron / scheduler setup
- Alerts configured
- Documentation updated

FINAL NOTE

Automation is not about writing scripts. It is about solving real problems reliably and at scale. Keep improving, stay curious.

VERIQTA

PRODUCTION CASE STUDY

Building an Infrastructure Monitoring Tool

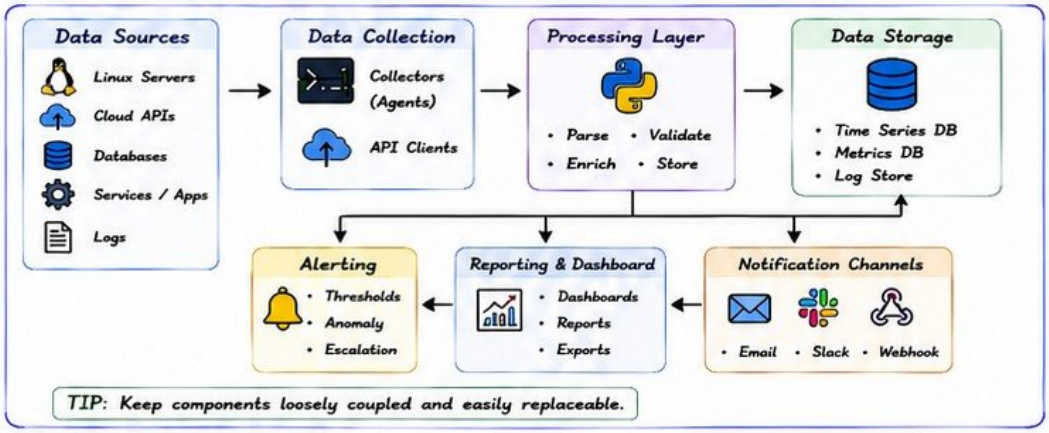
Real world project: building a Python based monitoring tool to collect metrics, trigger alerts, and generate reports for Linux servers and cloud resources.

REMEMBER

- Monitor what matters.
- Alert on real issues.
- Keep it simple and reliable.
- Automate everything.
- Document and improve.

1 ARCHITECTURE OVERVIEW

High level architecture of the monitoring tool.



TIP: Keep components loosely coupled and easily replaceable.

KEY BENEFITS

- Real-time visibility
- Faster incident response
- Historical analysis
- Capacity planning
- Better reliability

TECH STACK

- Python 3
- FastAPI / Flask
- PostgreSQL / TimescaleDB
- Prometheus (optional)
- Grafana (dashboard)
- Redis (cache / queue)
- Celery (background tasks)
- Docker

2 DATA COLLECTION

Collect metrics and logs from all sources.

```python
import paramiko, json, time

def collect_cpu(host, user, key):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(
    paramiko.AutoAddPolicy())
    client.connect(host, username=user, key_filename=key)
    stdin, stdout, _ = client.exec_command('top -bn1 | head -5')
    output = stdout.read().decode()
    client.close()
    return {'host': host, 'cpu': output, 'time':time.time()}
```

TIP

Collect only what you need. Batch requests and handle errors gracefully.

Collection Best Practices

- Use secure connections (SSH / TLS)
- Handle timeouts and failures
- Collect at the right interval
- Normalize data
- Use encryption
- Use proper intervals
- Store minimal data

3 ALERTING

Trigger alerts when thresholds are exceeded.

```python
alert_rule = {
    'name' : 'High CPU Usage',
    'condition': 'cpu > 90',
    'severity' : 'critical',
    'channels' : ['slack', 'email'],
    'cooldown' : '5m'
}
# Send alert
requests.post(WEBHOOK_URL, json=alert_rule)
```

TIP

Alert on significant issues, not on noise.

Alerting Best Practices

- Meaningful thresholds
- Avoid alert storms
- Escalate by severity
- Track acknowledgements
- Maintenance windows
- On-call notifications

4 REPORTING

Generate dashboards and reports.

TIP

Good reports answer what happened, why, and what next.

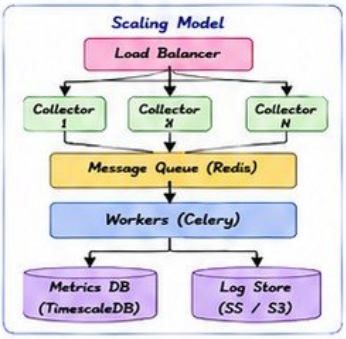
Reporting Best Practices

- Keep dashboards simple
- Export to PDF / CSV
- Schedule automated reports
- Include trends & history

5 SCALING CONSIDERATIONS

Prepare the tool for growth.

- **Horizontal scaling** — Run multiple collector workers behind a load balancer.
- **Data Storage** — Use TimescaleDB (TimescaleDB) and partition old data.
- **Queue & Async Processing** — Use Redis + Celery for background tasks.
- **High Availability** — Deploy in multiple AZs / regions.
- **Monitoring the Monitor** — Instrument the tool itself (health checks, metrics, logs).



TIP

Design for growth from day one. Measure, monitor and iterate.

LESSONS LEARNED

Automated monitoring saves hours of manual work.
Good monitoring reduces MTTR (mean time to resolve).

- Start small, validate, then expand.
- Keep scripts idempotent and well documented.
- Infrastructure monitoring is a never-ending journey.

SUCCESS METRICS

- > 99% data collection success rate
- < 5 min alert delivery time
- < 10 min incident detection time
- High dashboard adoption

FINAL NOTE

A monitoring tool is only as good as the signal it provides. Keep it simple, reliable, and valuable.

```
top -bn1 | head -n 5      # CPU snapshot
df -h                     # Disk usage
uptime                    # System uptime
curl http://localhost:8000/health   # Health check
```

PROJECT CHECKLIST

- Collectors implemented
- Alerting configured
- Dashboards created
- Logs stored securely
- Reports automated
- Documentation complete

NEXT STEP

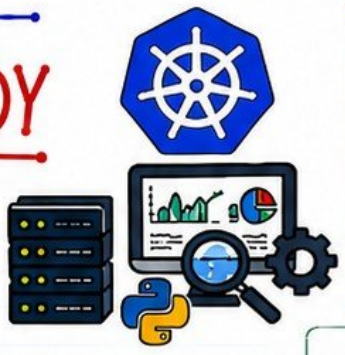
Add AI/ML Anomaly Detection in your monitoring pipeline.

VERIQTA

PRODUCTION CASE STUDY

Kubernetes Operations Automation

Real world project: building a Python automation framework to maintain Kubernetes clusters, monitor health, and automatically remediate common issues.



REMEMBER

- Automate repetitive tasks.
- Monitor continuously.
- Remediate safely.
- Log everything.
- Test automation regularly.

KEY BENEFITS

- Reduces manual operational work
- Faster issue detection and recovery
- Consistent cluster maintenance
- Improved reliability and uptime
- Lower operational cost

TECH STACK

- Python 3
- Kubernetes Python Client
- PyYAML
- Prometheus (metrics)
- Slack / Email (alerts)
- Cron / Kubernetes CronJob
- Git (version control)
- Docker (packaging)

① SOLUTION OVERVIEW

High level architecture of the automation system.

Triggers
- Cron Jobs
- Events
- Alerts
- On-demand

→ **Automation Engine**
- Python Scripts
- Kubernetes API
- Business Logic

→ **Kubernetes Cluster**
- Nodes
- Pods
- Services
- Resources

→ **Actions**
- Maintenance
- Remediation
- Scaling
- Cleanup

→ **Observability**
- Logs
- Metrics
- Alerts
- Reports

TIP: Idempotent automation + safe guards = reliable operations.

② CLUSTER MAINTENANCE

Automating routine maintenance tasks.

Example: Remove Completed Jobs

```python
from kubernetes import client, config
from datetime import datetime, timedelta

config.load_kube_config()
batch_v1 = client.BatchV1Api()
now = datetime.utcnow()
threshold = now - timedelta(days=1)

jobs = batch_v1.list_job_for_all_namespaces().items
for job in jobs:
    if job.status.succeeded:
        completed_time = job.status.completion_time
        if completed_time and completed_time < threshold:
            print(f"Deleting job {job.metadata.name}")
            batch_v1.delete_namespaced_job(job.metadata.name,
                                           job.metadata.namespace)
```

TIP

Perform maintenance during low traffic windows.

Maintenance Tasks

- Clean up completed jobs and pods
- Remove unused deployments/replicasets
- Rotate logs and prune old resources
- Update images and configurations
- Node drain and cordon automation
- Resource quota enforcement

③ HEALTH MONITORING

Continuously check cluster and workload health.

Example: Check Pod Health

```python
from kubernetes import client, config
config.load_kube_config()
core_v1 = client.CoreV1Api()
pods = core_v1.list_pod_for_all_namespaces().items

unhealthy = []
for pod in pods:
    if pod.status.phase not in ["Running"]:
        unhealthy.append(f"{pod.metadata.namespace}/"
                         f"{pod.metadata.name} - {pod.status.phase}")
if unhealthy:
    print("Unhealthy Pods Found:")
    for item in unhealthy:
        print(f" - {item}")
    # send alert
    # send_slack_alert(unhealthy)
```

TIP

Monitor everything that affects availability: pods, nodes, APIs, and resources.

Monitoring Checks

- Pod status and restarts
- Node readiness
- CPU / Memory usage
- Disk pressure
- API server availability
- Kubernetes events

④ AUTOMATED REMEDIATION

Automatically fix common issues safely.

Example: Restart CrashLoopBackOff Pods

```python
from kubernetes import client, config
config.load_kube_config()
core_v1 = client.CoreV1Api()
pods = core_v1.list_pod_for_all_namespaces().items
for pod in pods:
  for cs in (pod.status.container_statuses or []):
    if cs.state.waiting and cs.state.waiting.reason == "CrashLoopBackOff":
      print(f"Restarting {pod.metadata.namespace/{pod.metadata.name}")
      core_v1.delete_namespaced_pod(
        pod.metadata.name,
        pod.metadata.namespace,
        body=client.V1DeleteOptions(grace_period_seconds=0)
      )
```

TIP

Automation should be safe, idempotent, and have rollback plans.

Remediation Actions

- Restart failing pods
- Scale deployments automatically
- Replace unhealthy nodes
- Recreate failed jobs
- Fix stuck namespaces
- Clean failed PVC finalizers

⑤ BEST PRACTICES

Principles for safe and effective automation.

- **Start Small** — Automate one task at a time. Validate before scaling.
- **Use RBAC with Least Privilege** — Grant only the permissions required for the automation tasks.
- **Idempotent Scripts** — Run multiple times without causing side effects.
- **Add Logging and Alerts** — Log all actions and notify on failures and successes.
- **Test in Non-Production** — Always test automation in a staging cluster first.
- **Version Control Everything** — Store scripts, configs, and manifests in Git.

TIP

Good automation increases reliability. Bad automation causes outages.

OBSERVABILITY & REPORTING

- Store metrics in Prometheus
- Dashboards in Grafana
- Generate daily reports
- Export to CSV / PDF

SAMPLE CRONJOB

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cluster-maintenance
spec:
  schedule: "0 2 * * *"    # daily 2 AM
    jobTemplate:
      template: "spec:
                 "
        spec:
          containers: maintenance
            image: myrepo/maint:latest
            command: ["python", "maintenance.py"]
          restartPolicy: OnFailure
```

SAMPLE DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

PROJECT OUTCOME

- Reduced manual work by 80%
- Faster incident response
- Improved cluster stability
- Consistent operations
- Highly scalable automation

VERIQTA

PRODUCTION READINESS REVIEW

Python Automation Checklist

REMEMBER

- Automate with confidence.
- Secure everything.
- Monitor always.
- Document clearly.
- Improve continuously.

1) SECURITY CHECKLIST

- [ ] No hardcoded credentials or secrets
- [ ] Use environment variables / Secrets Manager
- [ ] Principle of least privilege (IAM)
- [ ] Validate and sanitize all inputs
- [ ] Use HTTPS and secure connections
- [ ] Handle sensitive data securely
- [ ] Keep dependencies updated
- [ ] Use virtual environments
- [ ] Enable logging without exposing secrets
- [ ] Follow OWASP security best practices

TIP: Security is not a feature, it is a requirement.

2) PERFORMANCE CHECKLIST

- [ ] Use efficient algorithms and data structures
- [ ] Avoid unnecessary loops
- [ ] Use caching where appropriate
- [ ] Optimize API calls and retries
- [ ] Use connection pooling
- [ ] Process data in batches
- [ ] Profile code for bottlenecks
- [ ] Monitor memory and CPU usage
- [ ] Use async / concurrent execution
- [ ] Reduce I/O wait time

TIP: Measure. Optimize. Repeat.

3) RELIABILITY CHECKLIST

- [ ] Implement proper error handling
- [ ] Use retries with exponential backoff
- [ ] Add timeouts for all external calls
- [ ] Validate external responses
- [ ] Use idempotent operations
- [ ] Implement circuit breaker pattern
- [ ] Log all failures with context
- [ ] Write automated tests
- [ ] Monitor and alert on failures
- [ ] Ensure graceful degradation

TIP: Design for failure. Test for failure.

4) MAINTAINABILITY CHECKLIST

- [ ] Follow clean code principles (PEP 8)
- [ ] Use meaningful names and docstrings
- [ ] Modularize code into functions / classes
- [ ] Avoid code duplication
- [ ] Write unit tests for core logic
- [ ] Use logging instead of print()
- [ ] Maintain configuration in one place
- [ ] Version control all code
- [ ] Keep dependencies in requirements.txt
- [ ] Add type hints for better readability

TIP: Easy to read code is easy to maintain.

5) PYTHON AUTOMATION CHECKLIST

- [ ] Script runs without manual intervention
- [ ] All required configurations are externalized
- [ ] Logging is clear and structured
- [ ] Errors are handled and logged
- [ ] Scripts are idempotent and safe to re-run
- [ ] Parameters are validated
- [ ] Tested in staging environment
- [ ] Necessary permissions are granted
- [ ] Documentation is complete
- [ ] Deployment process is automated

TIP: Automation should be repeatable and predictable.

6) SENIOR ENGINEER RECOMMENDATIONS

- ✓ Think in Systems → Understand the full workflow, not just the script.
- ✓ Design for Observability → Good logs, metrics, and alerts save hours of debugging.
- ✓ Automate Incrementally → Start small, validate, then scale your automation.
- ✓ Handle Edge Cases → Production breaks on the 1% you did not test.
- ✓ Document Everything → Your future self and your team will thank you.
- ✓ Continuously Improve → Refactor, optimize, and remove technical debt.
- ✓ Own Your Code → Take responsibility for reliability and outcomes.

PRODUCTION MINDSET

- Reliability over speed
- Simplicity over complexity
- Clarity over cleverness
- Monitoring over assumptions
- Automation with guardrails

"A script that works once is good.
A script that works every time in production is engineering."

NEXT LEARNING PATH

Advanced Python for Platform Engineering, Cloud Automation, and AI-driven Infrastructure Automation

Advanced Python Concepts	Async Programming & Concurrency	Cloud SDKs (boto3, azure, google-cloud)	Infrastructure as Code with Python	Kubernetes Automation (Python Client)	Event-Driven Automation	AI & ML for Operations	Platform Engineering Patterns

TIP: Keep building real projects. The best way to learn is to automate what matters.